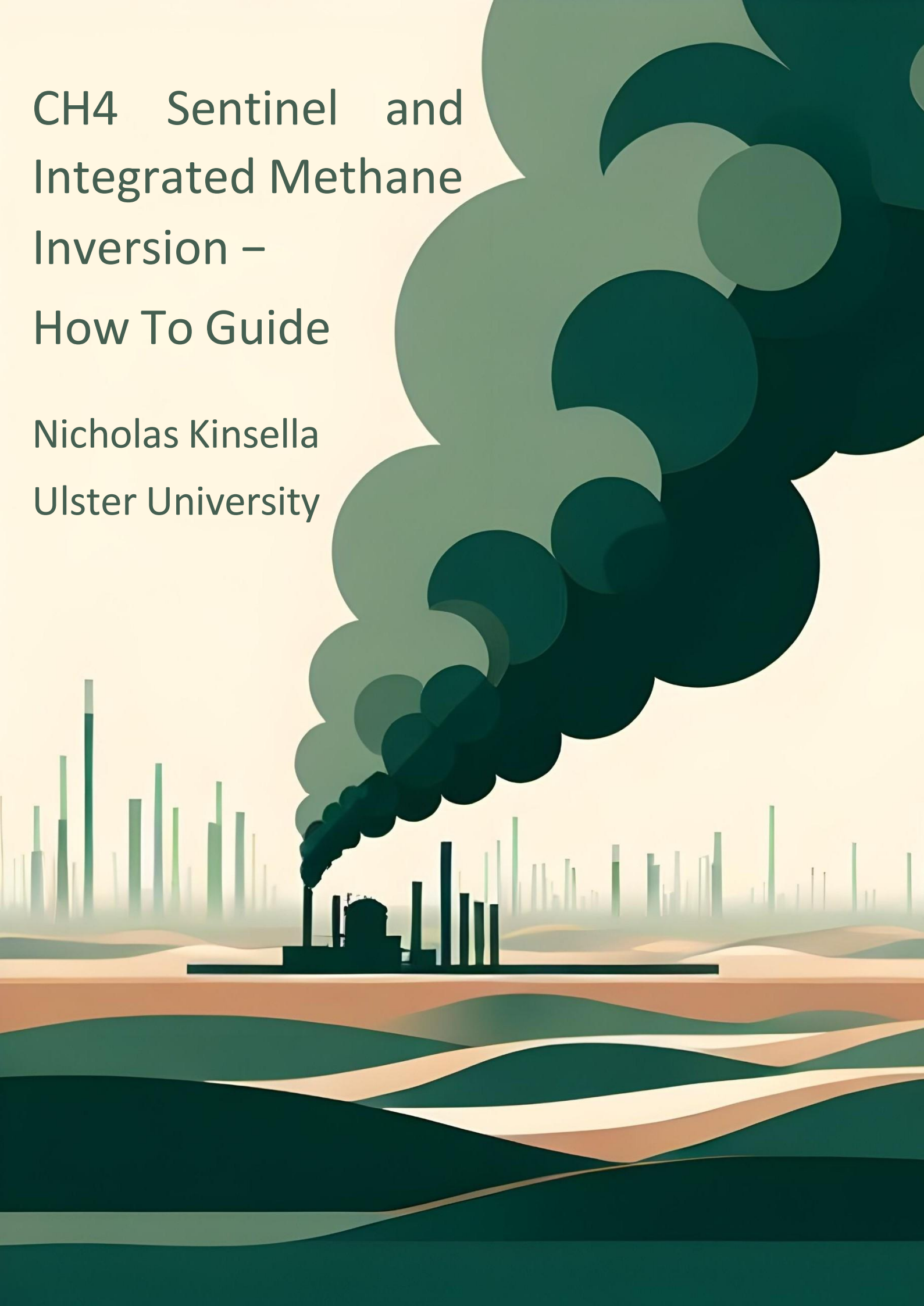
CH4 Sentinel and Integrated Methane Inversion – How To Guide

Nicholas Kinsella

Ulster University



Contents

1. Introduction

Methane (CH_4), a greenhouse gas with a warming potential 28 times greater than carbon dioxide over a 100−year period, has seen its atmospheric concentration increase over 2E0% since the industrial revolution. Despite CH_4's shorter atmospheric lifespan, it still has contributed to at least a quarter of anthropogenic warming since the Industrial Revolution (Pandey et al., 2023; Vigano et al., 2008).

Mismanaged oil and gas fields can produce significant CH_4 emissions (Jackson et al., 2020; Pandey et al., 2023) the 2021 Global Methane Pledge (GMP), was created to reduce anthropogenic emission levels by 30% of 2020 levels by 2030, with 157 countries participating (GMP, 2024; International Energy Agency, 2022; Malley et al., 2023).

Satellite missions have in recent years improved their CH_4 measurement capabilities, offering advantages over ground−based detectors (Parker et al., 2011). This guide outlines the use of two tools that can be used to monitor methane emissions.

- The Integrated Methane Inversion (IMI): An Amazon Web Services based tool for measuring methane emissions using the Sentinel−EP methane product and the GEOS−Chem model of atmospheric chemistry (Varon et al., 2022). Full details of the tool can be found here: https://imi.readthedocs.io/
- CH4 Sentinel: This creates a high−resolution map that can show super emitting CH_4 point sources for selected Algerian petrochemical fields for a specific date. It will also provide an estimation of the emission rate in t/h. This tool can be downloaded or cloned from the following Github repository:

  https://github.com/Nick-Kinsella/CH4-Sentinel

2. Integrated Methane Inversion (IMI)

2.1 Methodology

The Sentinel−5P Tropospheric Monitoring Instrument (TROPOMI) Methane product provides atmospheric CH4 concentrations in parts per billion (ppb). Its data has been shown to be in close agreement with ground−based measurements from the Total Carbon Column Observing Network (Liu et al., 2021; Lorente et al., 2021). Additional steps are required to estimate ground level CH4 emissions from these observed whole atmosphere concentrations, because CH4 can travel significant distances from its source and may be present in high altitudes over otherwise low−emission areas, and vice versa (Varon et al., 2022), resolving this requires an atmospheric transport model to be applied to the data.

The IMI (Integrated Methane Inversion) is a Python−based tool developed by Varon et al. (2022) on Amazon Web Services to estimate emissions in a selected area (fig. 2). Official bottom−up emission inventories of CH_4 are known to often be inaccurate (Dowd et al., 2023; Elkind et al., 2020; Karimi et al., 2021; Sherwin et al., 2024; Vaughn et al., 2018; Wang et al., 2024). The IMI seeks to improve on the bottom−up inventories by comparing them to top−down measurements by Sentinel−5P. Thereby providing a more accurate figure of emissions than official figures.

The inversion process starts with a "prior estimate of CH_4 emissions" based on official records such as the Global Fuel Exploitation Inventory (Scarpelli et al., 2022). Like all gasses CH_4 is affected by weather and other atmospheric influences, so next an atmospheric transport model is applied to the prior estimate data to create a model of predicted atmospheric concentrations, that should be seen by Sentinel−5P. The prediction is then compared to Sentenel−5P's real observations. The "inversion" involves adjusting the predicted concentrations so that they better match the observed concentrations. This results in a "posterior estimate" which should be a better reflection of reality.

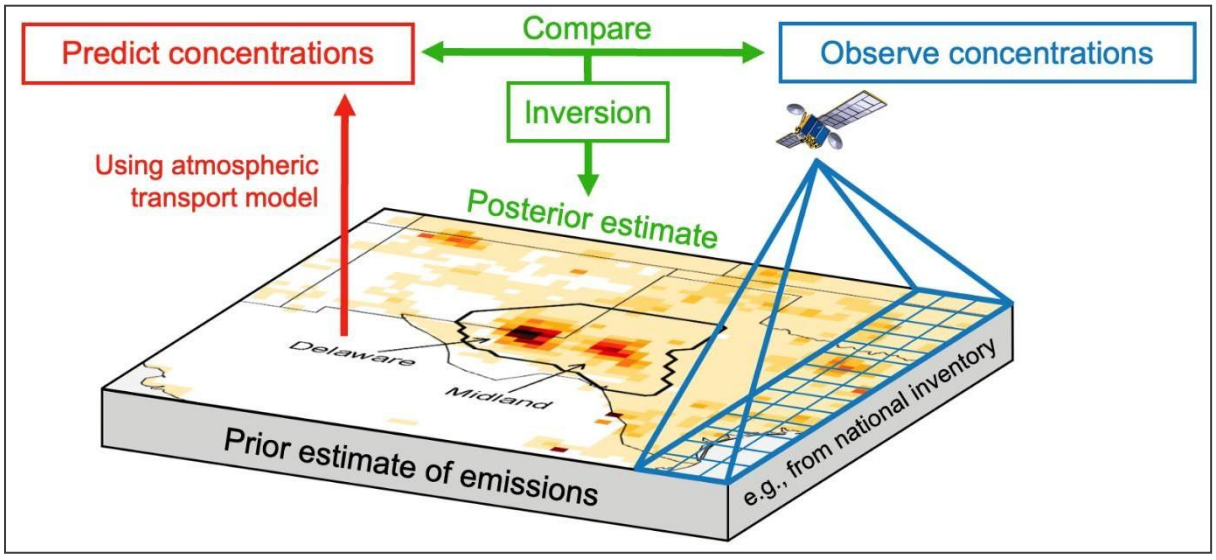


Figure 2: IMI processing step (Integrated Methane Inversion, 2024).

2.2 Setup

The following steps will show you how to setup a AWS account and run the IMI tool

2.2.1 Creating an AWS account

Creating an AWS account requires providing some personal and bank card information, as it is not a free service. The cost of running the IMI will be provided as an estimate prior to you running the full analysis. Open the following link in a browser http://aws.amazon.com/ and click on create an AWS account (fig. 3). Thereafter, follow the prompts.

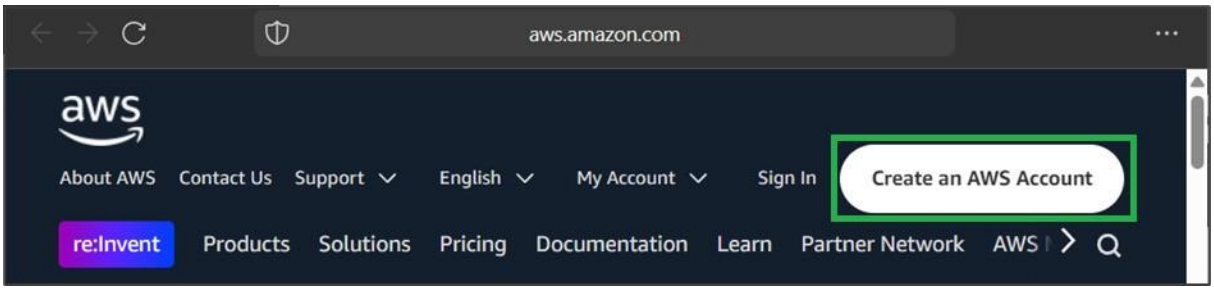


Figure 3: AWS sign in and create account landing page.

2.2.2 Add Amazon Simple Storage Service (S3) user permissions

This is an optional step if you wish to store your results long term. Firstly navigate to the AWS management console by clicking on the "Sign In to the Console" button or by using the "My Account" dropdown (fig. 4).

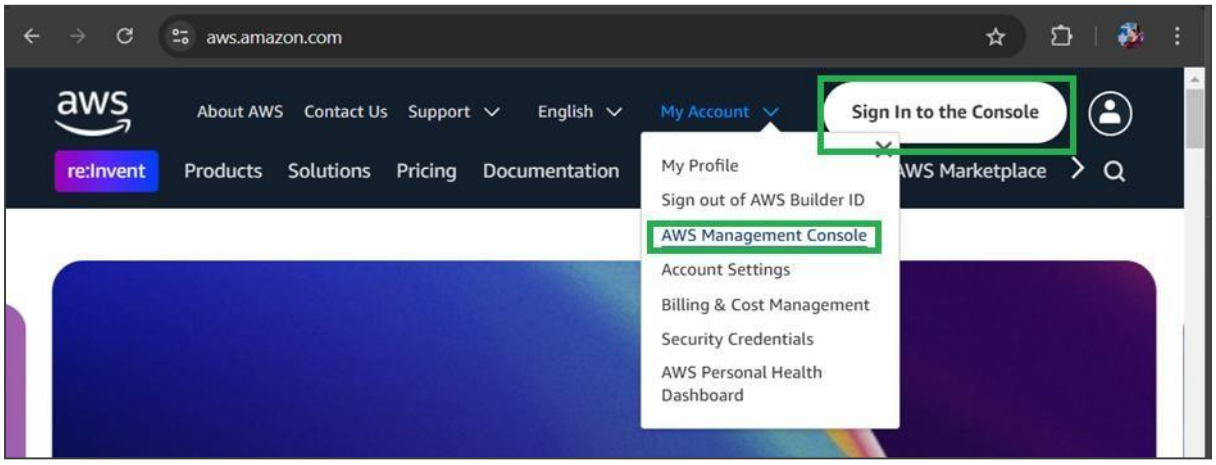


Figure 4: Sign in to my Console access on AWS landing page.

Once on the management console page, click the magnifying glass (fig. 5)

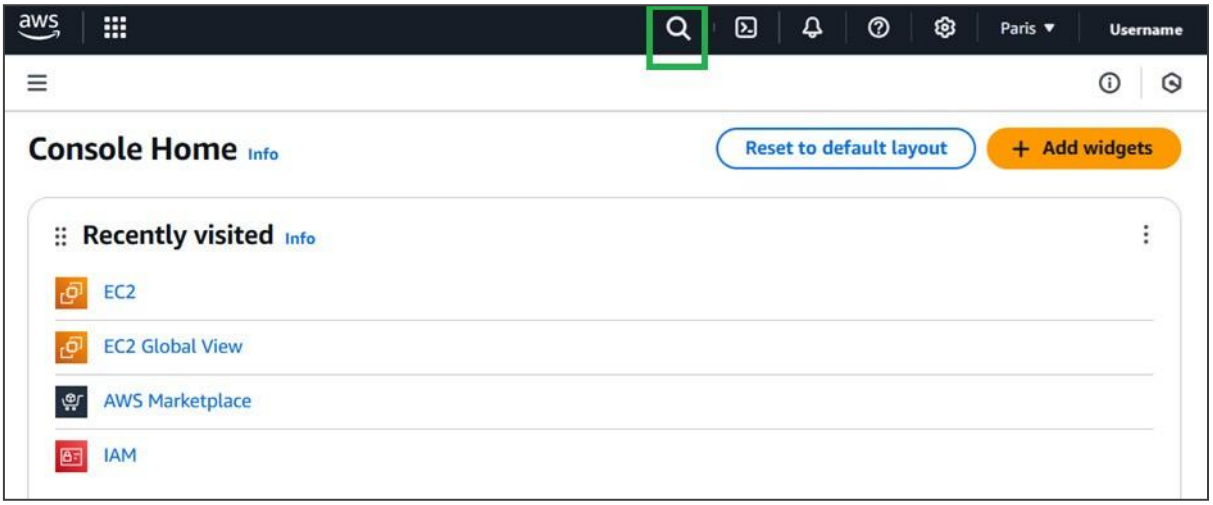


Figure 5: Location of search function in AWS console.

Then search for "IAM" (fig. 6).

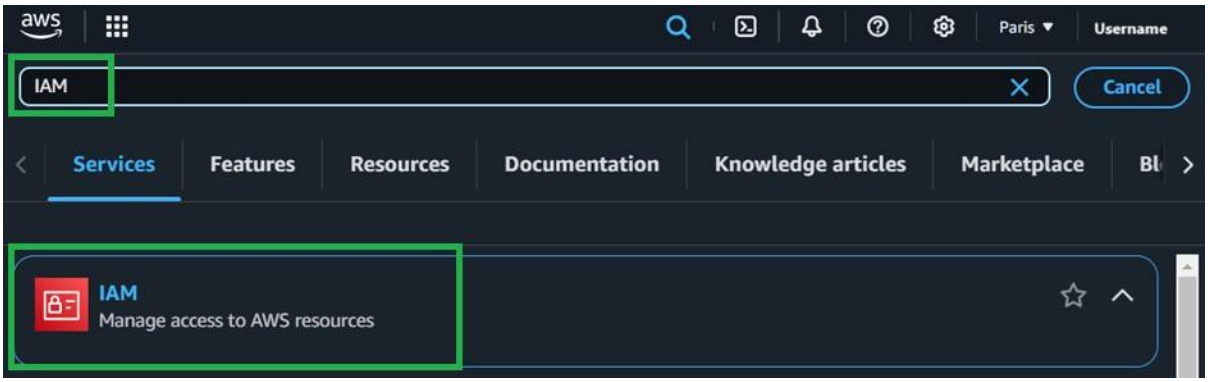


Figure 6: search function finding the AWS resource management access.

Under the "Access management" sidebar, click on Roles and then "Create role" (fig. 7)

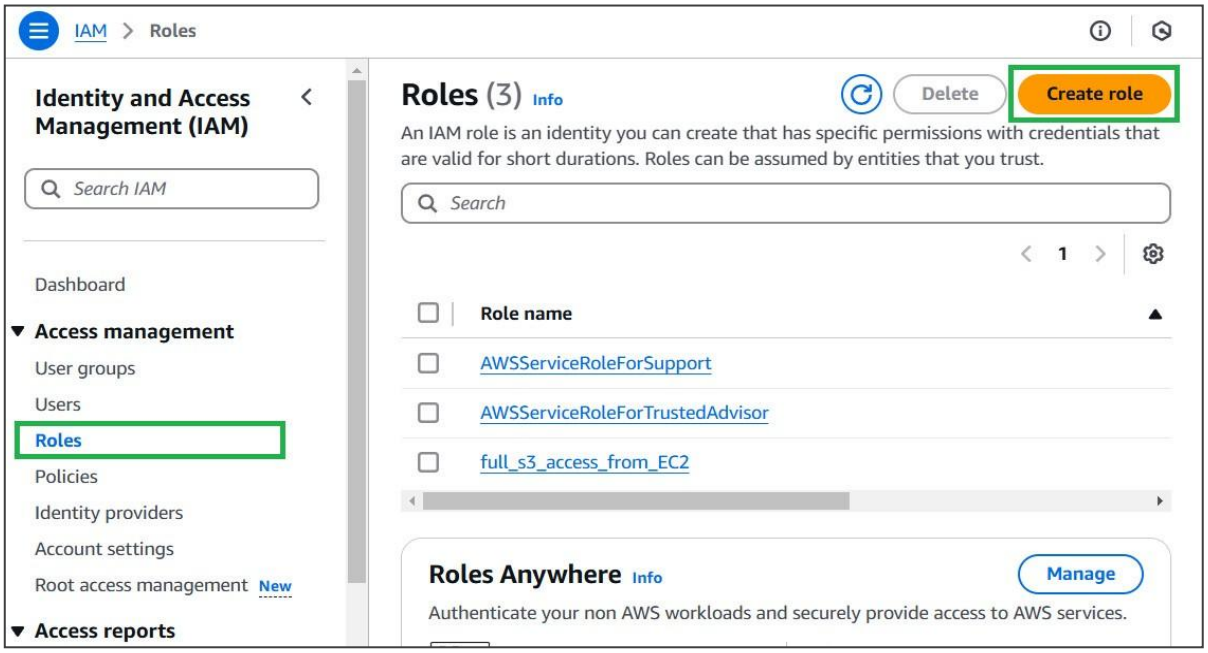


Figure 7: create role button location

In the “Use Case”, select EC2 and then click “next” (fig. 8).

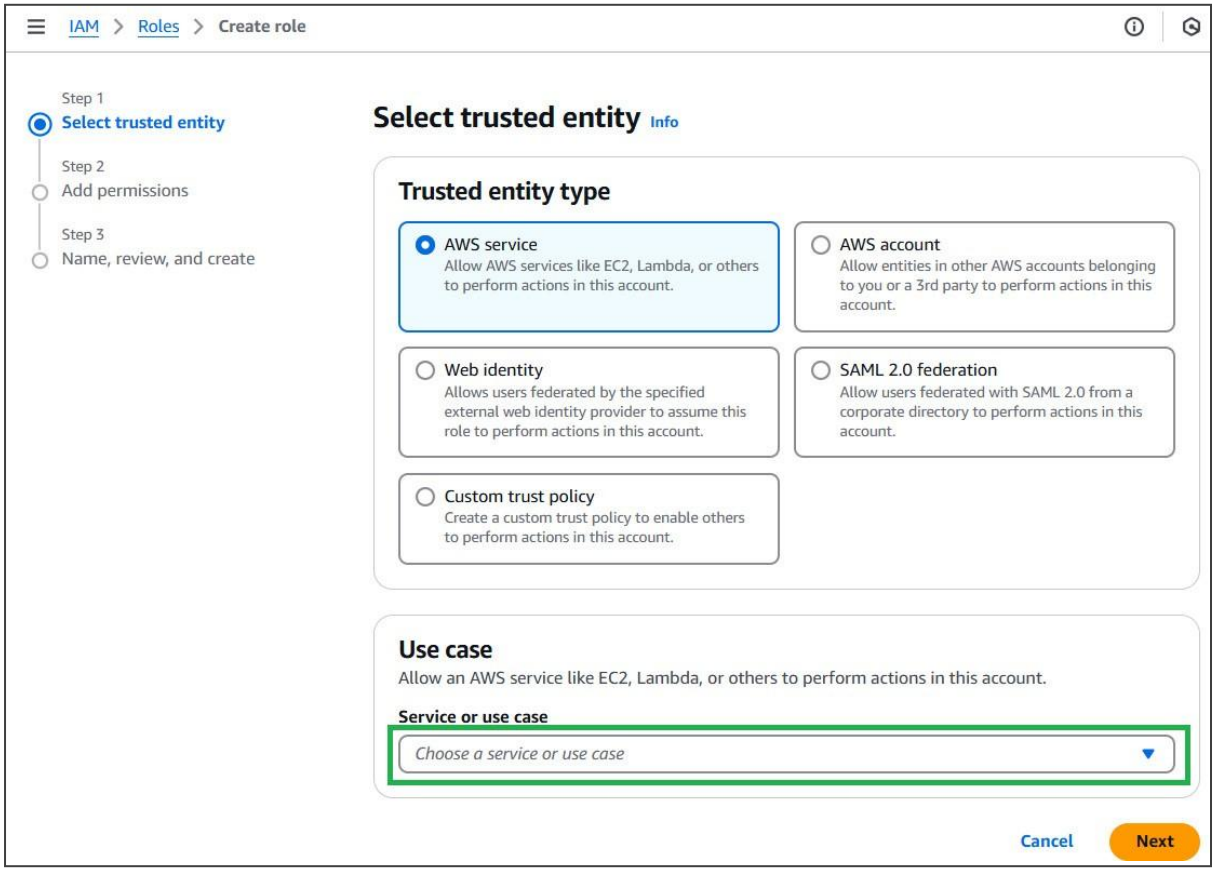


Figure 8: Use case selection location.

In the “Add permissions” screen type S3 in the search box and select AmazonS3FullAccess (fig. 9). Click next.

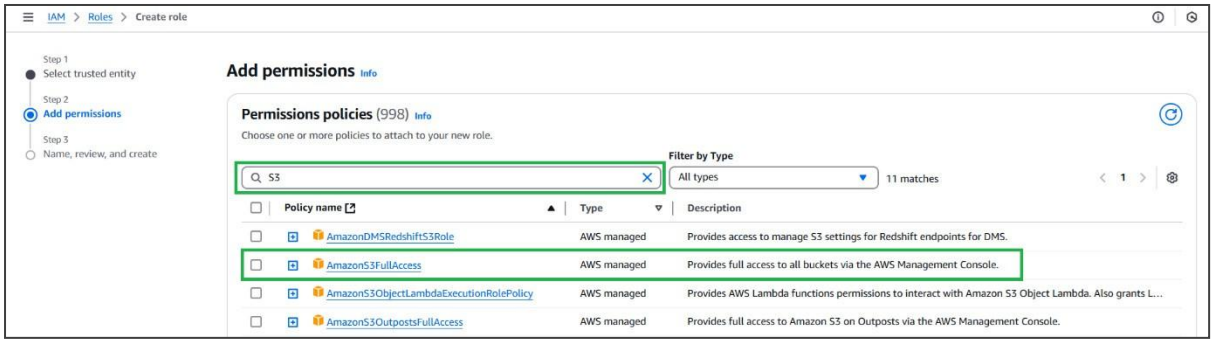


Figure 9: Use case selection location.

Lastly, choose a descriptive name for the role like “full_S3_access_from_EC2”. Click “create role” at the bottom of the page (fig. 10).

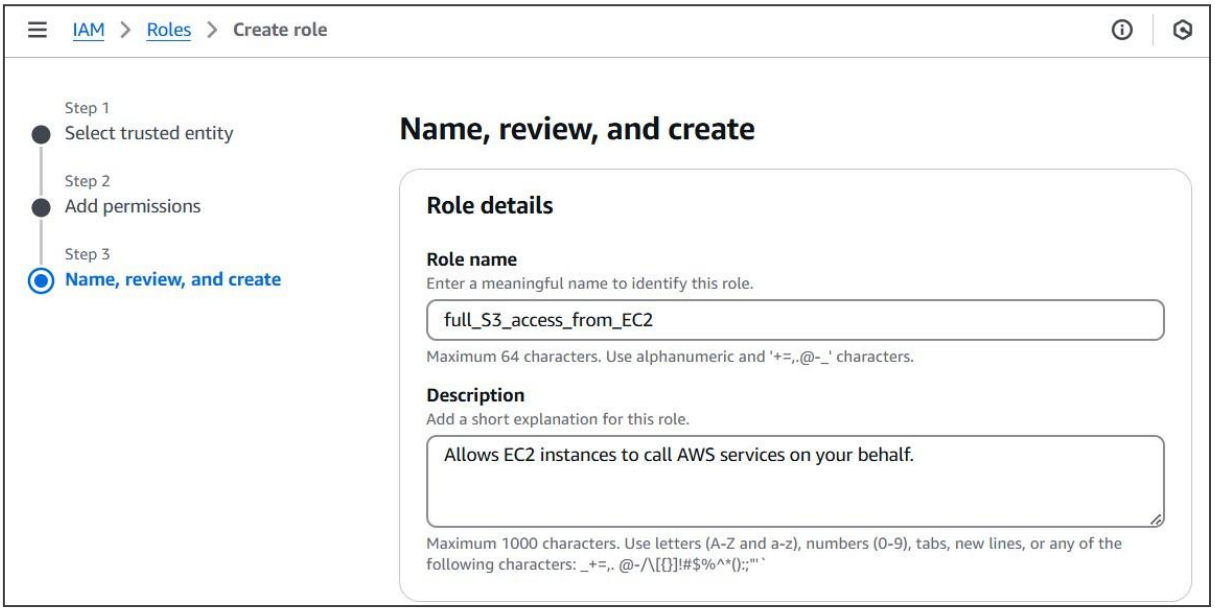


Figure 10: Create roll naming screen.

Now a new IAM role is created.

2.2.3 Launching an IMI instance

Navigate to https://aws.amazon.com/marketplace/pp/prodview-hkuxx4h2vpjba or search for "Integrated Methane Inversion". You should find the page shown in figure 11. Click "view purchase options"

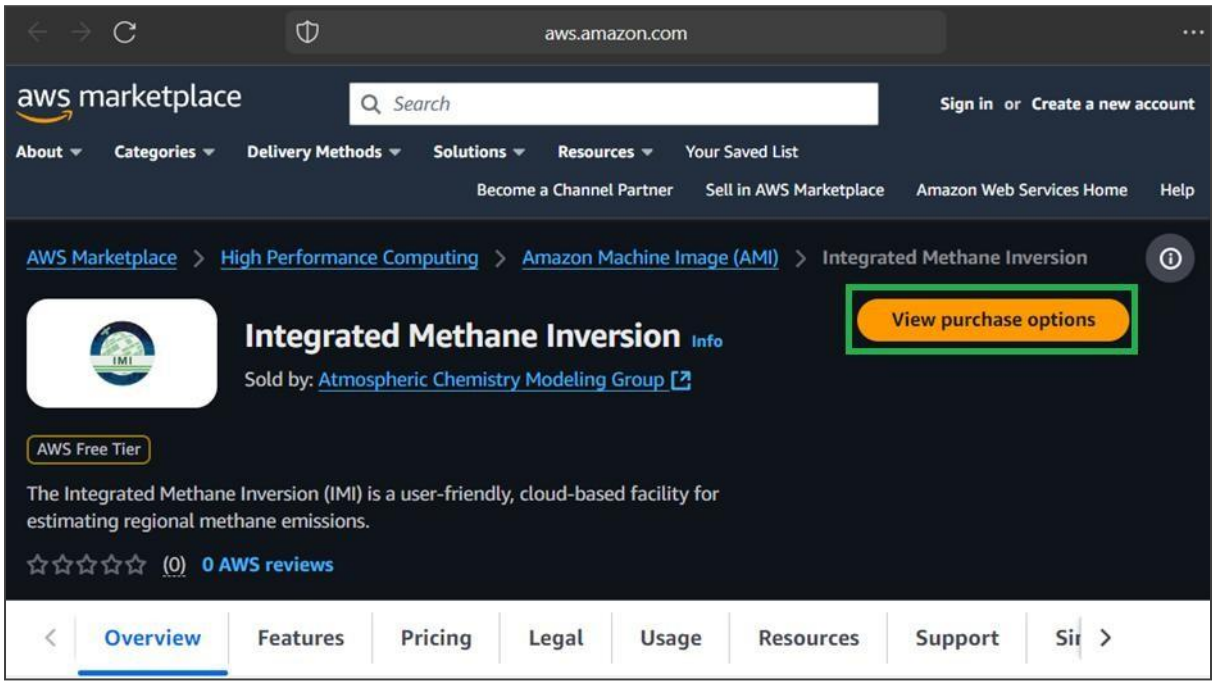


Figure 11: IMI landing page and purchase options button.

Next click "continue to configuration" (fig. 12).

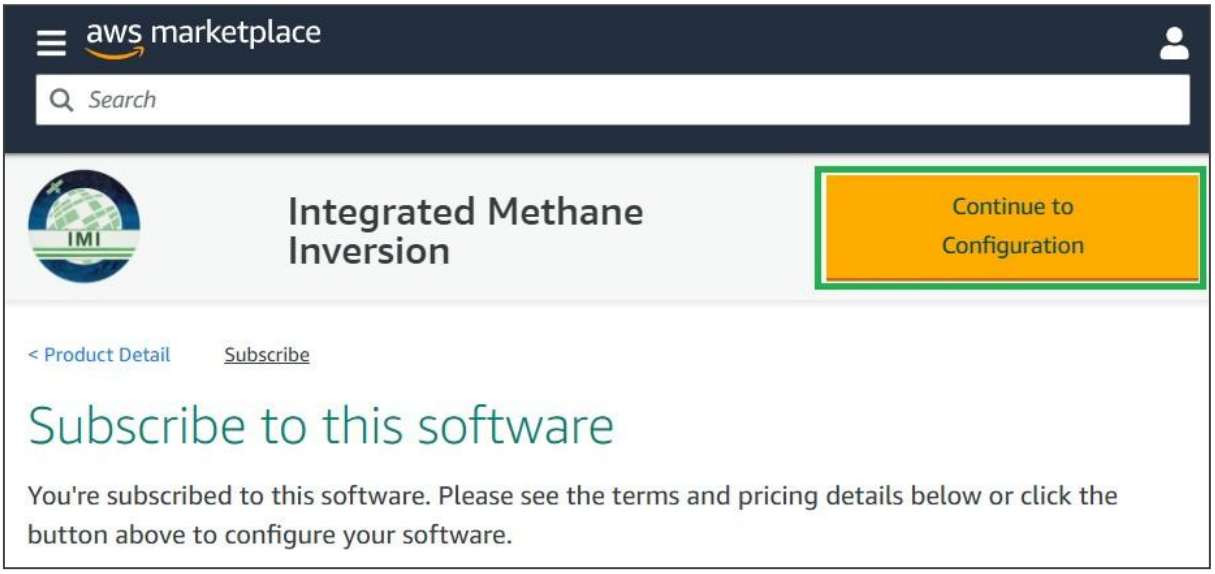


Figure 12: Subscription page with continue to configuration button.

Choose your preferred region and IMI version, then click *"Continue to Launch."* (fig. 13) Opting for a region closer to your physical location can enhance network connectivity but may lead to higher costs compared to selecting the region where GEOS−Chem data is hosted (us−east−1, N. Virginia).

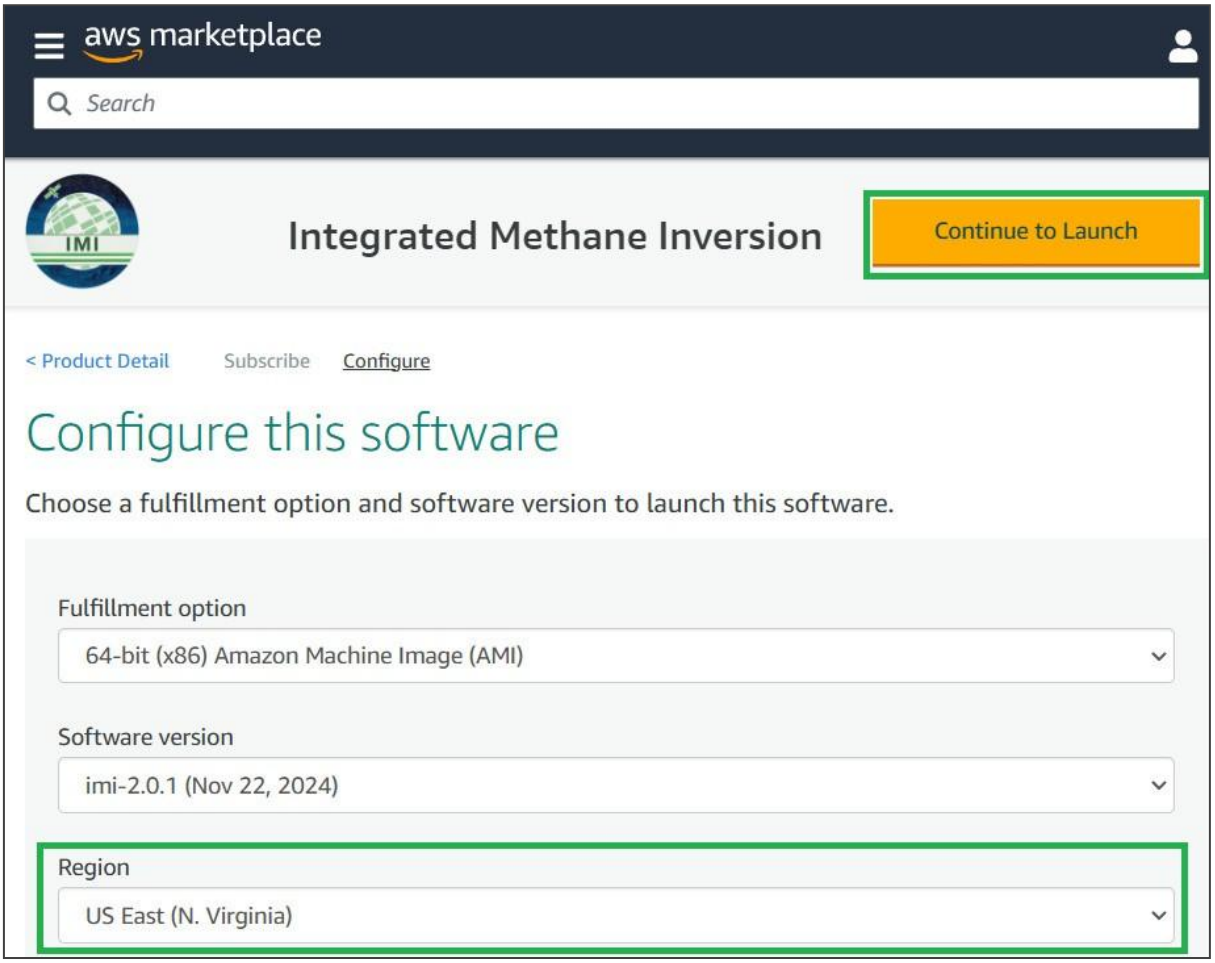


Figure 13: region configuration page and "continue to launch button".

In the "Launch this software" page, in the choose action prompt, select "Launch through EC2" and then click "Launch" (fig. 14).

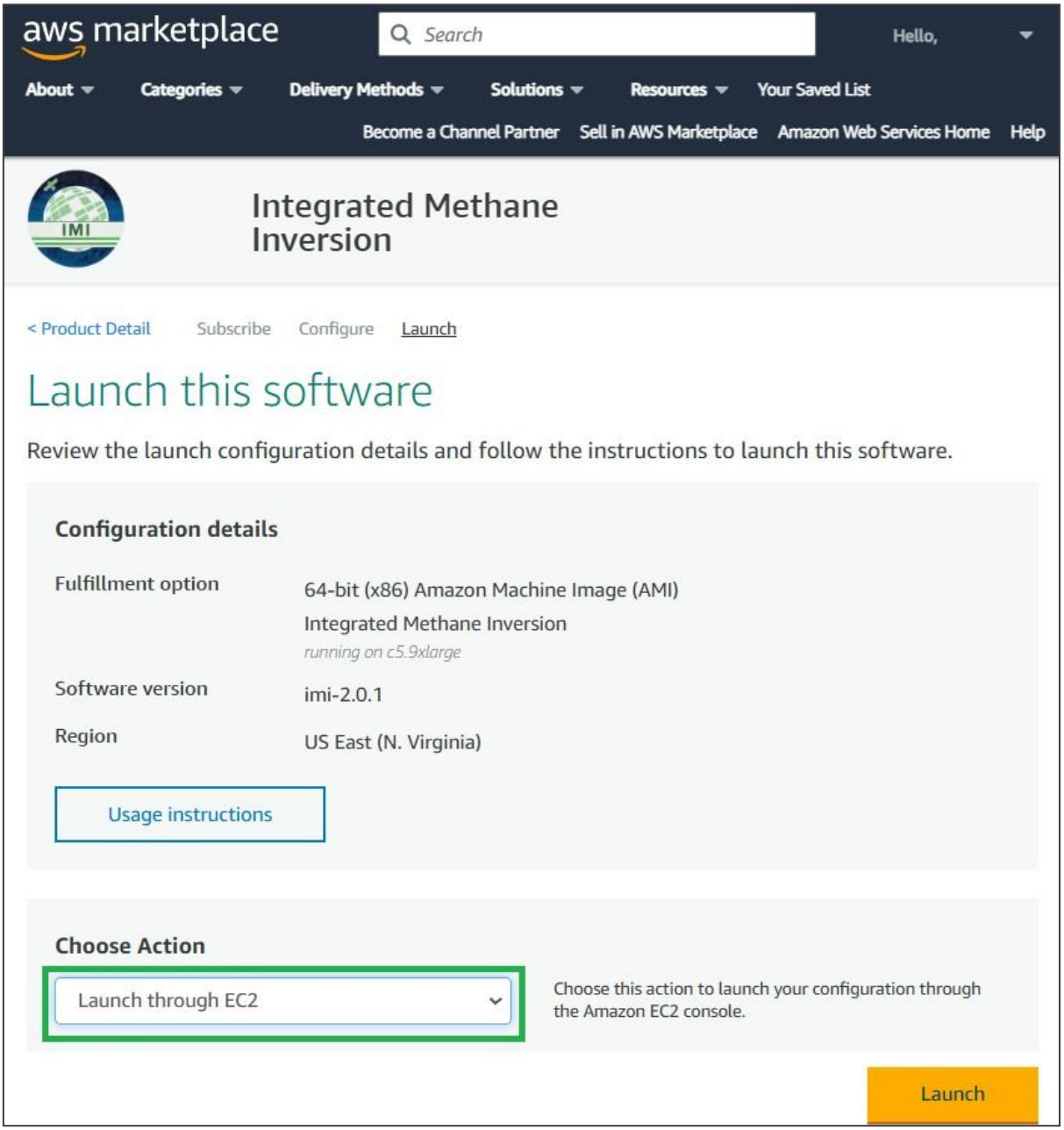


Figure 14: Launch software page.

Now it's time to determine the name of your instance and the hardware to run the IMI. The primary considerations are the number of CPUs and the amount of RAM. In the "Instance Type" section, you can choose from a wide range of options. The IMI will perform faster with a higher number of CPUs. The cE.9xlarge instance type is the recommended setting. It offers 36 CPU cores and 72GB of RAM. However, you can choose a different instance type with more or fewer cores and memory based on your specific needs (fig. 15).

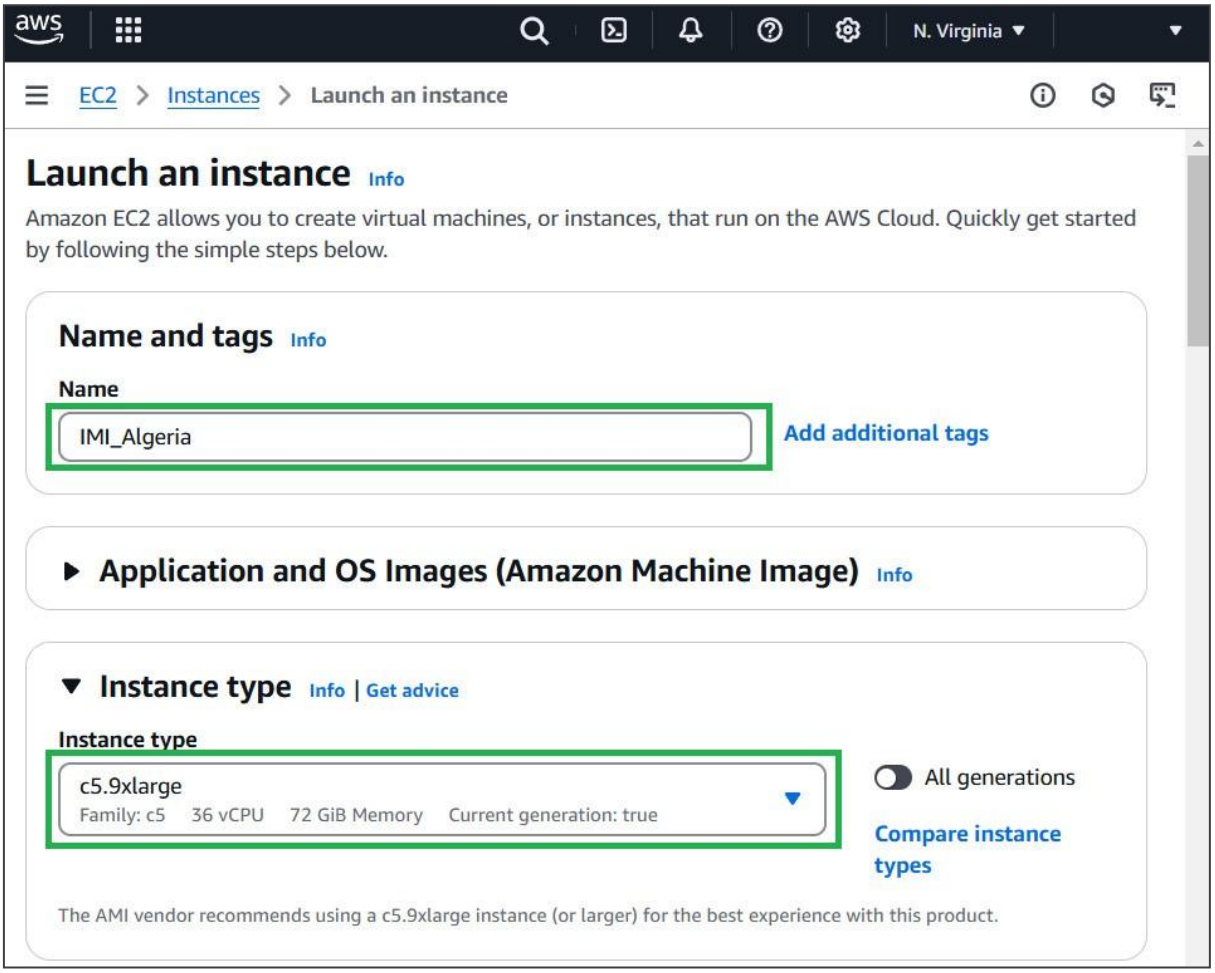


Figure 15: Hardware selection page.

In the next step, you'll create a new SSH key pair or select an existing one. This key pair acts as the password for accessing your server via SSH. To create a new key pair, click “Create new key pair”. In the dialog box, assign a name to your key pair (e.g., imi_testing) and click “Create key pair”. Once created, the key pair will be available for future use, and you can simply select it from the dropdown menu (fig. 16).

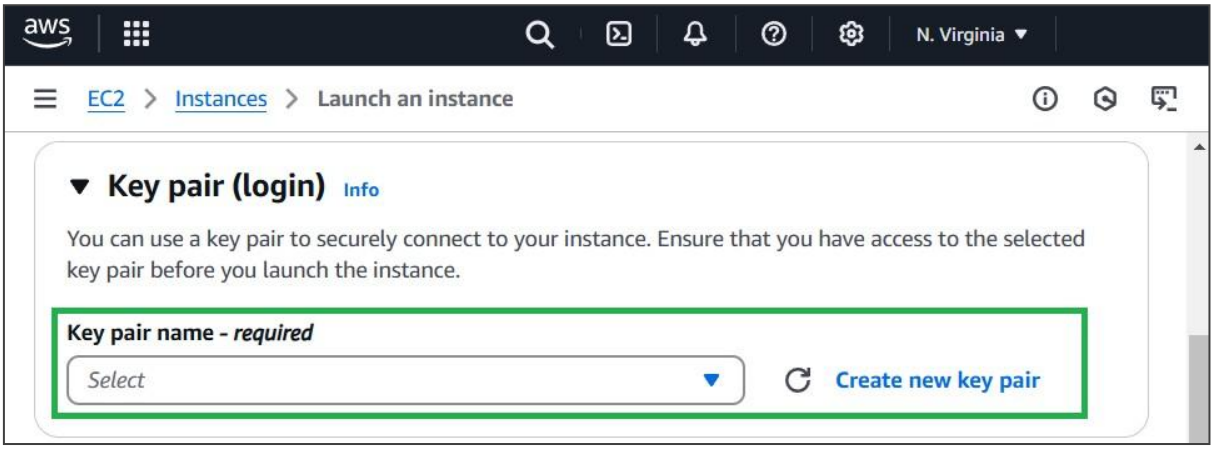


Figure 16: keypair selection.

If you are creating a key pair, use the settings shown in figure 17, choosing an appropriate name. Click "create key pair" once you have finished, saving it in a memorable directory that will be used later.

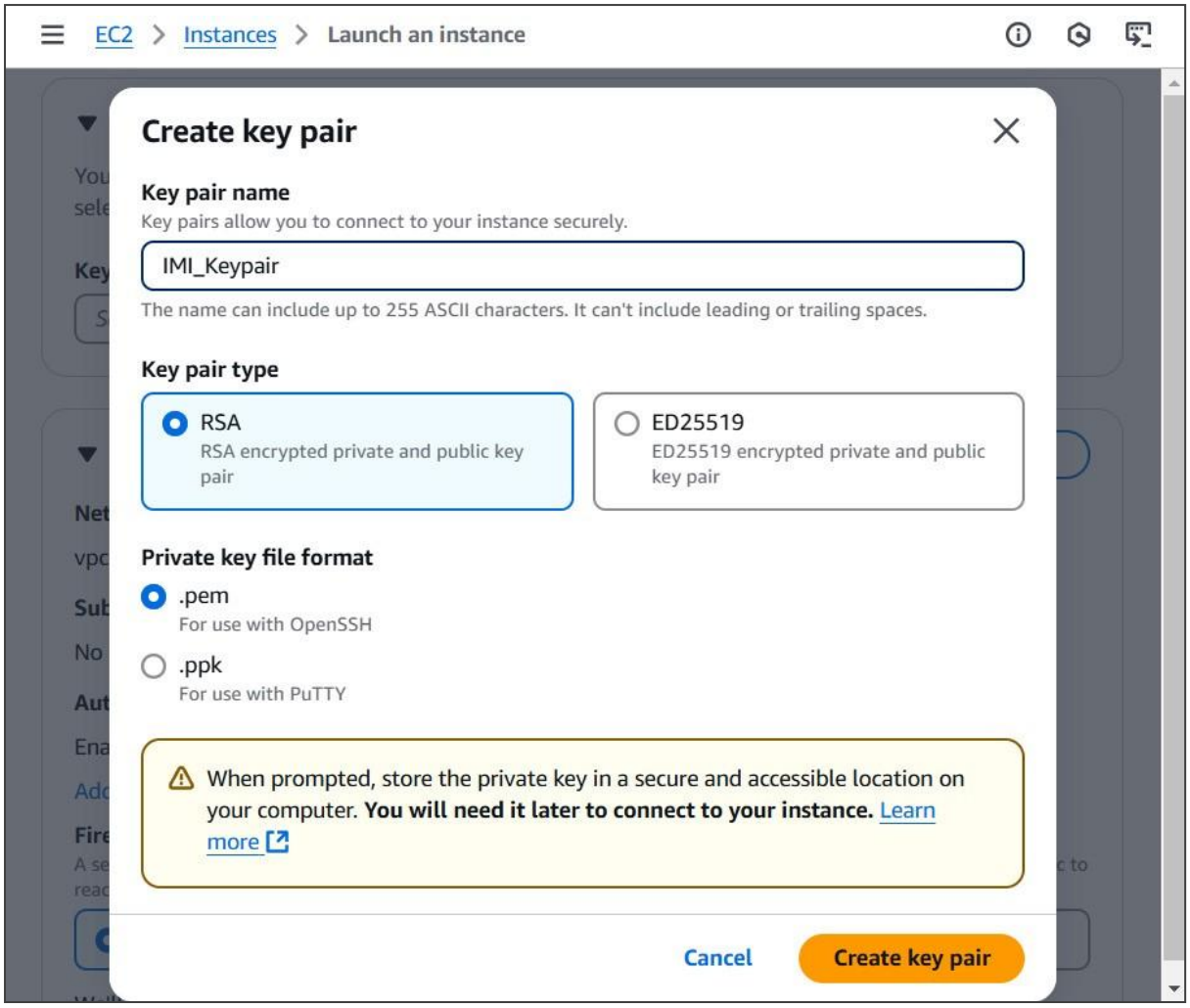


Figure 17: keypair setup.

Proceed to the "Configure Storage" section (fig.18), where you will select the size of the storage volume. Storage costs are around USD $100 per month per TB of space. The amount of storage you choose will

depend on how long you intend to analyse an area. According to an example provided by the IMI team, 100 Gibibytes is sufficient for a 1–week inversion of an area of around 10,000km^2, whereas 5 terabytes is enough for 1 year. Also, 10,000km^2 is the minimum recommended area of analysis.

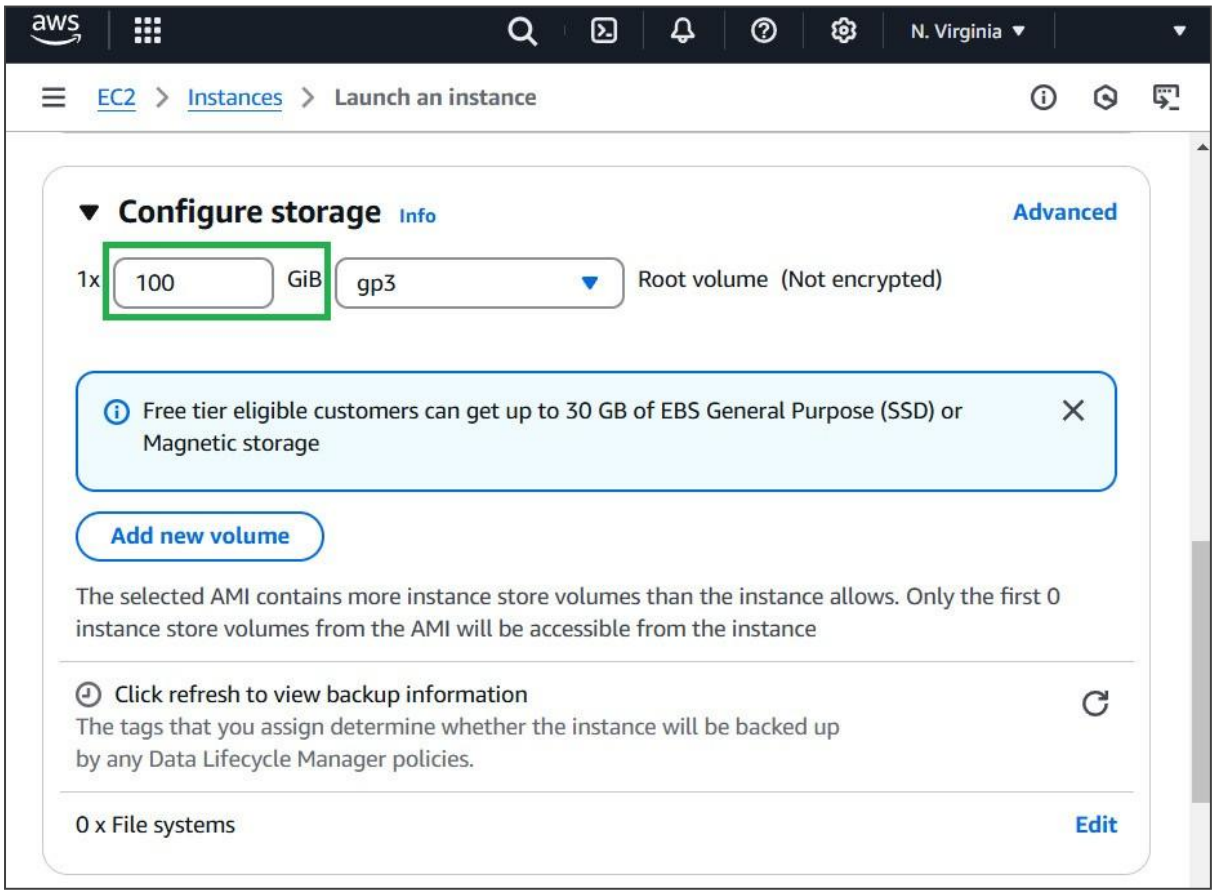


Figure 18: storage configuration section.

Next, open the "Advanced Details" section (fig. 19) and choose the IAM role you created in step 3 from the "IAM Instance Profile" dropdown. This will grant your EC2 instance access to S3 for uploading output data. You can leave all other configuration settings in the "Advanced Details" section as the default values.

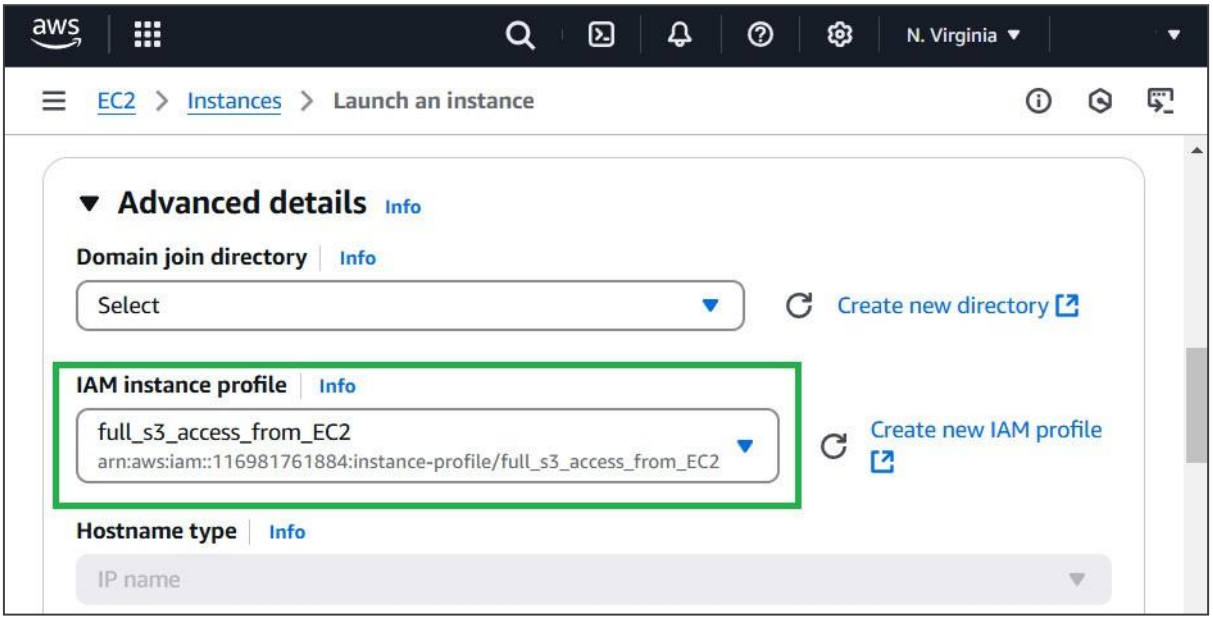


Figure 19: advanced details section.

Finally click the launch instance button in the section showed in figure 20.

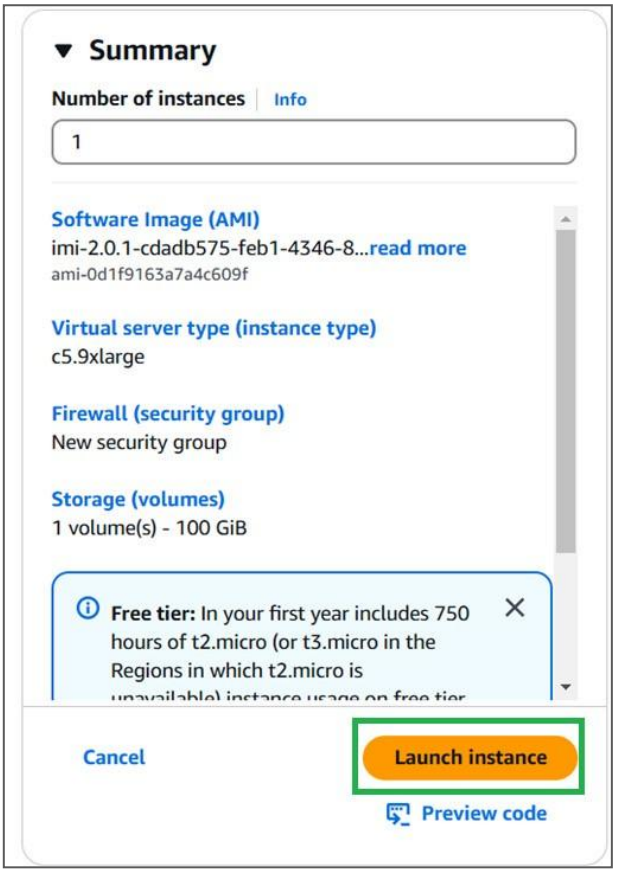


Figure 20: Summary section with "launch instance" button.

2.2.4 Connecting to instance

To connect to the instance and run the IMI, a program called Git will need to be downloaded and used. You can download Git from the following URL: https://git-scm.com/downloads/win, choosing the appropriate version of the software for your system (fig. 21).

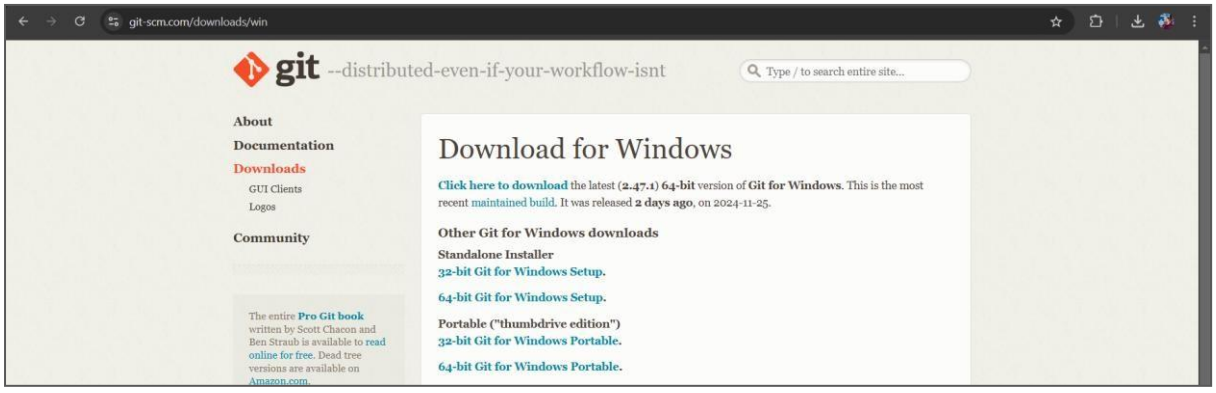


Figure 21: download page for Git.

Once downloaded, run and follow the instructions on the installation wizard (fig. 22).

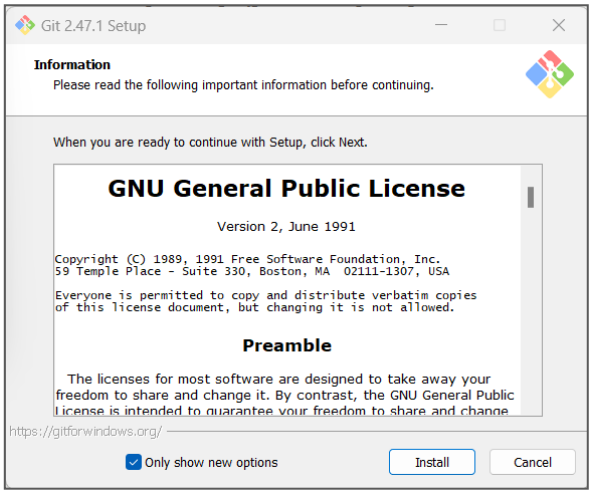


Figure 22: Git installation wizard.

Once completed, tick the box that says launch Git Bash (fig. 23). We will come back to it shortly.

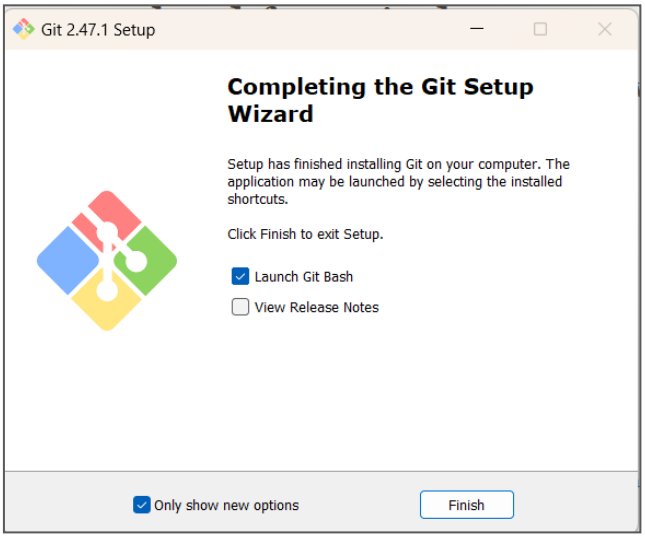


Figure 23: Git installation wizard.

Returning to the AWS management console, the first time you launch an instance it should be running already (fig. 24).

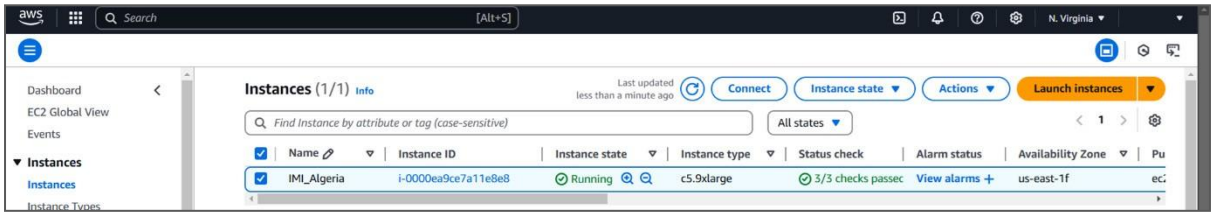


Figure 24: AWS management console Instance dashboard.

If it isn't running then select the instance using the tick box an then under "instance state" select "start instance" (fig. 25).

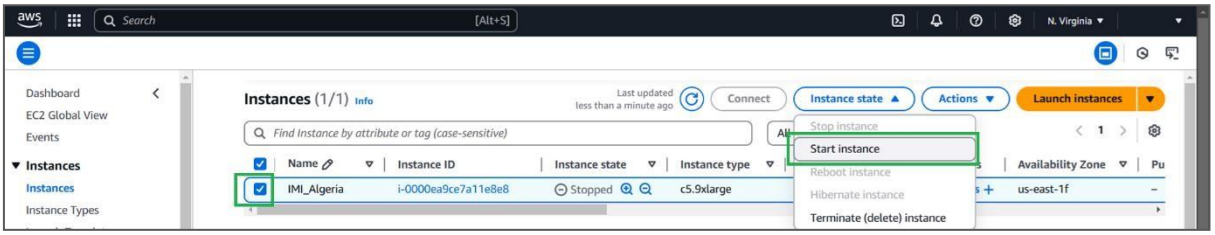


Figure 25: start instance button.

Once running, with the instance selected, select "actions" and then "connect" (fig. 26)

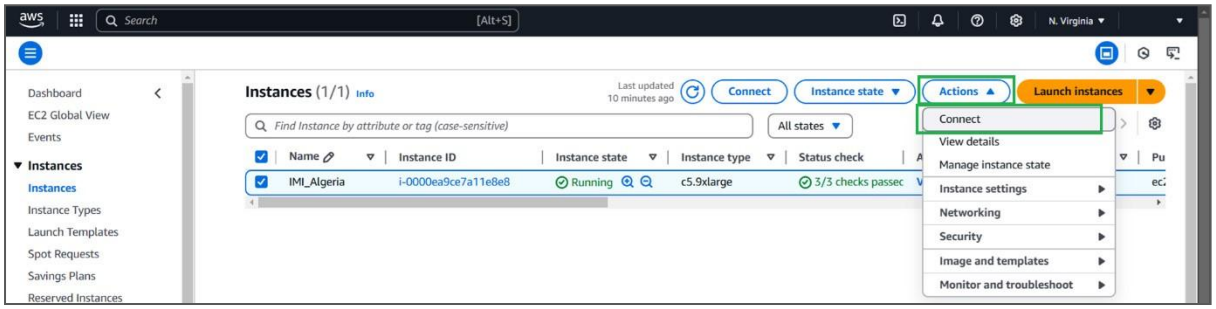


Figure 26: connect button.

In the connect to instance page, you have a list of instructions to follow using the Git Bash terminal you installed (fig. 27).

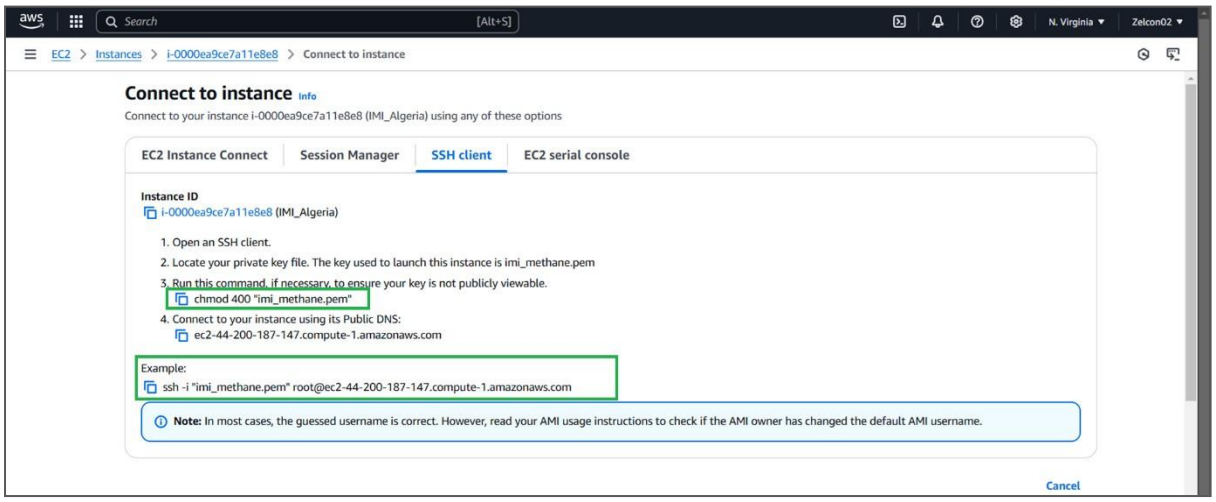


Figure 27: code to prevent keypair from being publicly visible and ssh connection command.

Change directory to where you keep your keypair by typing cd <path to your keypair folder> (fig. 28)

Figure 28: Gitbash console showing path to keypair folder.

Next you are going to enter in the command that ensures your keypair is not publicly viewable (fig. 29).

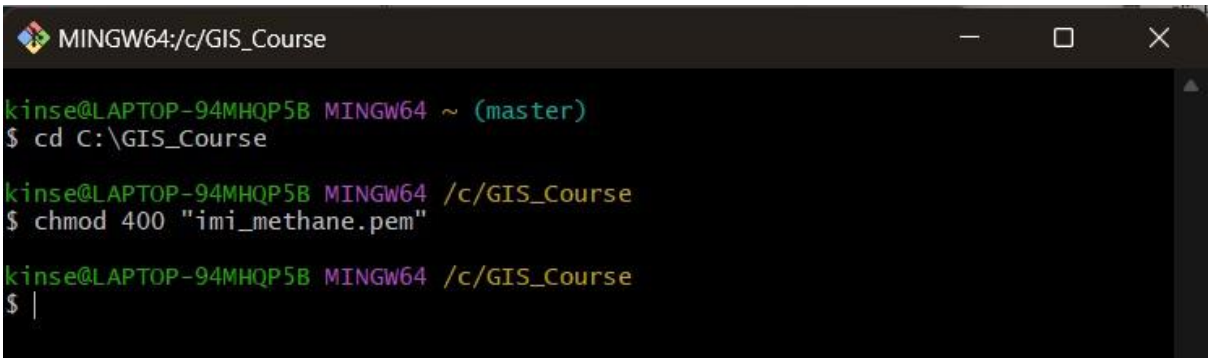


Figure 29: hiding your keypair.

Finally you are going to take the long "example" code (at the bottom of fig. 27) and change the word "root" to "ubuntu".

ssh –i "imi_methane.pem" root@ec2–44–200–187–147.compute–1.amazonaws.com

ssh –i "imi_methane.pem" ubuntu@ec2–44–200–187–147.compute–1.amazonaws.com

Then paste it into the Git Bash terminal using the right click function of the mouse. Run the code and answer "yes" to the prompt, "are you sure you want to continue connecting?" (fig. 30). Once this has run, you should receive a "welcome to ubuntu" message.

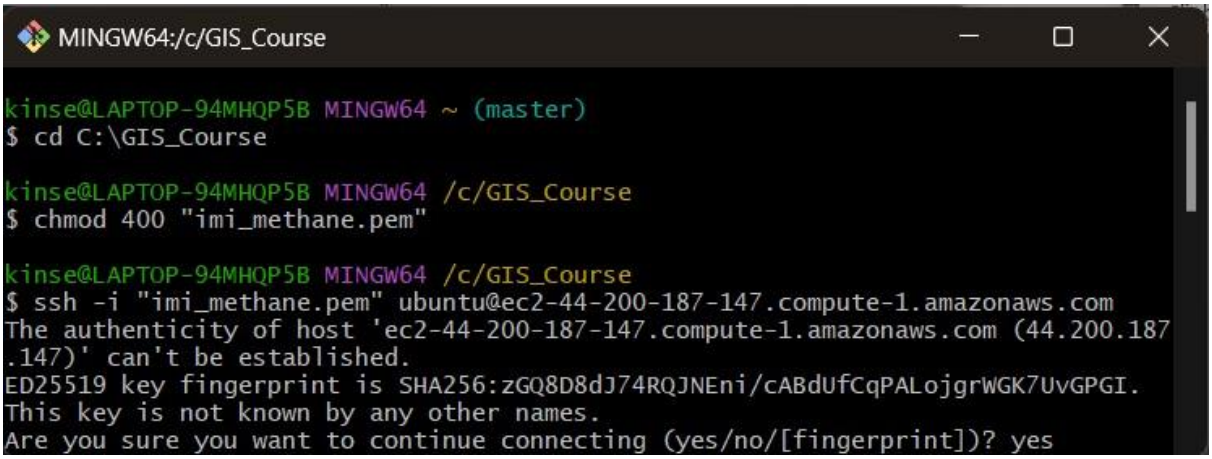


Figure 30: ssh connection to your instance.

2.3 Configuring and running the IMI Preview

In the Git Bash terminal, navigate to the IMI setup directory by typing:

cd ~/integrated_methane_inversion

Next open the inversion configuration file by typing the following:

emacs config.yml

The following screen should appear after a pause of a few moments (fig. 31)

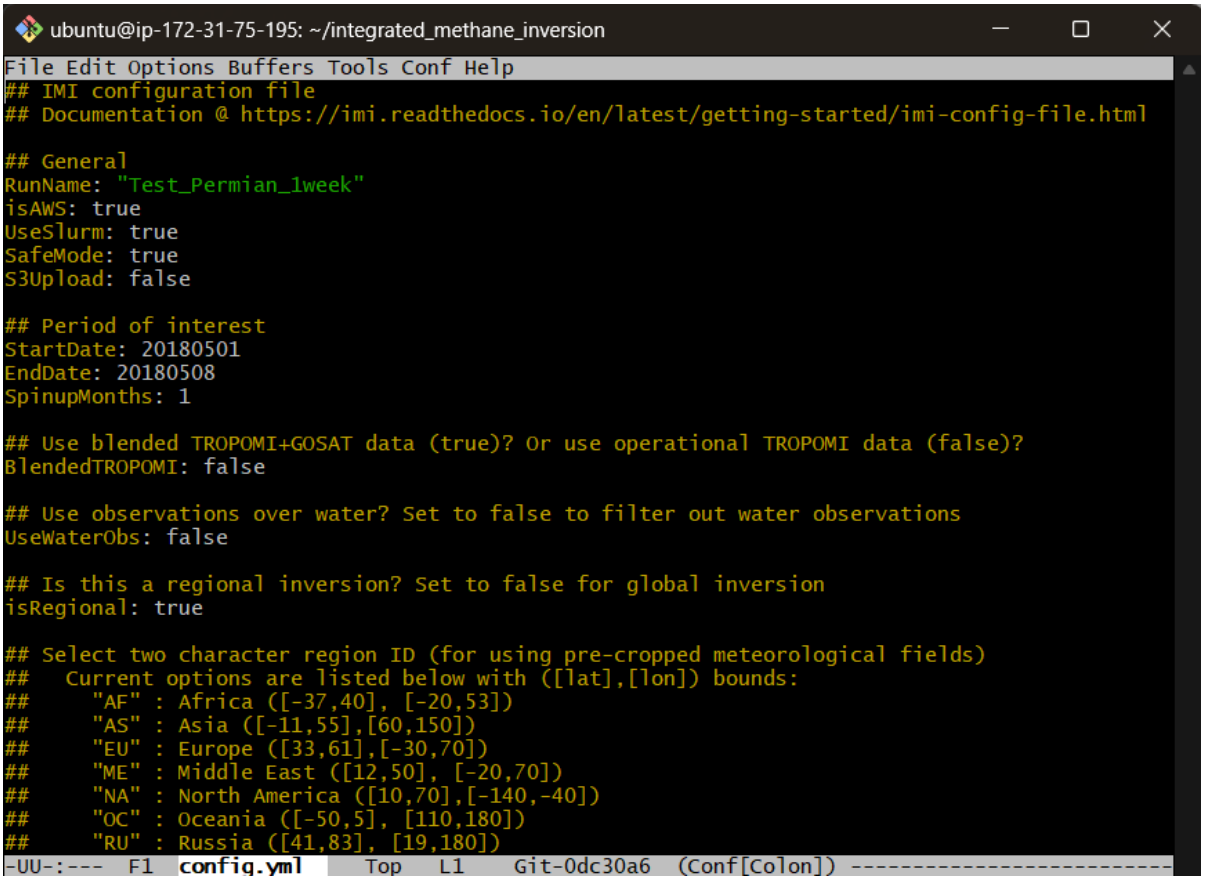


Figure 31: Beginning section of Config.yml

By default, this will run the IMI preview which will provide among other things, the expected cost of running the full inversion.

Choose an appropriate RunName and make a note of it. This must be unique for every inversion you run. Slightly further down, you can modify the date fields (format: YYYYMMDD) to choose the dates for the analysis (fig. 32).

Figure 32: RunName field and start and end dates

Bounding box settings are found further down the page in the "region of interest" section (fig. 33). As you are going to be looking at the African continent the RegionID must be set to "AF" (fig.33). As a rule of thumb, the LongMin and LongMax cocrdinates should be separated by at least 2 degrees as should LatMin and LatMax.

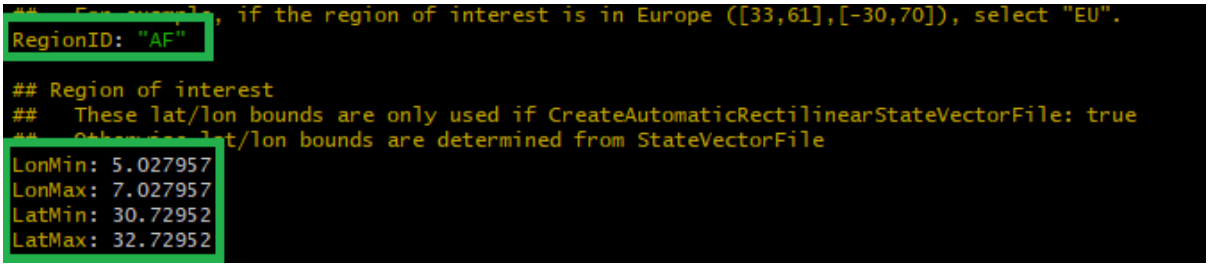


Figure 33: Analysis bounding box settings

Next there is a set of coordinates related to the Permian Basin default example that you don't want and should delete (fig.34)

Figure: 34: Point source information for Permian Basin example to be deleted.

Still further down is the IMI preview section which is by default set to "true". You will change this to "false" when deciding to go ahead with the inversion. For now don't touch it. (fig. 35).

Figure 35: Config.yml DoPreview field set to true

Once you are satisfied with the settings, press F10 on your keyboard and using the arrow keys, navigate to "save" to save any changes you made. Afterwards repeat the process, navigating to "Exit", then press enter (fig. 36).

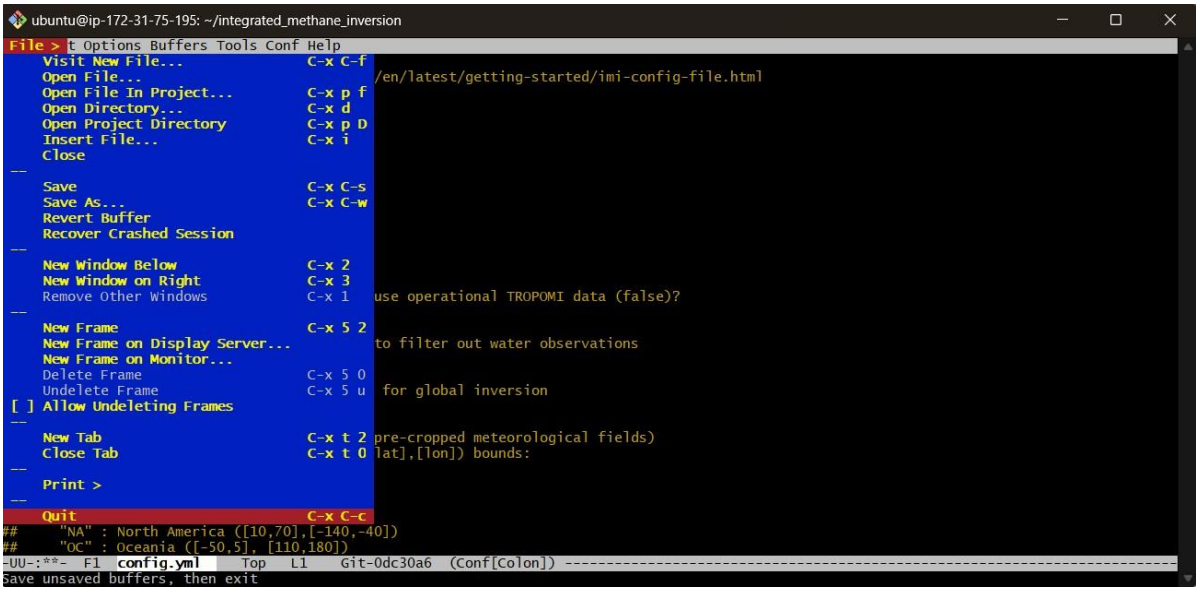


Figure 36: File menu in Config.yml

After exiting, run the following command to initiate the inversion preview:

sbatch run_imi.sh

and the following command to follow its progress:

tail –-follow imi_output.log

Once the script has finished running it will look like figure 36 showing that the IMI preview for 1 week on the Permian Basin took around 15 minutes (fig. 37).

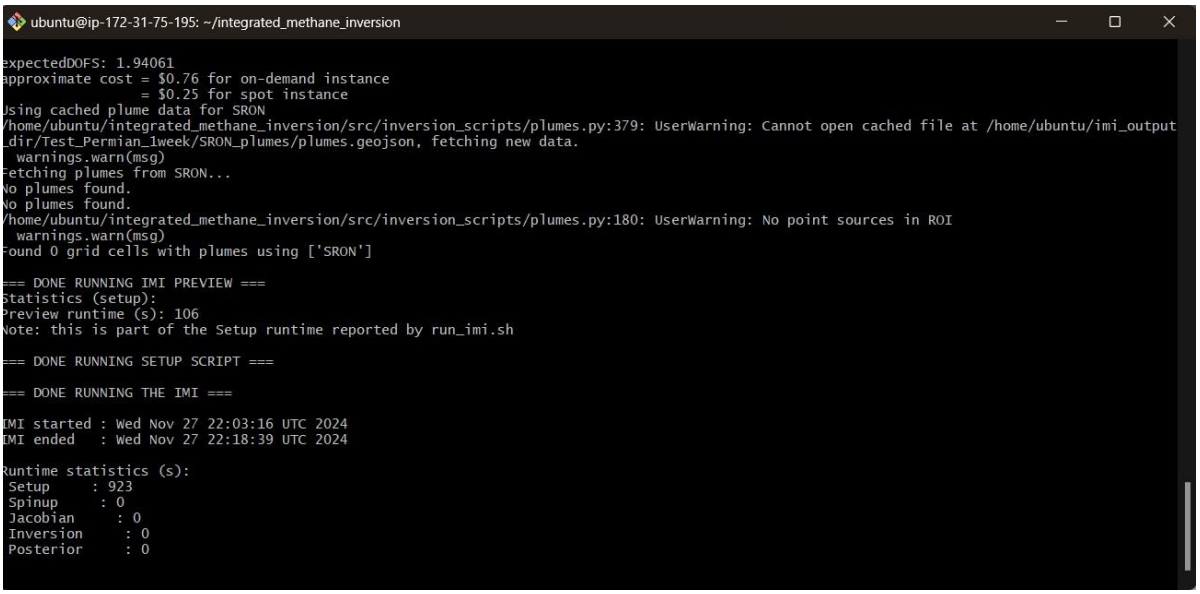


Figure 37: completed IMI Preview process.

2.3.1 Viewing the IMI Preview data

Open a new Git Bash terminal and log in as before (fig. 38)

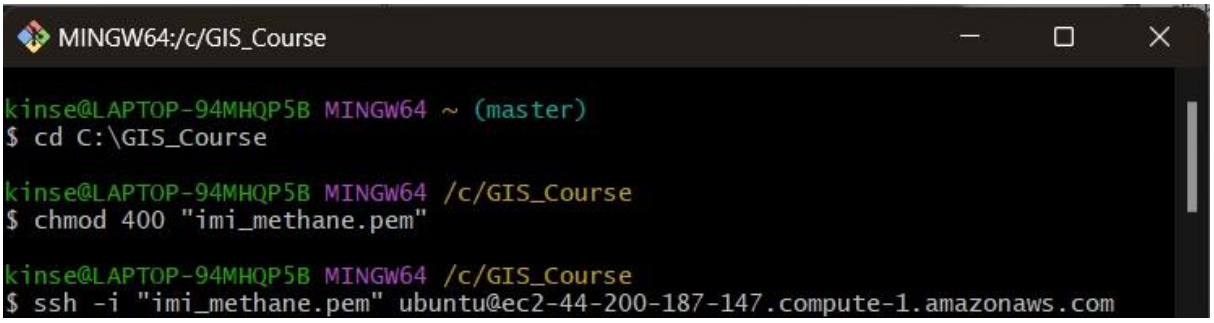


Figure 38: Login to instance

Once you have logged into the instance, enter the code:

jupyter notebook –-no–browser –-port 8080

This starts a Jupyter server on port 8080 and will print out two URLs with an authentication token. This is shown in figure 39.

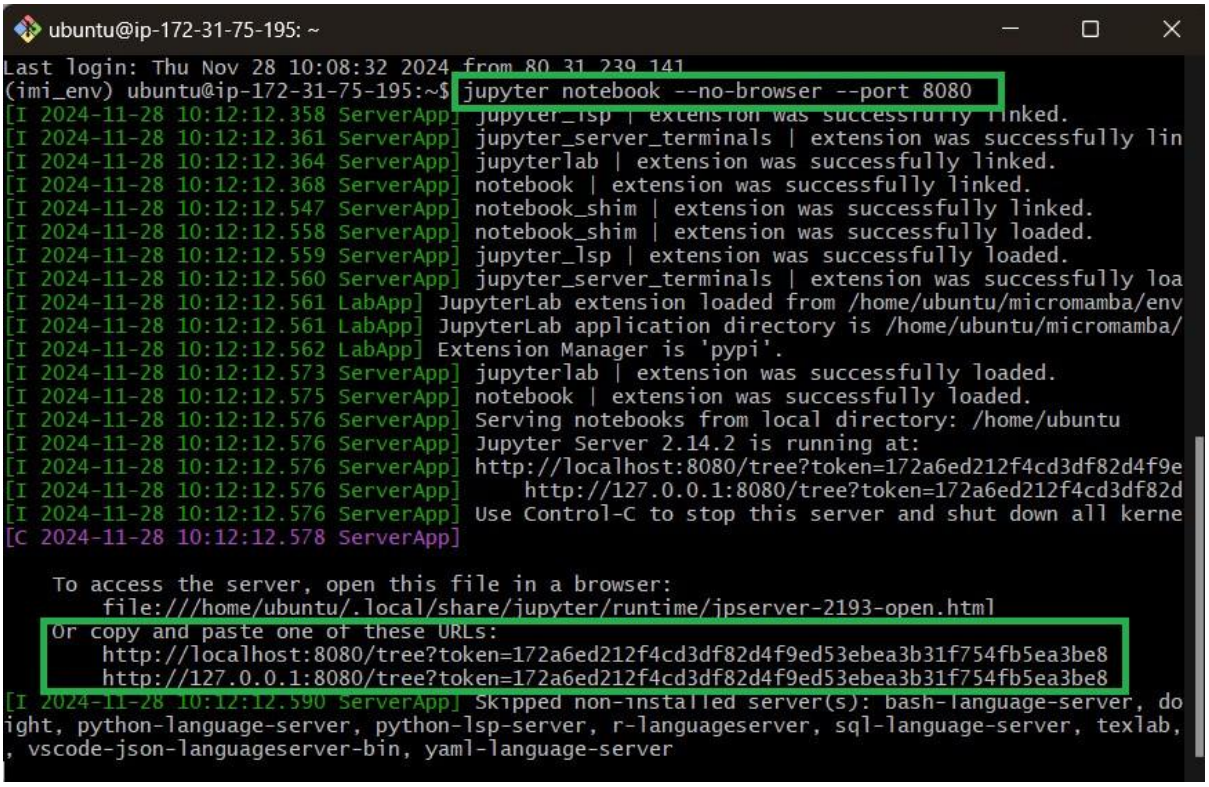


Figure 39: Jupyter server port on 8080 and URLs with authentication tokens.

Before you use these links you need to open an ssh tunnel from the ec2 instance to your local computer via port 8080. Open a new Git Bash terminal. Instead of logging into the instance as usual, you are going to use the following command to create the tunnel, changing the private key path to where your private key is stored on your local machine, and the host name to the same as the one shown in figure 40 on your connect to instance page.

Format: ssh −NL 8080:localhost:8080 −i /path/to/private_key ubuntu@<host−name>

Example: ssh −NL 8080:localhost:8080 −i "C:\GIS_Course\imi_methane.pem" ubuntu@ec2−3−238−122−2E4.compute−1.amazonaws.com

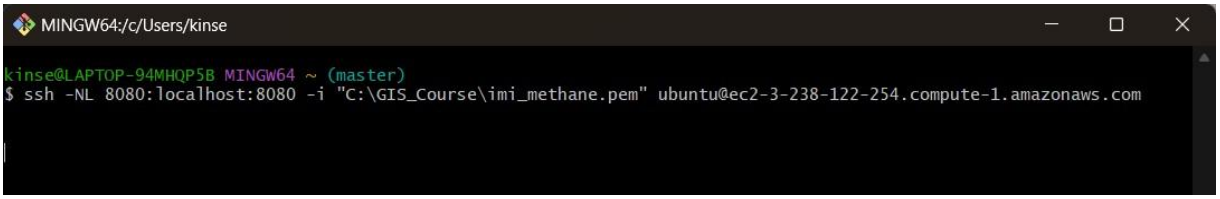


Figure 40: ssh tunnel command

Once you do this, nothing will seem to have happened in this Git Bash window but in the other window, commands will run to create the tunnel. You can now use one of the two URLs provided in figure 39 to access and view the IMI preview data (fig. 41).

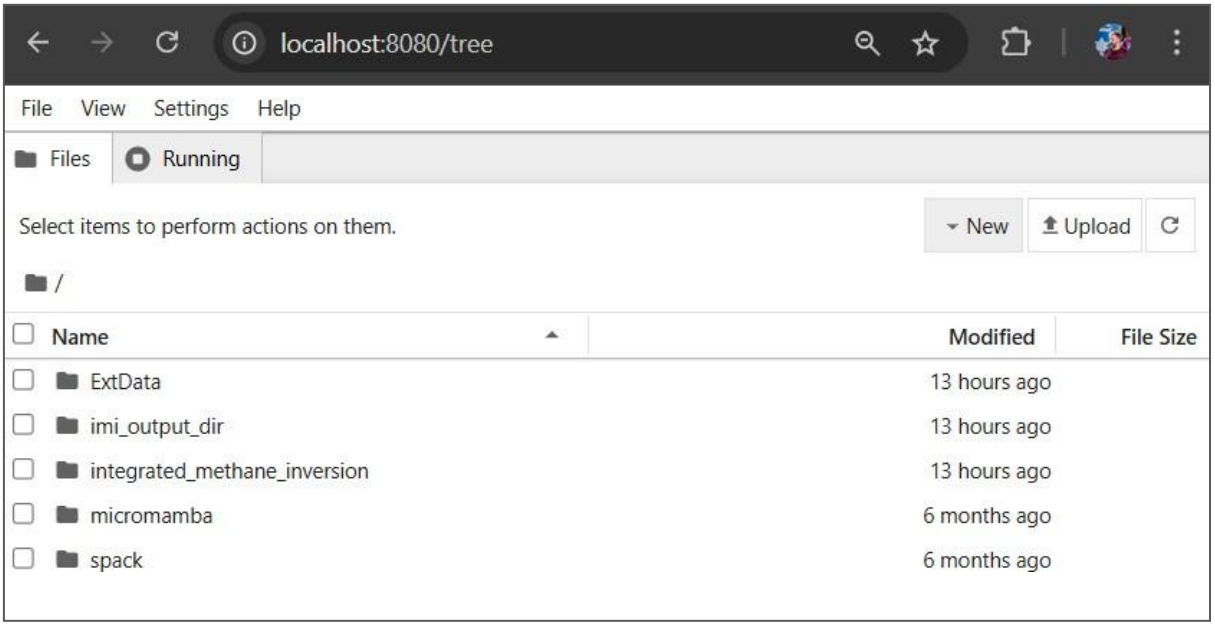


Figure 41: file directory of the IMI Preview

Once you have access to the file directory shown above, navigate to:

/imi_output_dir/(Your RunName)/preview/

From there you can download and view the IMI preview data (fig. 42)

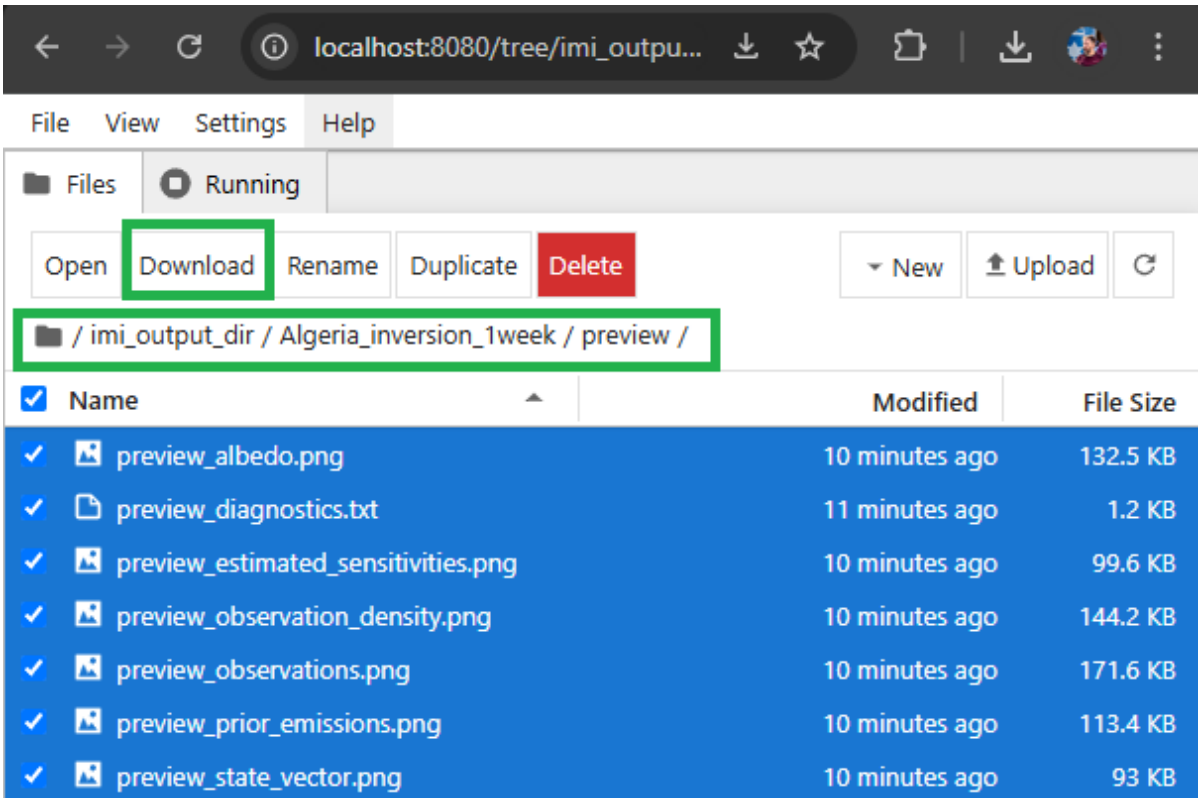


Figure 42: Location of IMI Preview data and download button

The IMI Preview will take approximately 15 minutes for a study size of 10,000km^2 of 1 week duration, with the total time scaling linearly.

2.3.2 Understanding the IMI Preview data

The IMI preview contains one .txt file and six maps, four of which are of interest. We will go through each of these in turn and explain which can help decide if you should proceed with the inversion. The first two maps to examine are the visualizations of TROPOMI whole atmosphere data (preview_observations.png) and the official "prior" emission estimates (preview_prior_emissions.png). The spatial distribution between these two maps should become more corelated, the longer the period studied, as atmospheric concentrations average out across emission sources. A poor spatial correlation over longer periods may suggest that the "prior" inventory data does not accurately reflect the actual emissions (Fig. 43).

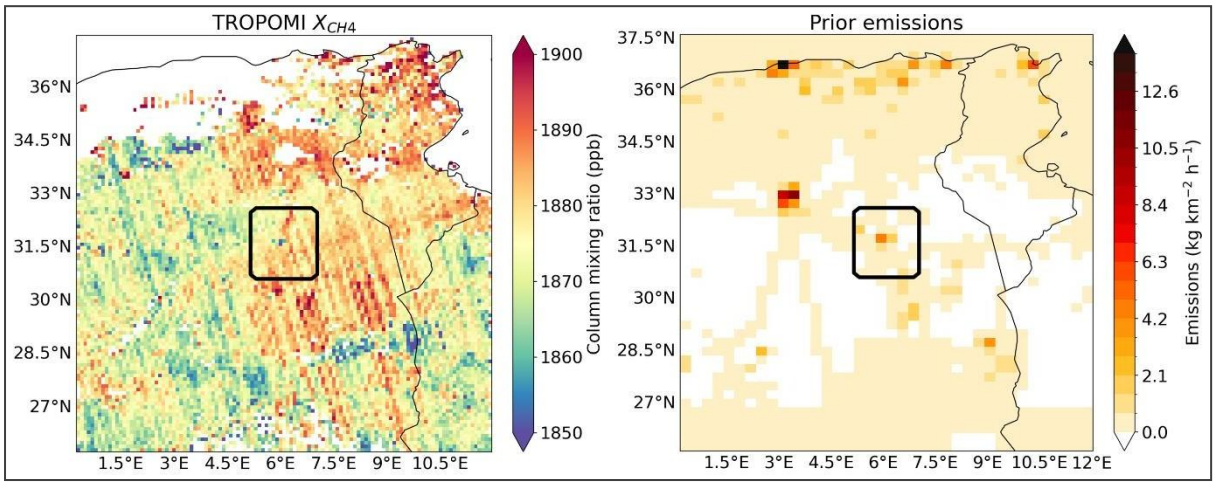


Figure 43: Mean TROPOMI Data for 1 week (left) and mapped "prior" inventory emissions (right)

The next map is of the observation density (preview_observation_density.png). In this example, the preview represents one week in May 2018, so the maximum observation density expected is 7 as Sentinel−EP has a daily return period (fig. 44). If there is significant cloud cover during the selected period, low observational density will be evident. In such cases, you may decide not to proceed with the inversion due to a lack of data.

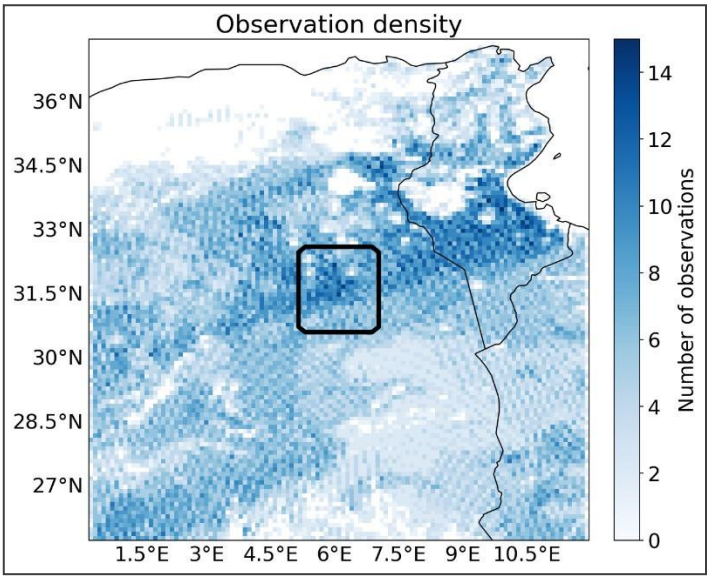


Figure 44: Observation density for 1 week

TROPOMI's measurements of surface albedo in the SWIR spectrum are visualized in the map labelled "preview_albedo.png" (fig. 45). Albedo is a critical factor in measuring gases like methane (CH_4) because variations in surface reflectivity can lead to inaccurate readings. The IMI method accounts for these differences, but areas with very low albedo may overestimate emissions, while areas with high albedo may underestimate them (Barré et al., 2021). This map does not say much about if one should go ahead with an inversion or not, but examining it, users can identify potential false readings from the final IMI, particularly if the spatial patterns of albedo and methane retrievals appear similar.

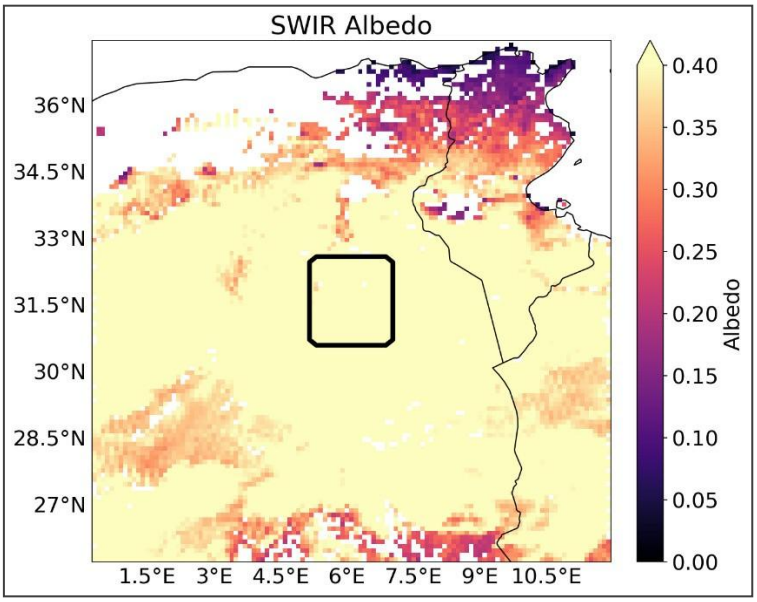


Figure 45: TROPOMI map of SWIR Albedo

The file "preview_diagnostics.txt" provides an estimate of the inversion's cost (shown in Figure 46) and a value called "expectedDOFS". DOFS, or Degrees of Freedom for Signal, measures how much the TROPOMI observations contribute to the final methane emission estimate compared to prior official inventory data. The DOFS value is influenced by the observation period; longer observation periods tend to increase DOFS, leading to more accurate results. A DOFS value of 1 is the minimum required to get meaningful emissions values, and higher values are preferred (Varon, et al. 2022). Shen et al. (2022)

found that a DOFS greater than 2 was needed to reliably estimate emissions for oil and gas basins. If your DOFS is too low, you can extend the observation period and run the IMI preview again to improve the result.

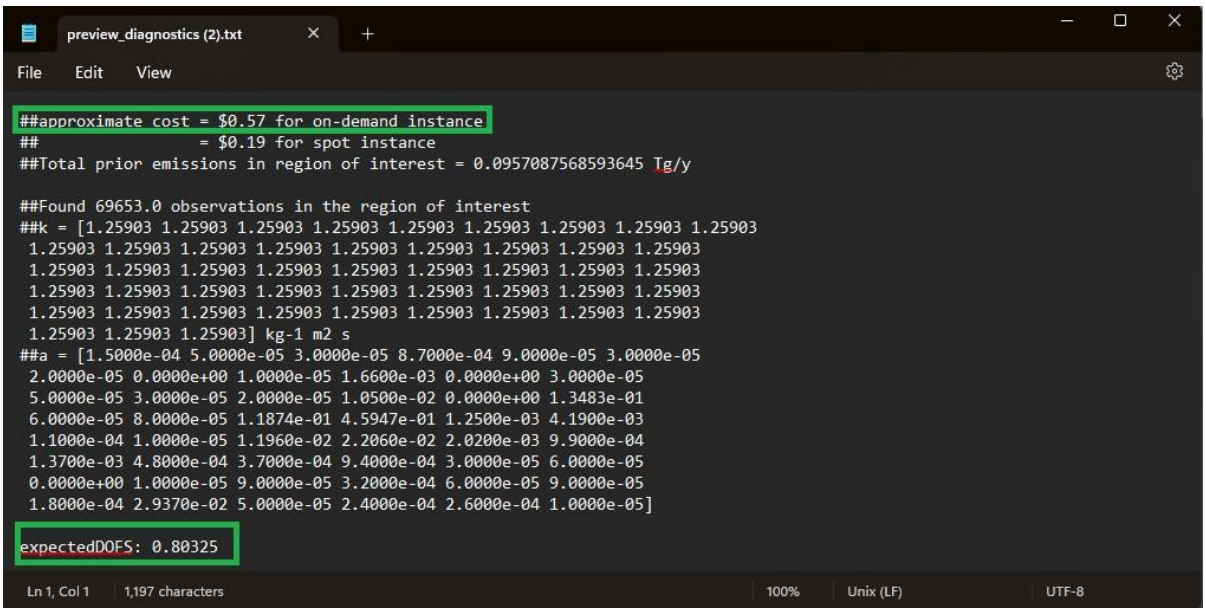


Figure 46: Preview_diagnostics.txt with estimated cost and DOES fields highlighted.

2.4 Configuring and running the IMI.

If the IMI preview looks correct and the cost is within your budget, you will now proceed with running the IMI. Open a new Git Bash terminal and connect to your instance as before (fig. 47).

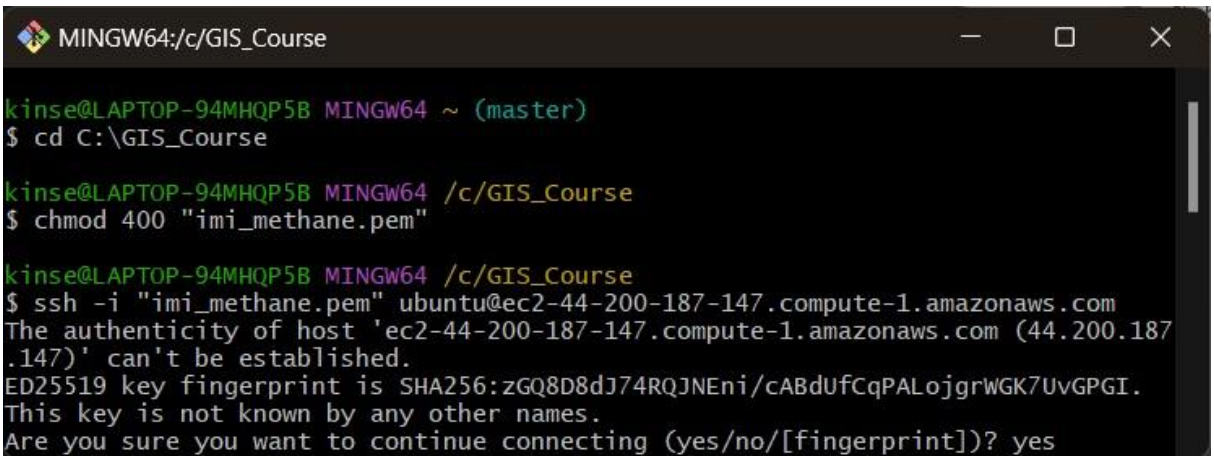


Figure 47: Connecting to instance.

In the Git Bash terminal, navigate to the IMI setup directory by typing:

cd ~/integrated_methane_inversion

Next open the inversion configuration file by typing the following:

emacs config.yml

The following screen should appear after a pause of a few moments (fig. 48)

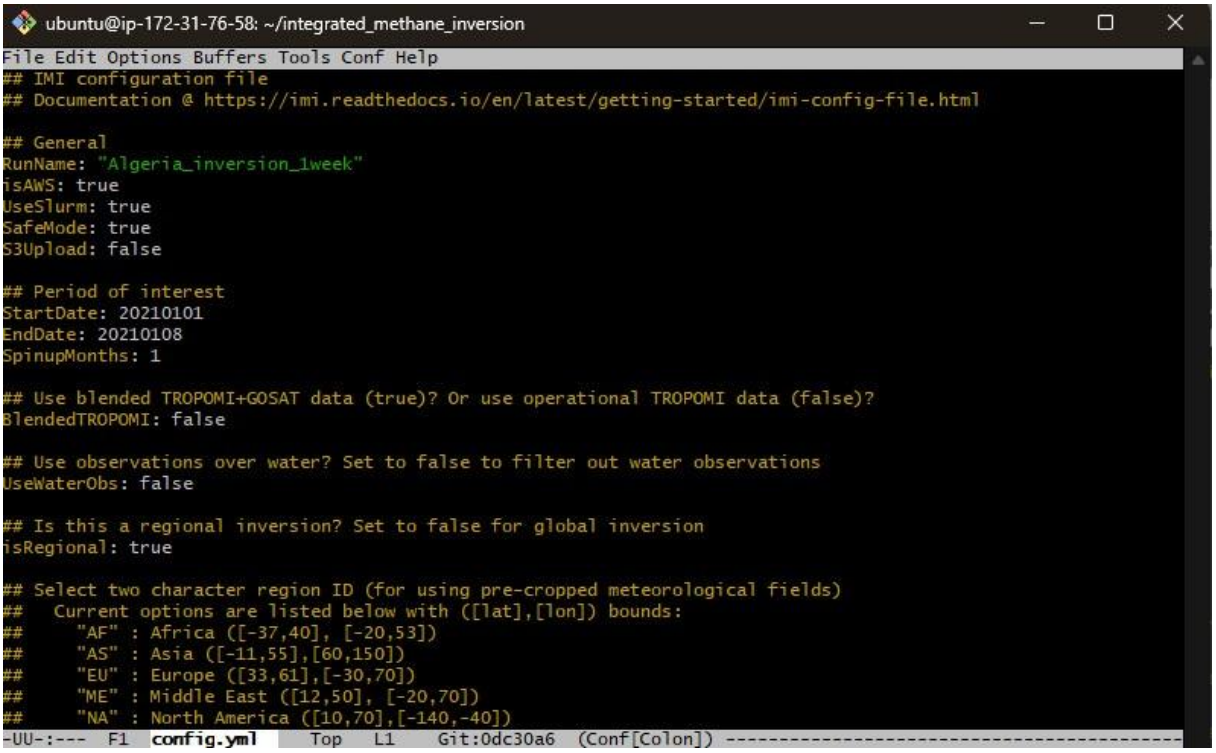


Figure 48: Config.yml, first section.

Before you changed the following fields to match the area and time you were interested in.

- StartDate
- EndDate
- LongMin
- LongMax
- LatMin
- LatMax

Now you are going to set the following fields to the values below to run the IMI instead of the IMI preview.

Setup modules

Turn on/off different steps in setting up the inversion

- RunSetup: true
- SetupTemplateRundir: false
- SetupSpinupRun: true
- SetupJacobianRuns: true
- SetupInversion: true
- SetupPosteriorRun: true

Run modules

Turn on/off different steps in performing the inversion

- DoHemcoPriorEmis: false
- DoSpinup: true
- DoJacobian: true

- ReDoJacobian: true
- DoInversion: true
- DoPosterior: true

IMI preview

NOTE: RunSetup must be true to run preview

- DoPreview: false

Once you have adjusted those settings, press F10 and save the changes (fig. 49), then press F10 again and Quit to return to the console.

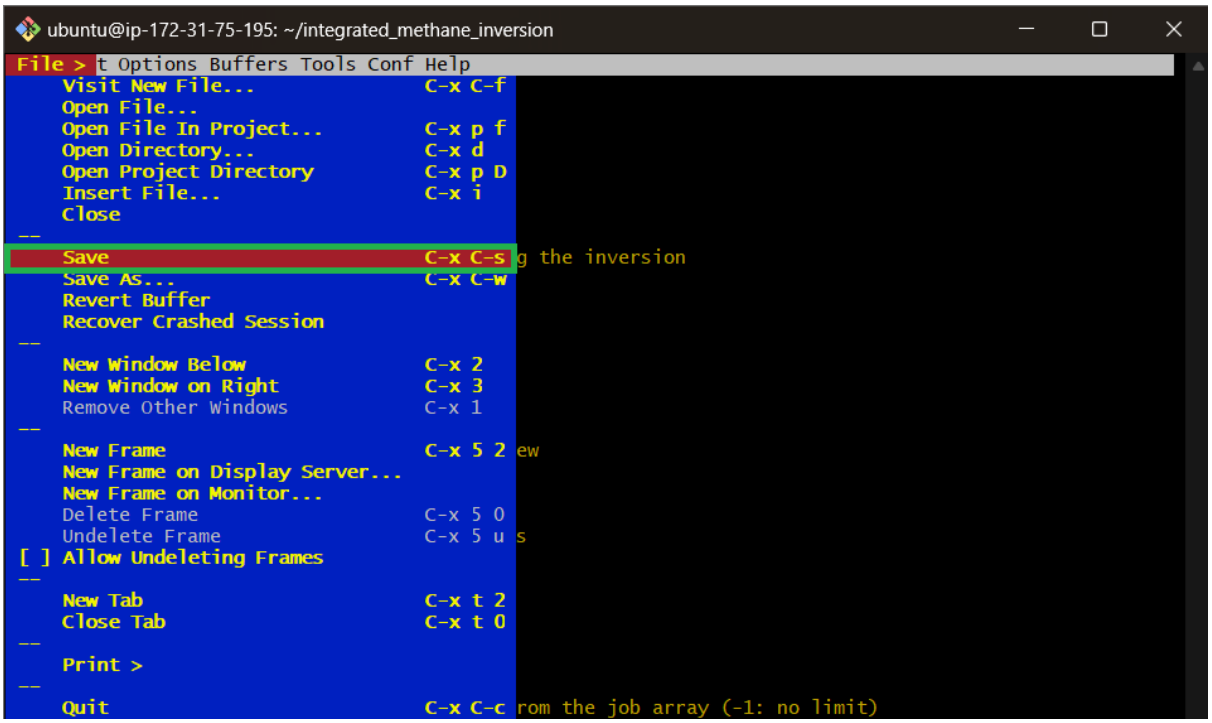


Figure 49: Location of the save button in the config.yml screen.

In the Git Bash terminal issue this command to run the IMI:

sbatch run_imi.sh

As before you can follow its progress by using the following command:

tail –-follow imi_output.log

As shown in figure 50.

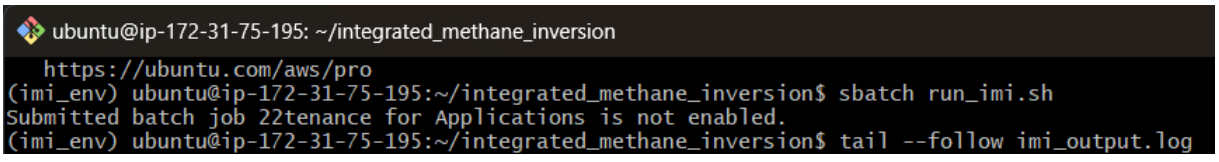


Figure 50: Commands running IMI and following progress.

At this point you can disconnect from your instance. The IMI will take approximately 2 and a half hours to run for a study size of 10,000km^2 of 1 week duration, with the total time scaling linearly.

2.4.1 Accessing the IMI data

Open a new Git Bash terminal and log in to your instance (fig. 51)

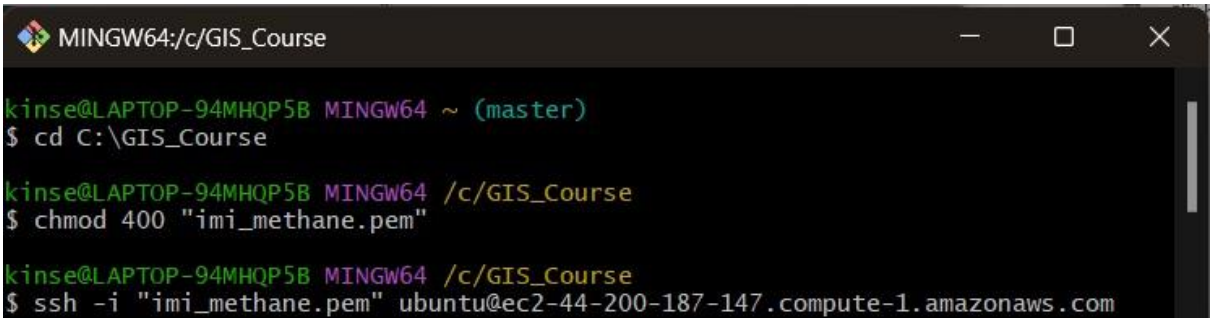


Figure 51: Connecting to instance.

Once you have logged into the instance, enter the code:

jupyter notebook –-no-browser –-port 8080

This starts a Jupyter server on port 8080 and will print out two URLs with an authentication token. This as shown in figure 52.

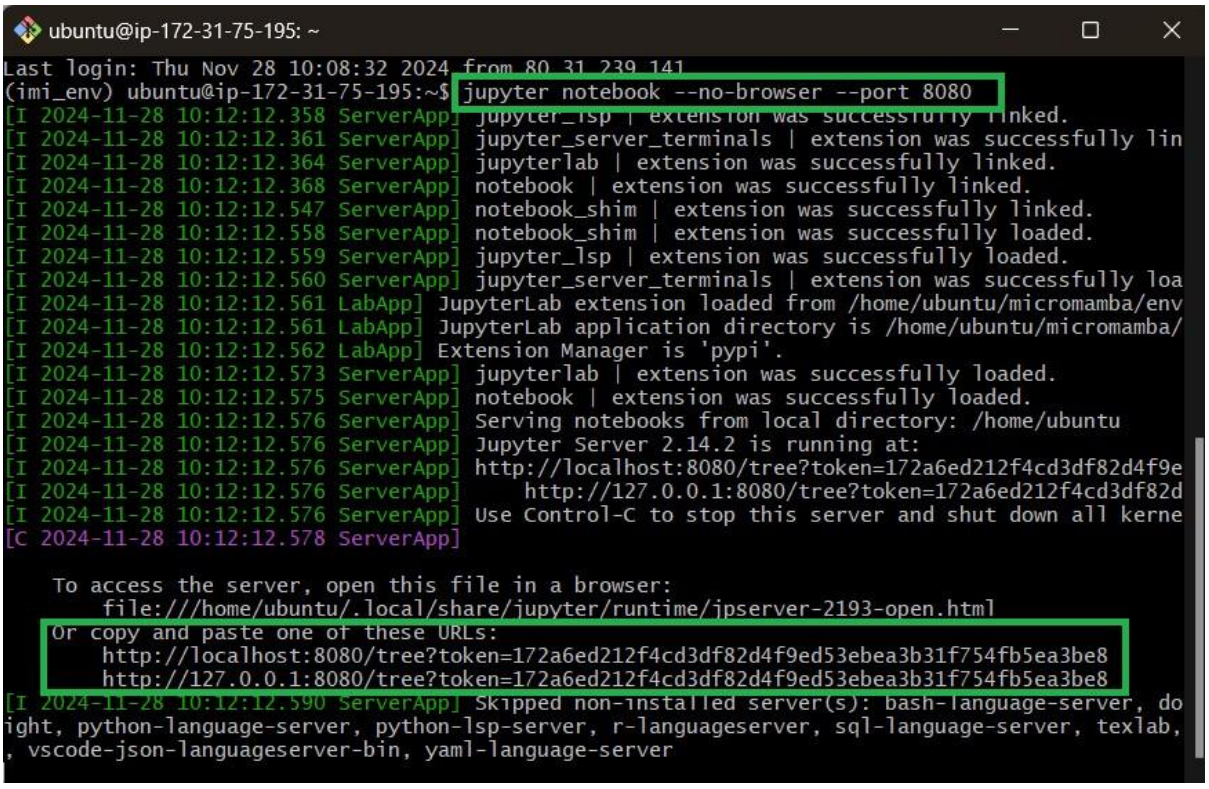


Figure 52: Jupyter server port on 8080 and URLs with authentication tokens.

Before you use these links you need to open an ssh tunnel from the ec2 instance to your local computer via port 8080. Open a new Git Bash terminal. Instead of logging into the instance as usual, you are going to use the following command to create the tunnel, changing the private key path to where your private key is stored on your local machine, and the host name to the same as the one shown in figure 27 on your connect to instance page (fig. 53).

Format: ssh –NL 8080:localhost:8080 –i /path/to/private_key ubuntu@<host–name>

Example: ssh –NL 8080:localhost:8080 –i "C:\GIS_Course\imi_methane.pem" ubuntu@ec2–3–238–122–2E4.compute–1.amazonaws.com

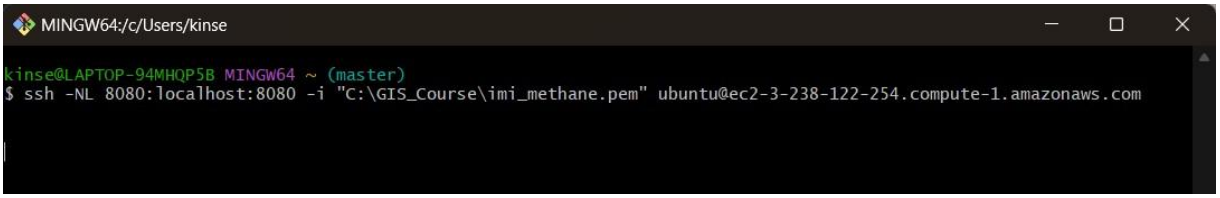


Figure 53: ssh tunnel connection

Once you do this, nothing will seem to have happened in this Git Bash window but there but in the background the tunnel will have been created. You can now use one of the two URLs provided in figure 61 to access and view the IMI preview data (fig. 54).

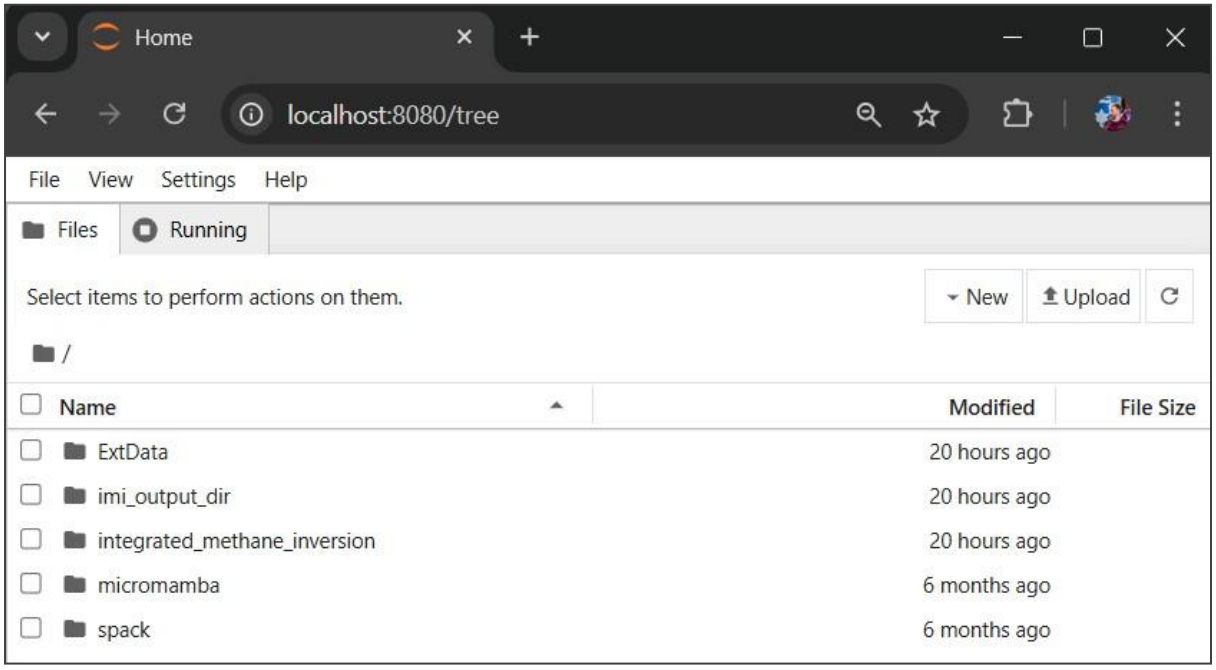


Figure 54: IMI file directory

Navigate to:

/imi_output_dir/Test_Permian_1week/inversion/

and look for “visualization_notebook.ipynb”. Open this and a Jupyter notebook will appear. In this notebook, select from the menu “Run” and then “Run all cells” (fig. 55).

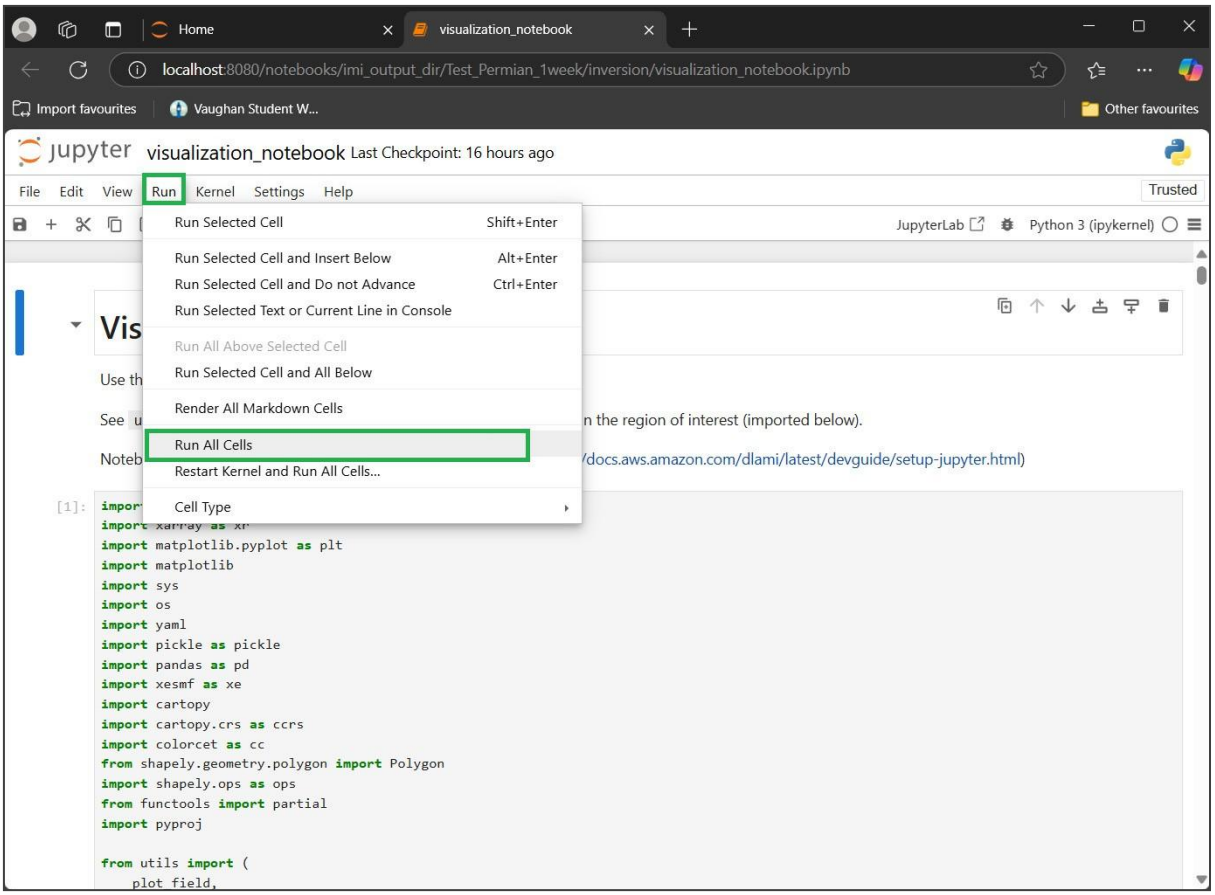


Figure 55: Run all cells command

2.4.2 Interpreting the IMI data

Once this is complete, scroll down to view the data. The first point of interest is code box 8 which shows the previous estimate for emissions based on official inventories (Prior) and the measured emissions based on Sentinel–EP and IMI (Posterior). As can be seen in figure 56, the official "prior" estimate is 0.095 Teragrams per year (Tg/y) below the IMI measured figure, which is 95,000 metric tonnes.

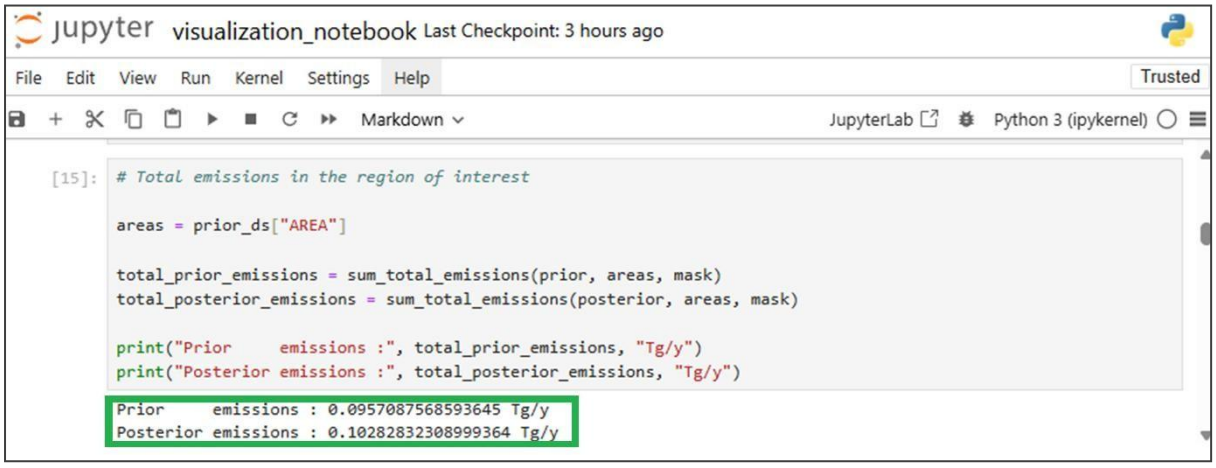


Figure 56: Official "prior" inventory emissions and Measured "posterior" emission totals.

Prior emissions: 0.0957087568593645 Tg/y

Posterior emissions: 0.10282832308999364 Tg/y

Code box 9 further visualises this and shows the official "prior" estimates on a map, whereas code box 10 shows the mapped IMI measured "posterior" emissions (fig. 57).

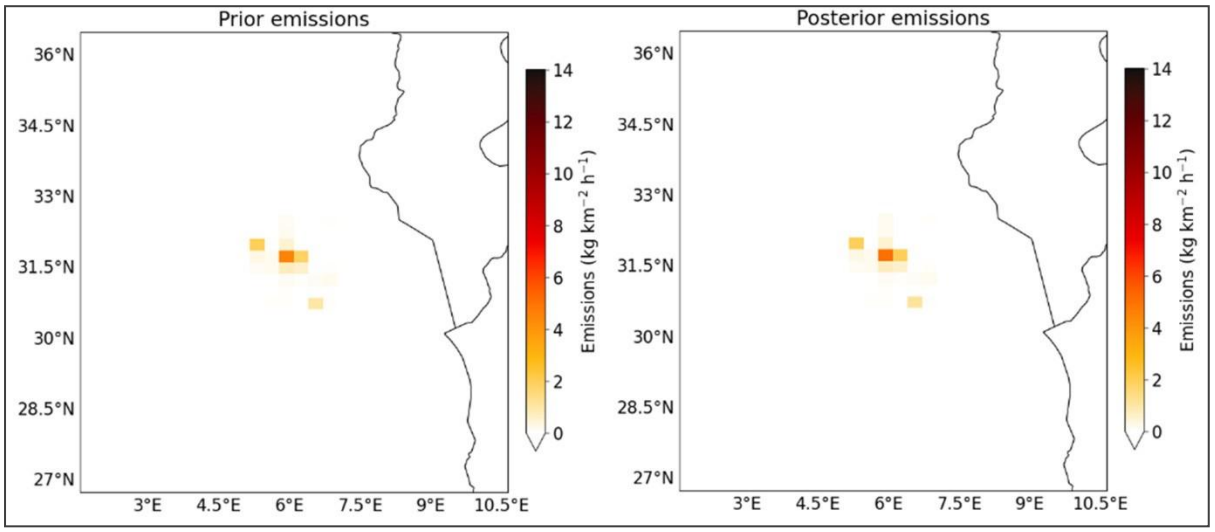


Figure 57: Mapped official "prior" inventory emissions and mapped "posterior" emissions

Code box 11 shows the sectoral emissions. This is determined by applying the ratios given by the "prior" estimates to the IMI results (fig. 58).

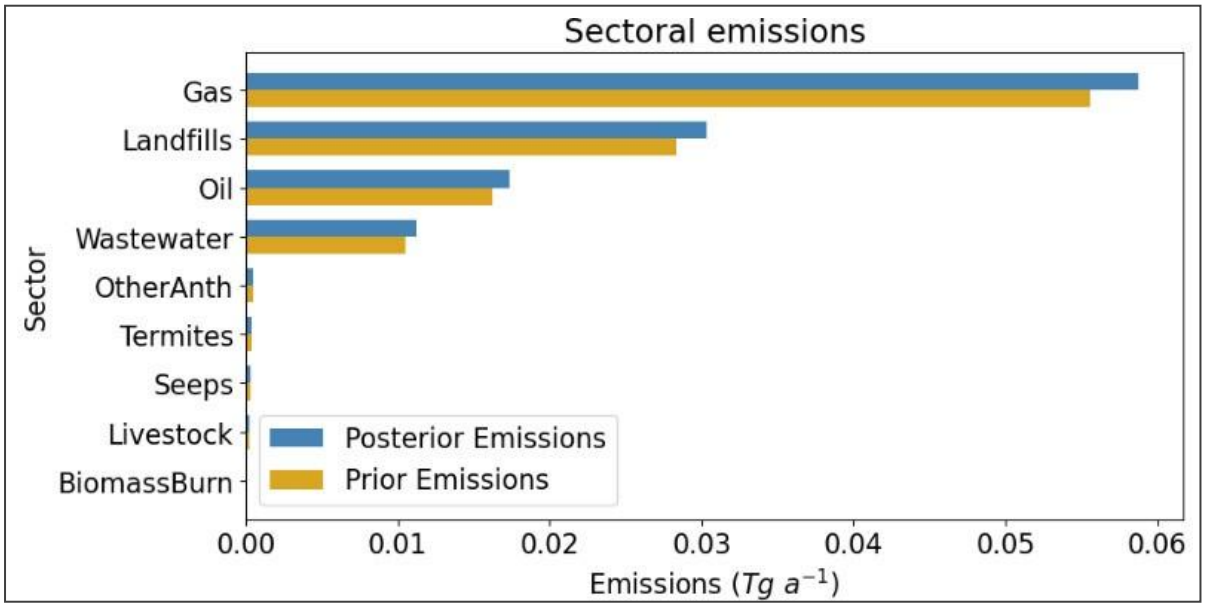


Figure 58: Emissions as divided by sector

If you wish to download any of the figures from the visualisation notebook, you can do so by navigating to:

/imi_output_dir/(Your RunName)/inversion/output/

Selecting the tick boxes for the figures and then clicking the download button (fig. 59)

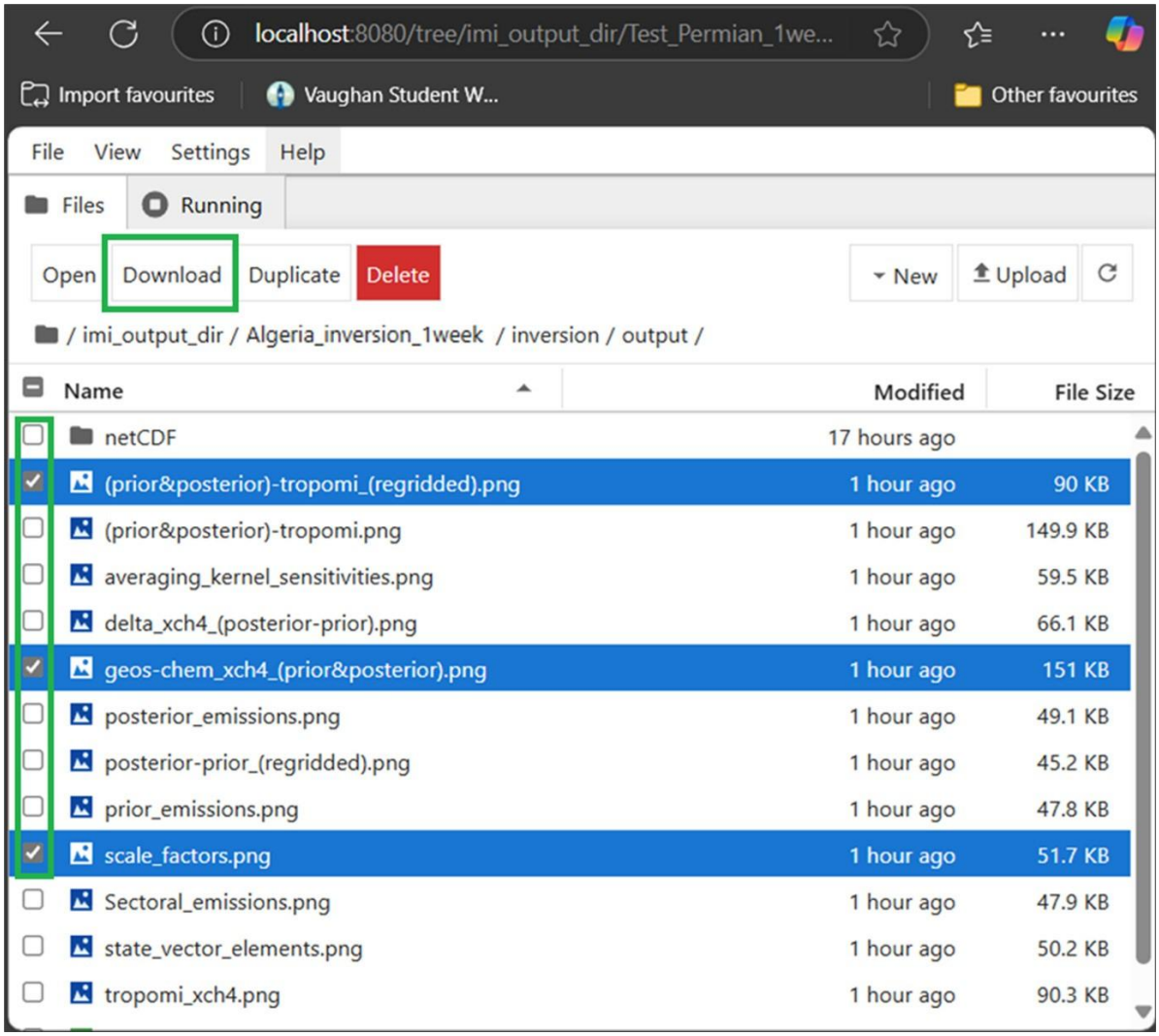


Figure 59:Location of check boxes and downloads.

2.6 Shutting down the instance

Once you have run the inversion and collected the data you need, you must shut down the inversion to avoid any unnecessary costs. Open a browser and navigate to https://aws.amazon.com/, then login to your console using the user details you set up previously (fig. 61).

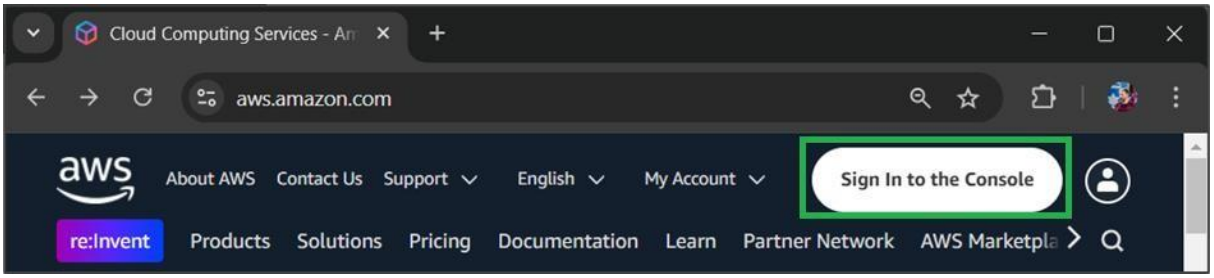


Figure 61: Sign into console button.

Once you have logged into your console, click on the EC2 Global view link (fig. 62).

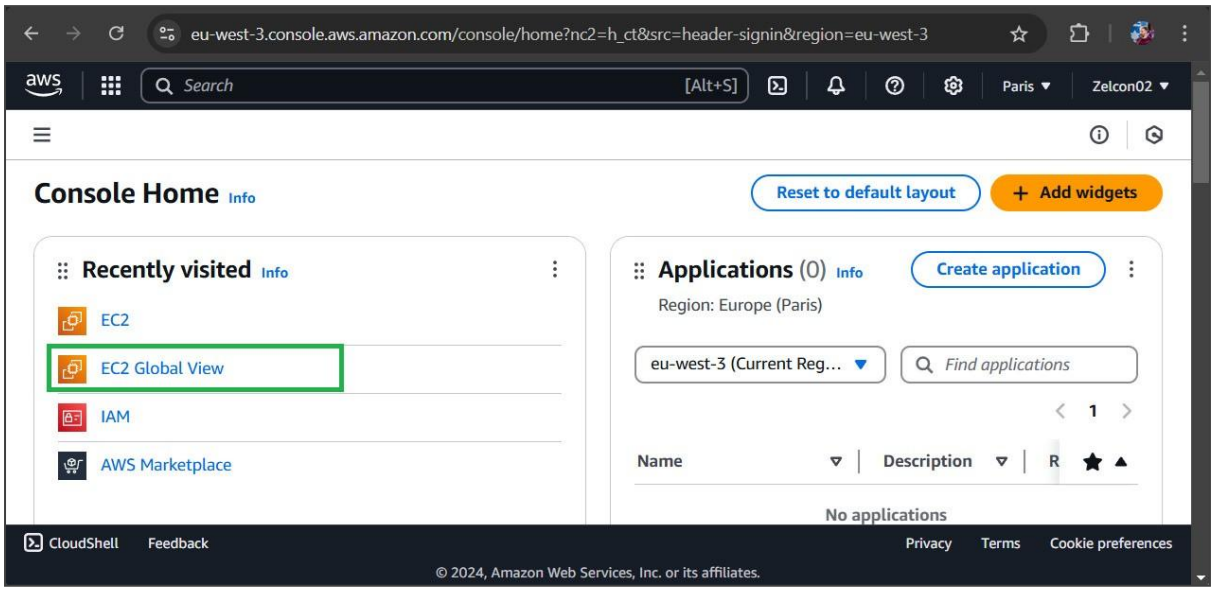


Figure 62: EC2 Global View button

You should see under “instances” 1 in 1 region (or however many instances you set up). Click on that link (fig. 63) and select “view all”.

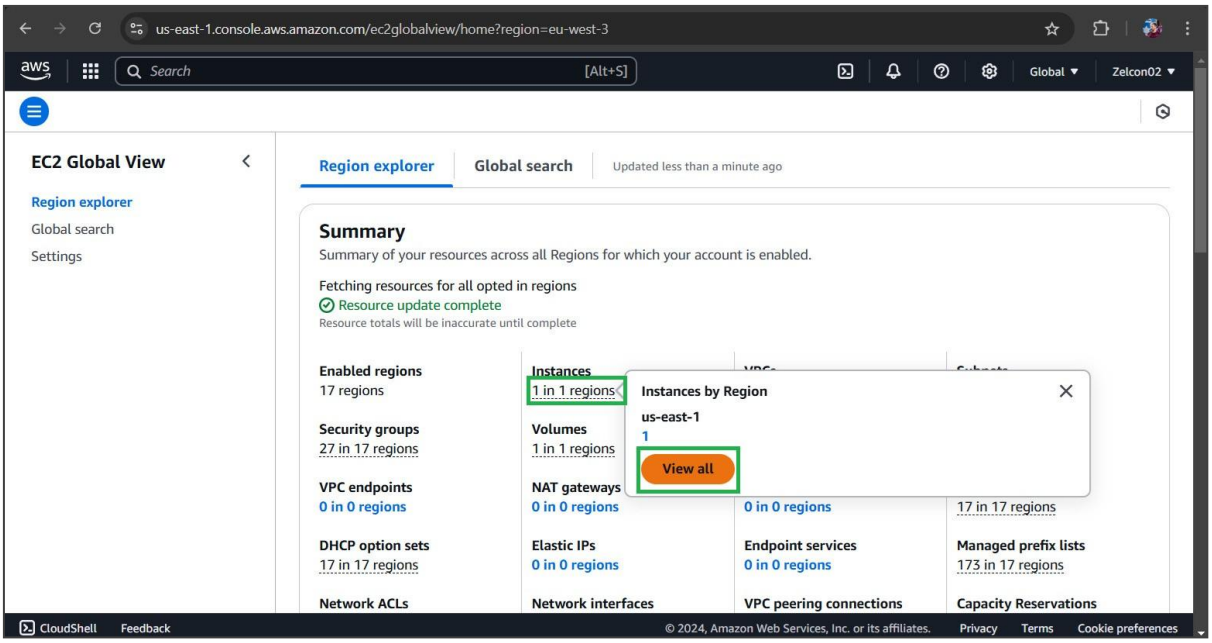


Figure 63: Instances field showing how many instances you have open.

Next click on the blue “resource ID” link (fig. 64)

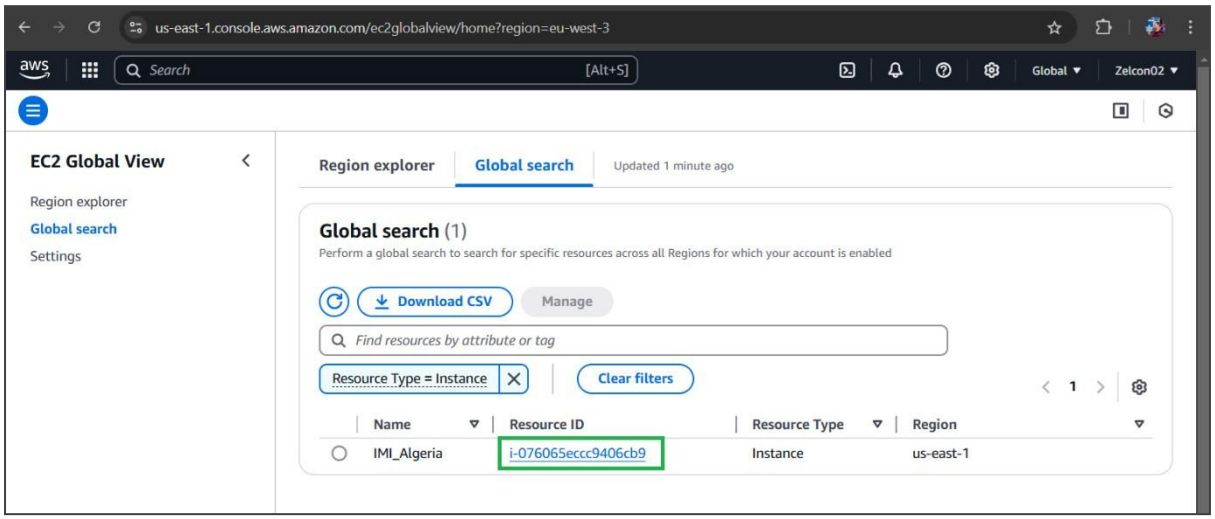


Figure 64: resource DI link location.

Finally open the “instance state” dropdown menu and select “stop instance” (fig. 65). Thereafter you should receive a message confirming the operation.

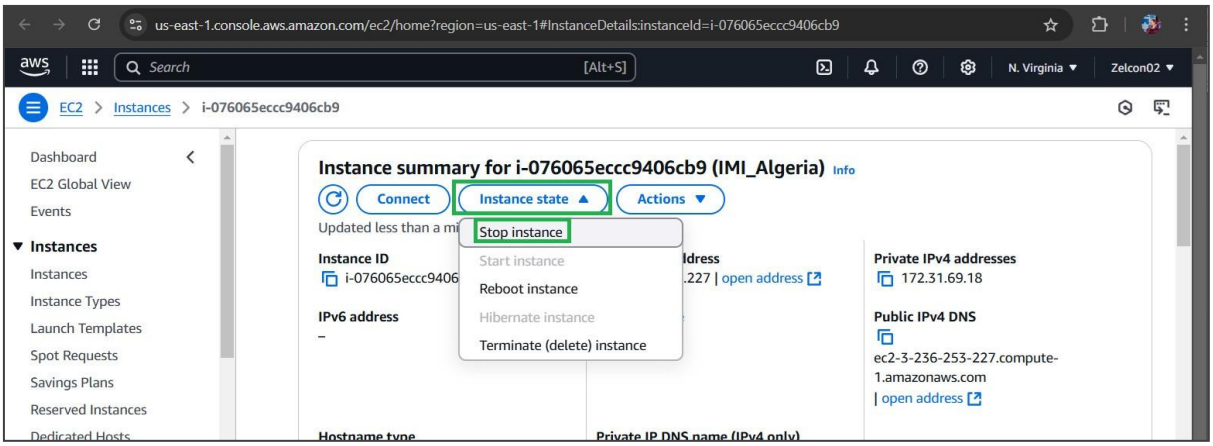


Figure 65: location of stop instance button.

2.7 Troubleshooting

If you are having problems running the IMI, please submit a ticket at the following Github page where the developers will attempt to help you out.

https://github.com/geoschem/integrated_methane_inversion/issues

3. CH4 Sentinel

3.1 Methodology

To help users locate the largest emission sources in a field, Sentinel 2's Multispectral Instrument (MSI) with a 20m spatial resolution and return frequency of 2−5 days was employed.

Varon et al. (2021) demonstrated that methane columns from point sources can be measured by exploiting the Band 11 and Band 12 of Sentinel−2's MSI. The software uses Sentinel 2's L1C data accessed via the Copernicus OpenEO Application Program Interface (API) (Musial et al., 2024). The data is filtered to only use scenes with a maximum of 5% cloud cover. No emission and active emission scenes were chosen as two sequential passes and processed as in Varon et al. (2021), first using a MBSP method:

$$MBSP = \frac{B12 - B11}{B11}$$

Where: *B12* is the Sentinel−2 SWIR−2 band and *B11* is the Sentinel−2 SWIR−1 band.

Once the MBSP raster had been calculated, the following equation was then used to calculate the multi−band−multi−pass raster:

$$MBMP = ActiveMBSP - NoMBSP$$

Where ActiveMBSP is the multiband single pass for the active emission scene and NoMBSP is the multiband single pass for the no emission scene.

Figure 66 provides an overview of the main steps in processing.

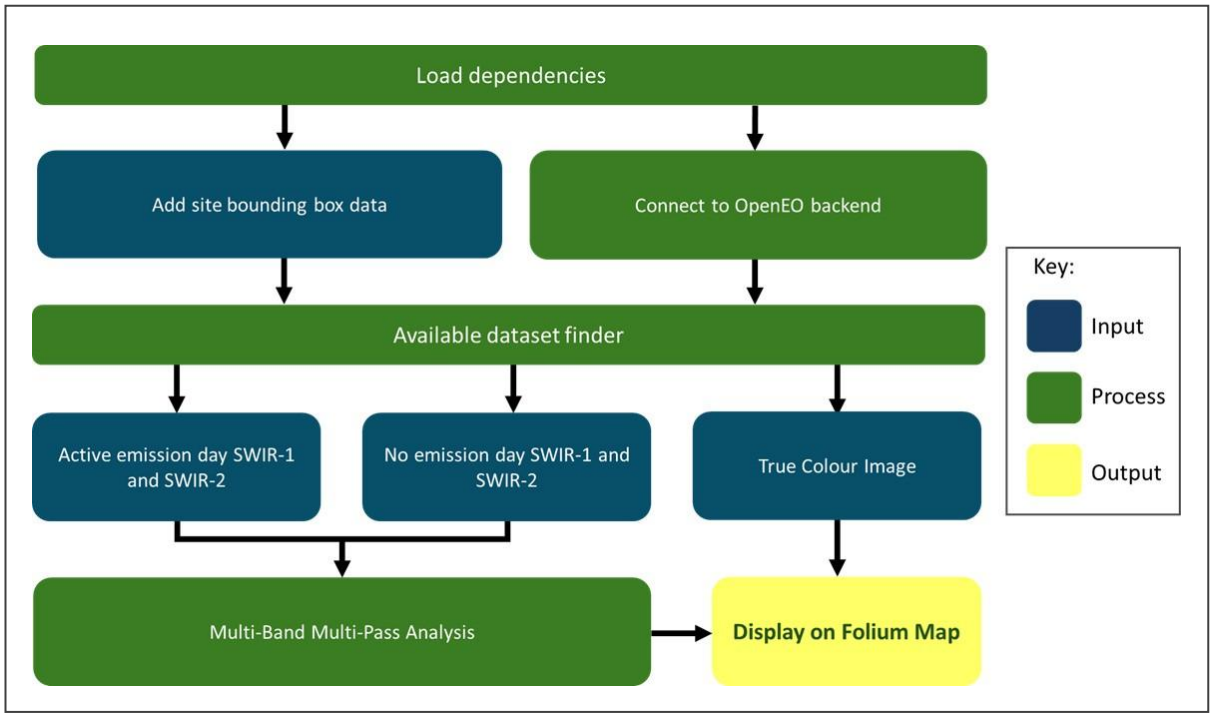


Figure 66: Flowchart of processes used for CH4 Sentinel

3.2 Dependencies

The dependencies contained in the environment.yml file are outlined in table 1.

Table1: dependencies required to use the tools.

Name	Version	Description
Python	3.12.2	Programming language to run the code
Jupyterlab	4.1.4	Computing environment for Python
Folium	0.20.0	Generates interactive maps with Leaflet.js.
Pandas	2.3.1	Data manipulation and analysis.
Geopandas	1.0.1	Geospatial data manipulation and analysis.
Matplotlib	3.9.4	Creates map visualizations
Rasterio	1.4.3	Reads and edits raster datasets.
Numpy	2.0.2	Performs numerical computations on arrays
Requests	2.32.4	Used fetch data from web services or APIs.
Rasterstats	0.19.0	Linear regression for brightness correction
OpenEO	0.43.0	API for Earth observation data processing
Shapely	2.0.3	Used to manage points, lines and polygon data
cdsapi	0.7.6	Python client for the Copernicus Climate Data Store API
xarray	2024.7.0	Works like pandas for labelled multi-dimensional arrays
scipy	1.13.1	Scientific computing; spatial analysis, statistics, filtering, segmentation
Scikit-learn	2.0.7	Machine learning library; regression, PCA, scaling, metrics.
xgboost	2.1.4	Gradient boosting machine learning algorithm.
geopy	2.4.1	Geocoding

3.3 Setup

Anaconda Navigator, containing the Python programming language and 'Conda', a package manager for creating shareable environments that tools like this can run on will be installed. This includes Jupyter Lab which will be used to run this tool's code.

3.2.1 Anaconda Navigator

To download Anaconda, navigate to https://docs.anaconda.com/anaconda/install/ and follow the instructions of your operating system.

3.2.2 Creating a Conda Environment

In the Anaconda Navigator side bar, click the 'Environments' tab. You will see the installed packages (fig.67).

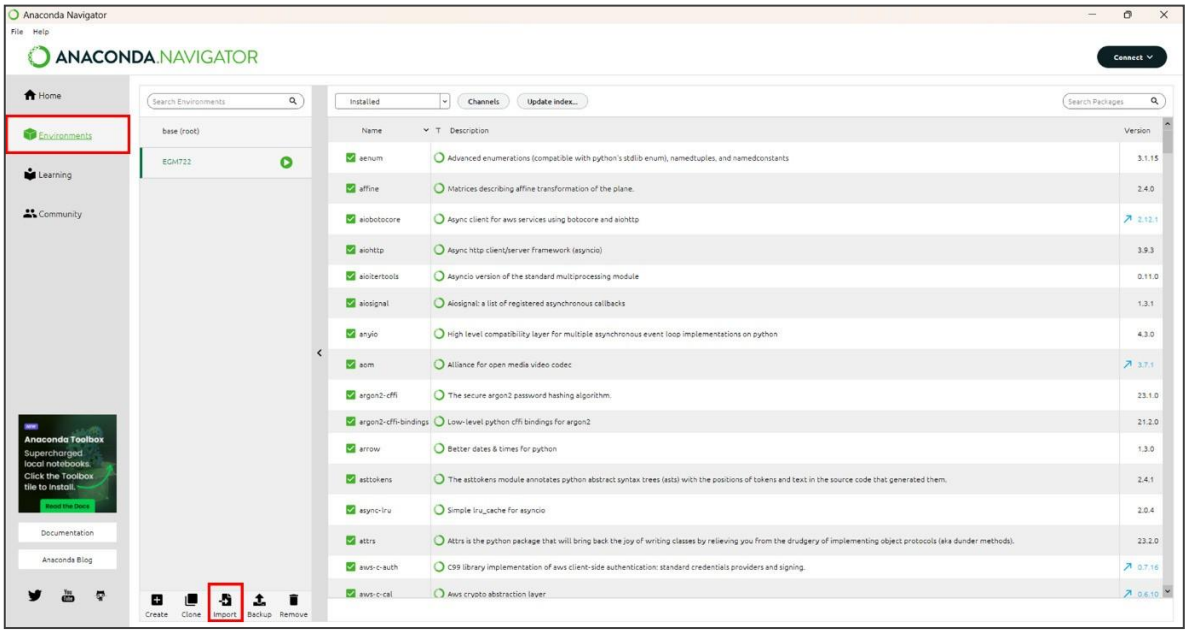


Figure 67: Environments tab of Anaconda Navigator with environments tab and import button highlighted in red.

Next click on the imports tab (fig.3) and select the file 'environment.yml' contained in the .zip file of the tool's download, choosing an appropriate name for the environment (fig. 68).

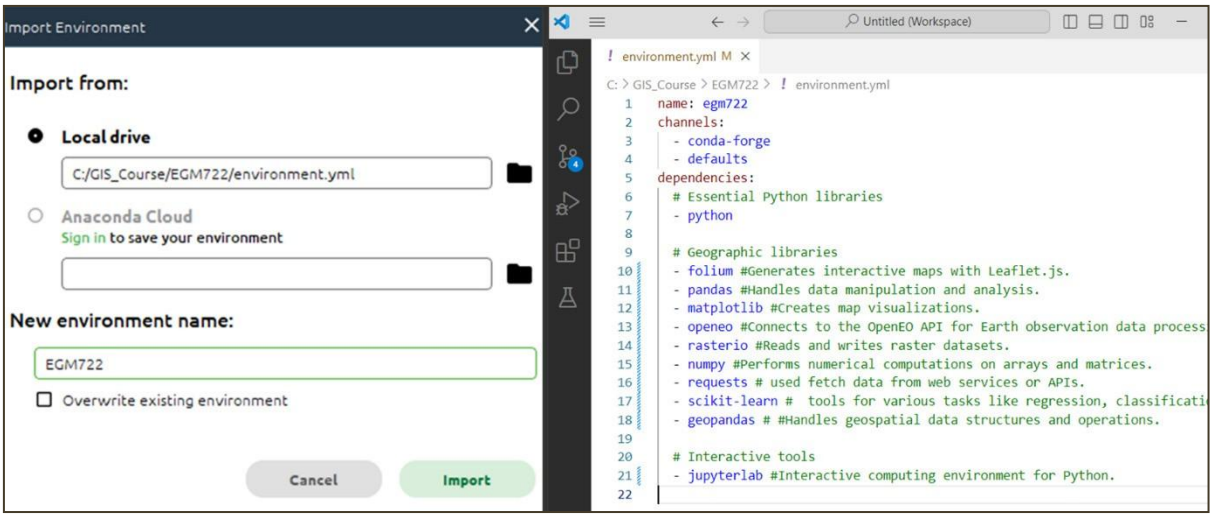


Figure 68: The import config box (left) and the contents of 'environment.yml' (right).

Click Import. The process of installing all the packages may take several minutes. Once finished you will be returned to the environments tab (fig. 67)

Next click on the 'Home' tab in Anaconda Navigator's sidebar (fig. 69).

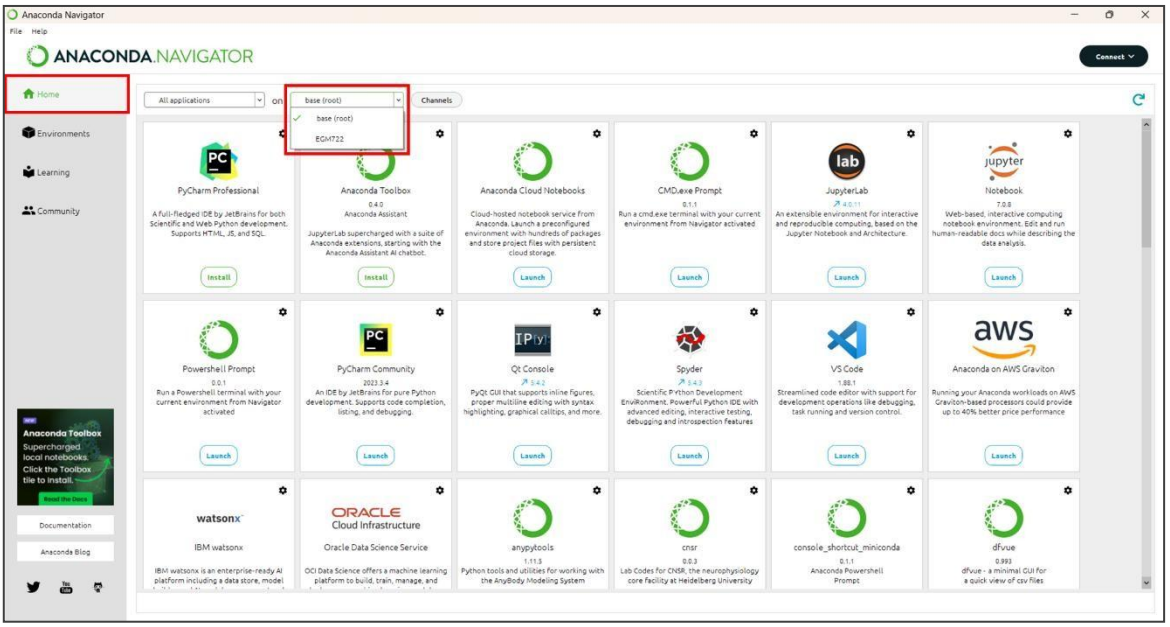


Figure 69: Anaconda Navigator with home tab and environment switching dropdown in red.

The dropdown highlighted in figure 69 should display two options, 'base (root)' and the name of your new environment (in figure 69 this is 'EGM722). Ensure your environment name is always selected here or the dependencies installed earlier will not be available to it.

3.2.3 Setting up Jupyter Lab

A configuration file ('.config') needs to be created to change the settings used by Jupyter Lab by default. Launch the CMD.exe prompt (fig.70)

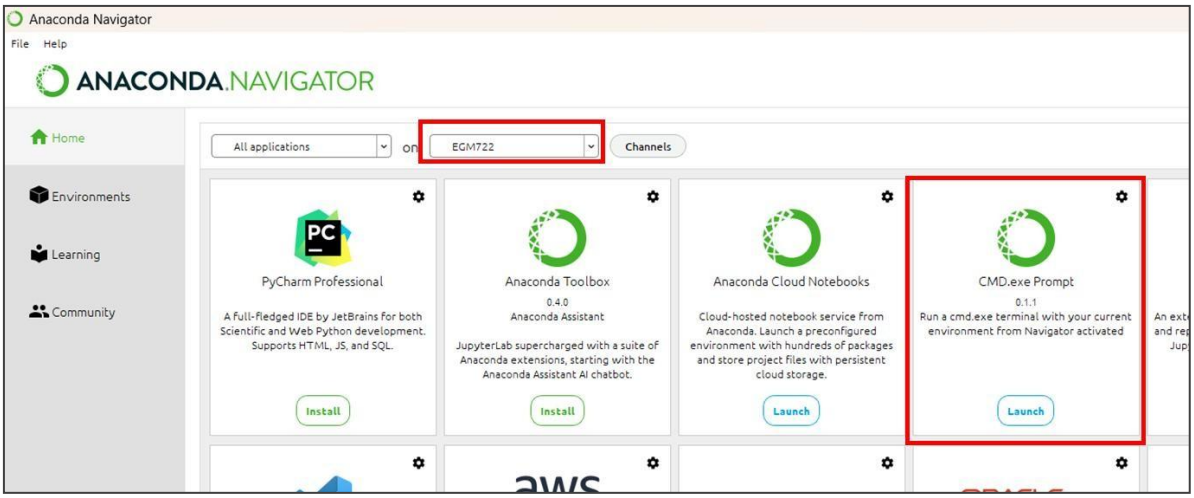


Figure 70: Highlighted locations of selected environment and CMD.exe Prompt

In the command prompt, enter the command:

```
jupyter lab --generate-config
```

This will create a new folder in your user directory called '.jupyter' containing a python script juptyer_lab_config.py. On Windows this is usually 'C:\Users\<your_username>'.

Jupyter lab will by default open in your user directory. Due to security restrictions, it is not possible to navigate to the parent directory of the launch location. So if Jupyter launches in 'C:\Users\RockyBalboa, it is not possible to move to 'C:\Users' or, 'C:\EGM722'. If the directory you are keeping your data in is outside your user directory, you will need to change the default opening folder to your data directory.

This location is also where you should store the following files and folder:

- CH4 Sentinel
- The folder "Data"
- Environment.yml

If your data directory is in your user directory, you should be able to click and navigate there using the interface of Jupyter Lab. If that is not the case, you will need to do the following:

Open an Anaconda Navigator CMD.exe prompt and type the following command:

```
jupyter --paths
```

This will show something like figure 71 although your file paths will be unique to you.

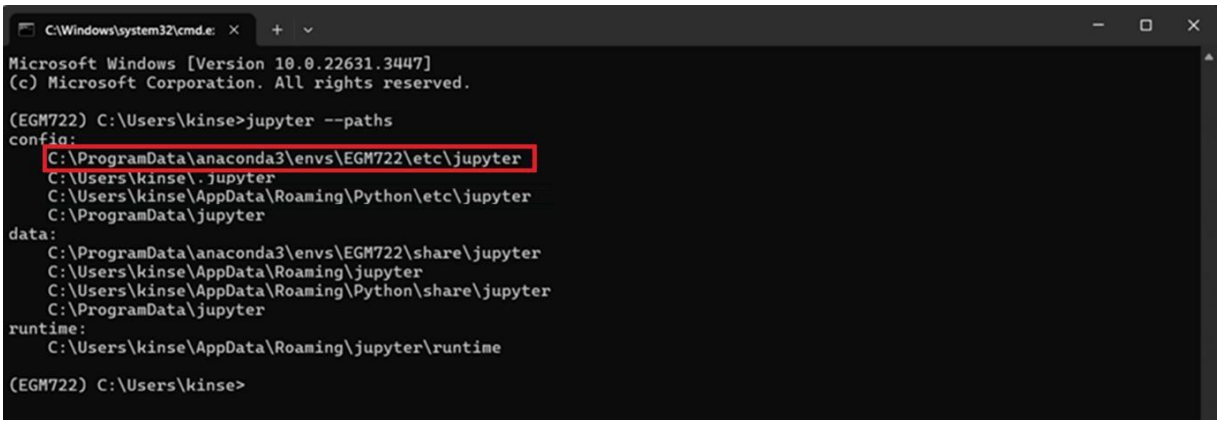


Figure 71: results of 'jupyter –paths' command showing path used by environment highlighted in red.

The 'jupyter_lab_config.py' file mentioned earlier needs to be copy pasted into that folder.

Once 'jupyter_lab_config.py' file has been moved, open it in Notepad++, Visual Studio Code or if you don't have those, Notepad. Using the shortcut 'CTRL + F' type in the following line: 'c.ServerApp.root_dir' (without quotes) and you should find the section highlighted in figure 72.

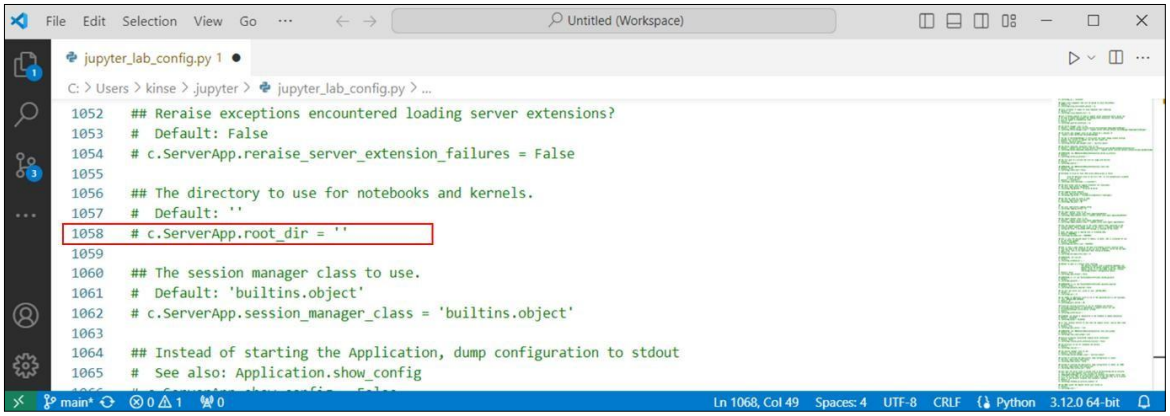


Figure 72: location of 'c.ServerApp.root_dir' in jupyter_lab_config.py

Remove the '#' and space from the start and add the path used by your environment between the quote marks, adding a 'r' beforehand (fig.73).

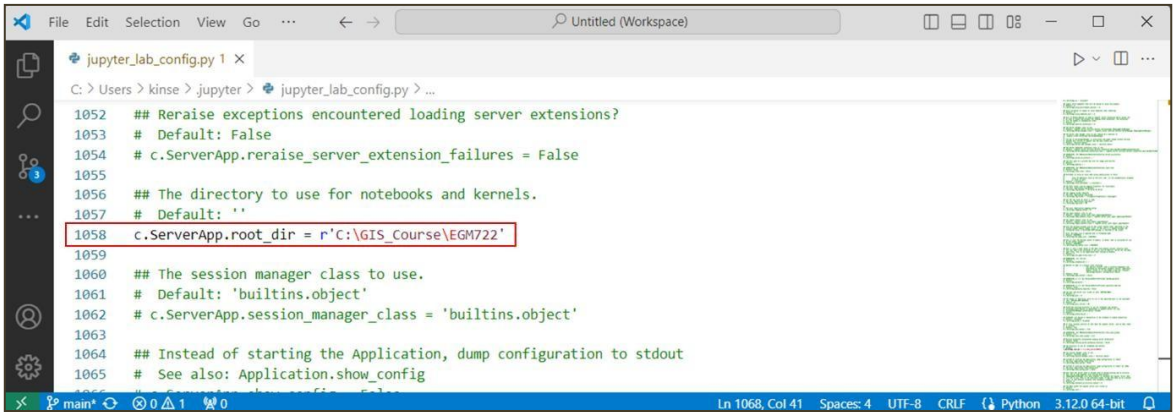


Figure 73: path to data directory added to jupyter_lab_config.py

Save and close this file and return to the Anaconda Navigator 'Home' tab. Launch Jupyter Lab and if you have followed the steps correctly, you should see that your data directory is automatically displayed (fig.74).

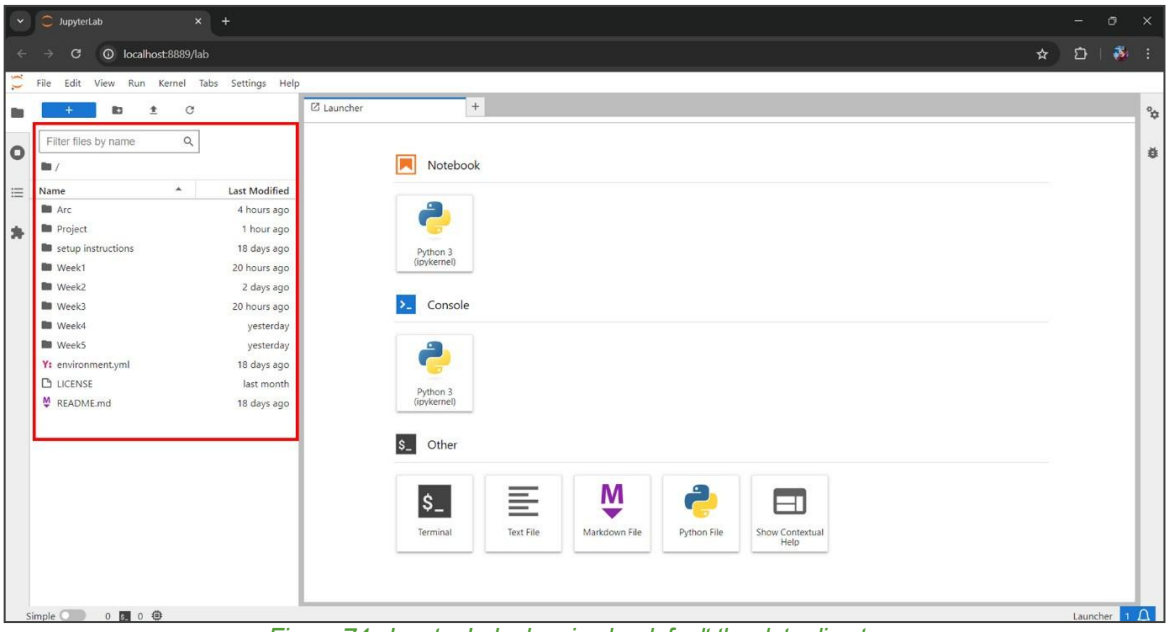


Figure 74: Jupyter Lab showing by default the data directory

3.2.4 Registering with Copernicus Data Space Ecosystem.

To access Sentinel-2 data, we will use an API called OpenEO that was installed automatically earlier. It requires an authentication. To do this, you need to complete a Copernicus Dataspace Registration. Go to https://dataspace.copernicus.eu/ and click the login button (fig. 78)

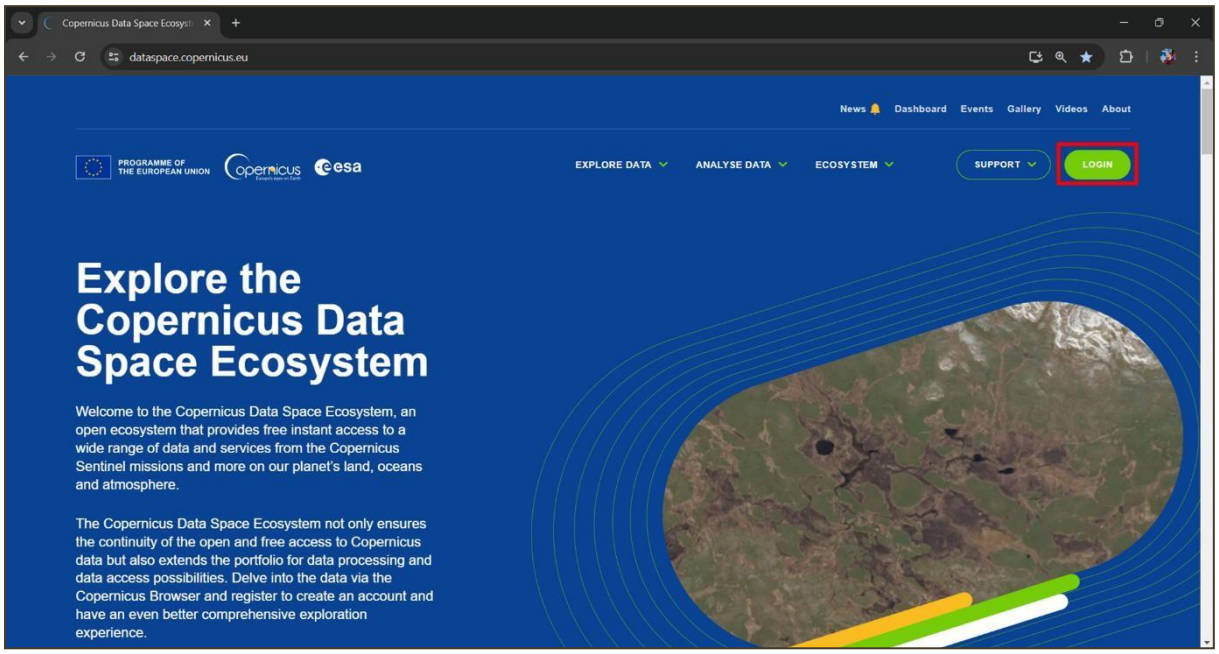


Figure 78: Copernicus Dataspace landing page with login button highlighted in red.

Next click the green 'register' button (fig.79):

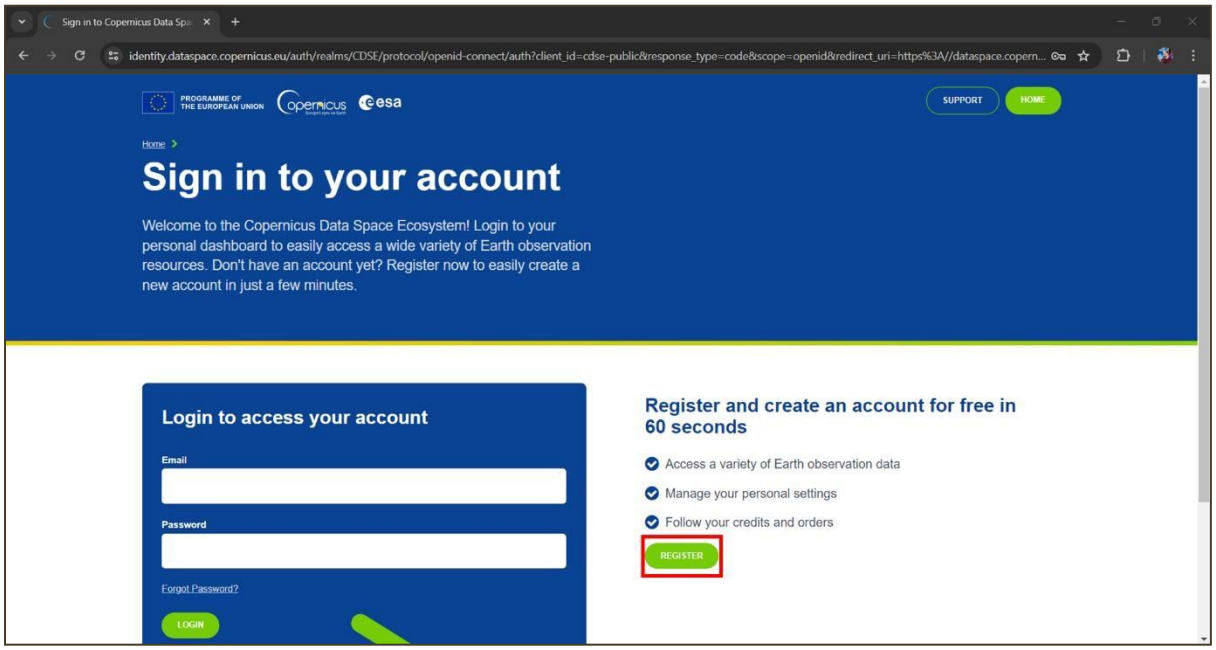


Figure 79: Copernicus Dataspace sign in page.

On the following page, fill out the application form and then at the bottom click the green 'register' button (fig. 80).

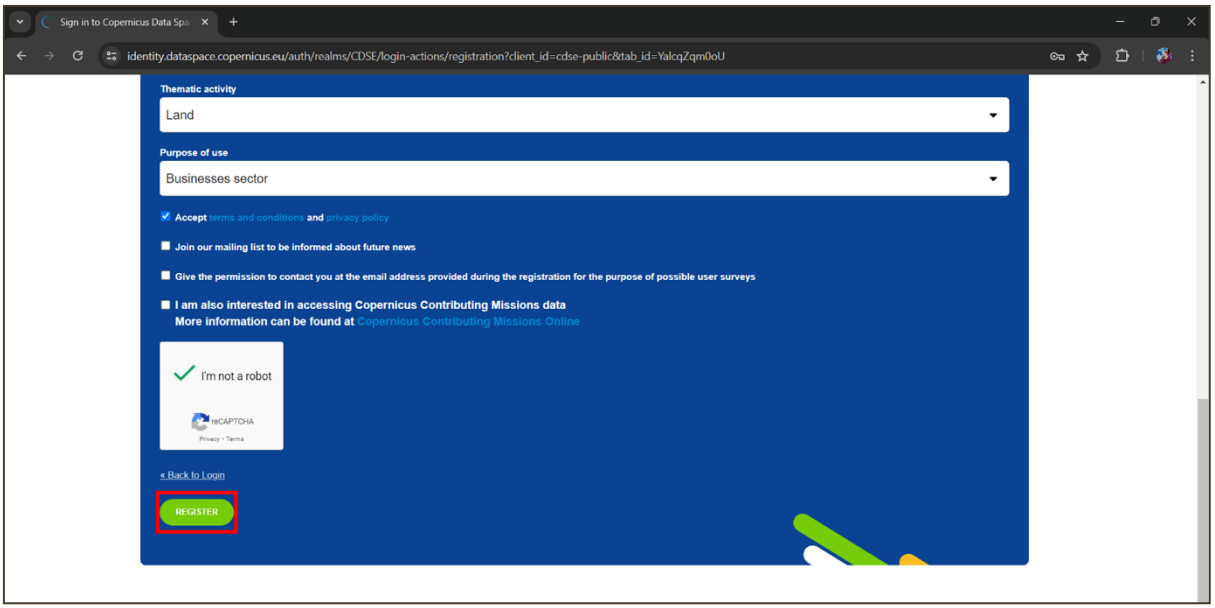


Figure 80: End of Copernicus registration page with register button highlighted in red.

Once registered, you will receive an email asking to verify your address. You can then log−in with your email and chosen password.

For any registration problems, email: help−login@dataspace.copernicus.eu

3.2.5 Running the tools in Jupyter−lab

Now that (almost) everything has been setup you can launch Jupyter Lab in the Anaconda Navigator (fig. 81). Remember as always that your project environment (here 'EGM722') should be selected and not 'base (root)'.

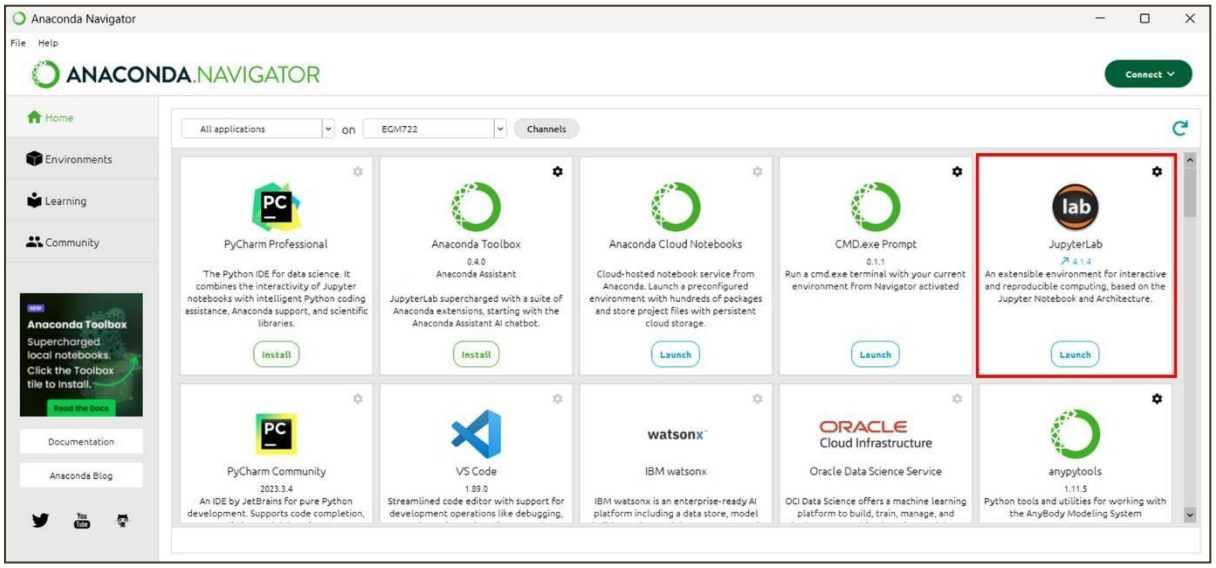


Figure 81: Location of Jupyter Lab in Anaconda Navigator highlighted in red.

Once Jupyter lab opens, you should see the three tools on the left (fig. 82).

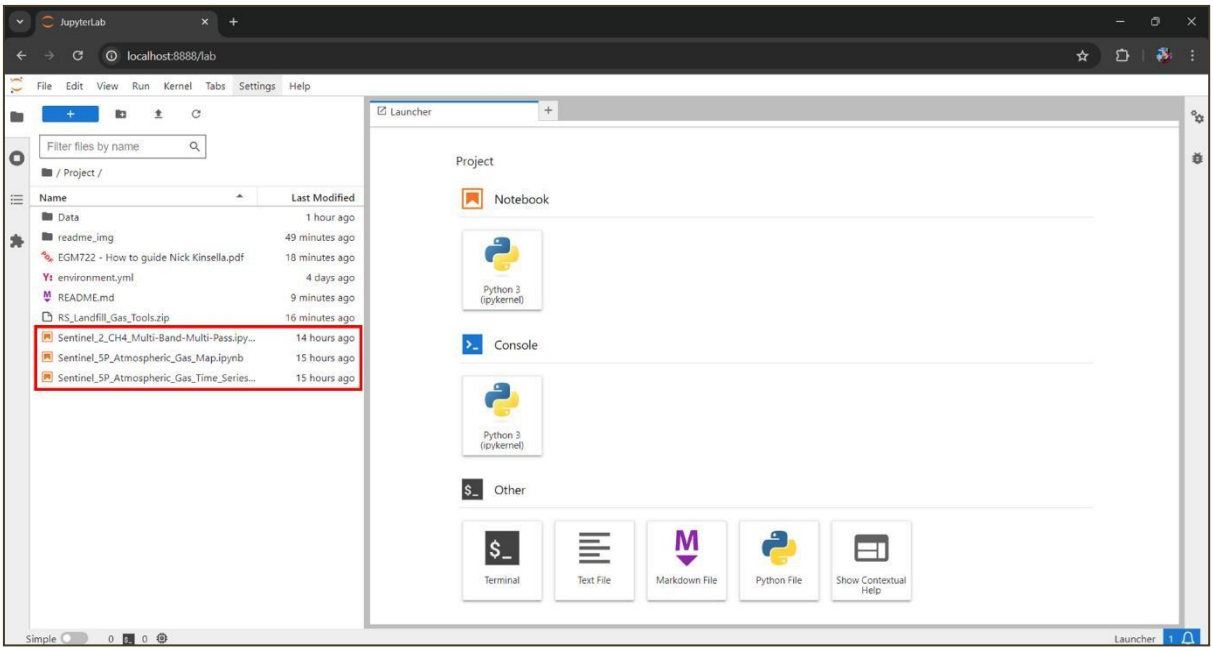


Figure 82: Location of tool scripts in Jupyter lab highlighted in red.

Click on one of the tools to open it. You can then follow the instructions, running the code by clicking the play button (fig. 83).

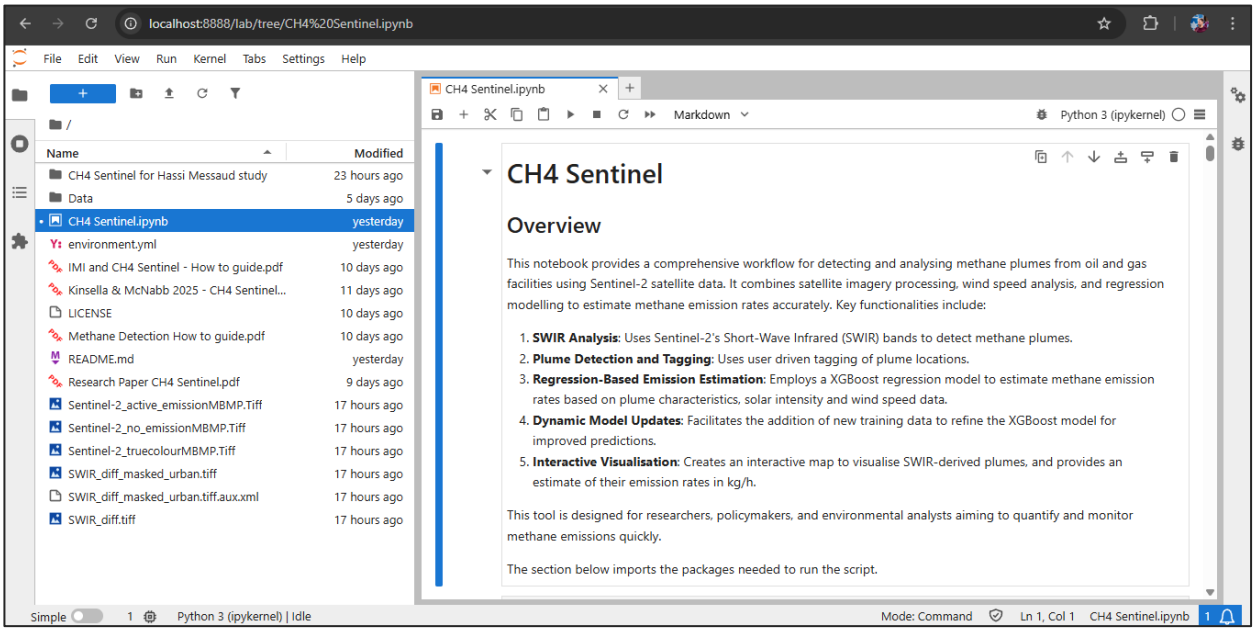


Figure 83: Location of the 'play' button which runs each of the code segments of the workbooks.

3.2.8 Authentication with OpenEO

The very first time one of the tools are run, the following section of code…

```
connection = openeo.connect(url="openeo.dataspace.copernicus.eu")
connection.authenticate_oidc()
```

… will provide you with a URL that will look something like this:

```
Visit https://auth.example.com/device?user_code=EAXD-RQXV to authenticate.
```

Copy this into your web browser and login using the Copernicus Data Space. Once this is complete, run tool's Python script again and it will receive an authentication token, printing the message:

```
Authorized successfully.
```

In future you may be prompted with a new URL to create a new authentication token, whereby you should repeat the steps of this section.

3.2.9 Installing CDS API

The Climate Data Store (CDS) Application Program Interface (API) will provide necessary atmospheric wind information that will enable the software to determine plume flow rate, enabling us to calculate the emission rate, known as 'gas flux'. Like the OpenEO API, this has also already been installed with the environment.yml file. To use it, first we need to register on the European Centre for Medium− Range Weather Forecasts website. In a browser, navigate to https://www.ecmwf.int/ and click 'Log in' (fig. 84).

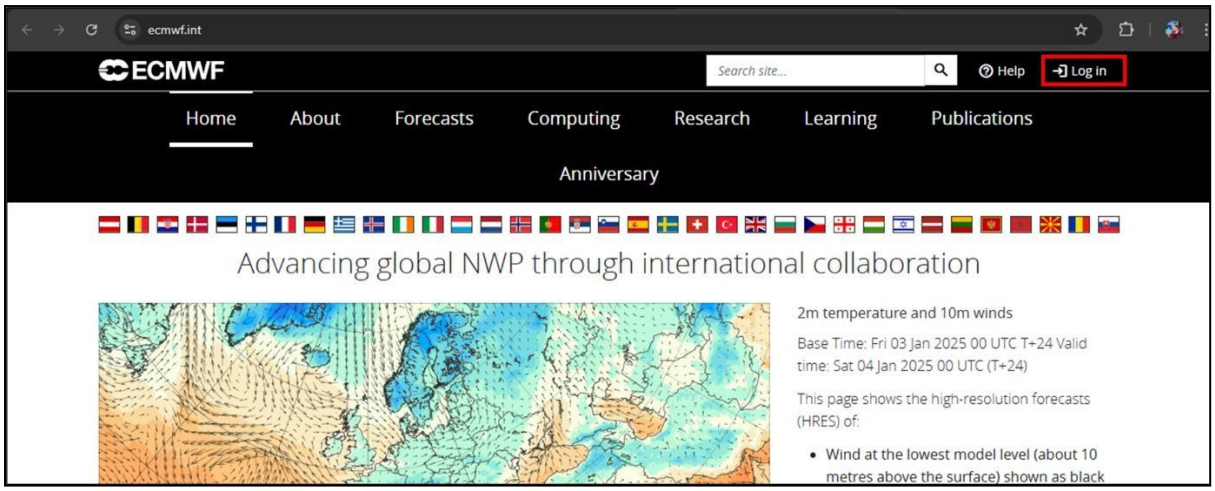


Figure 84: Login button for Copernicus Climate Data Store

Next click register (fig. 85).

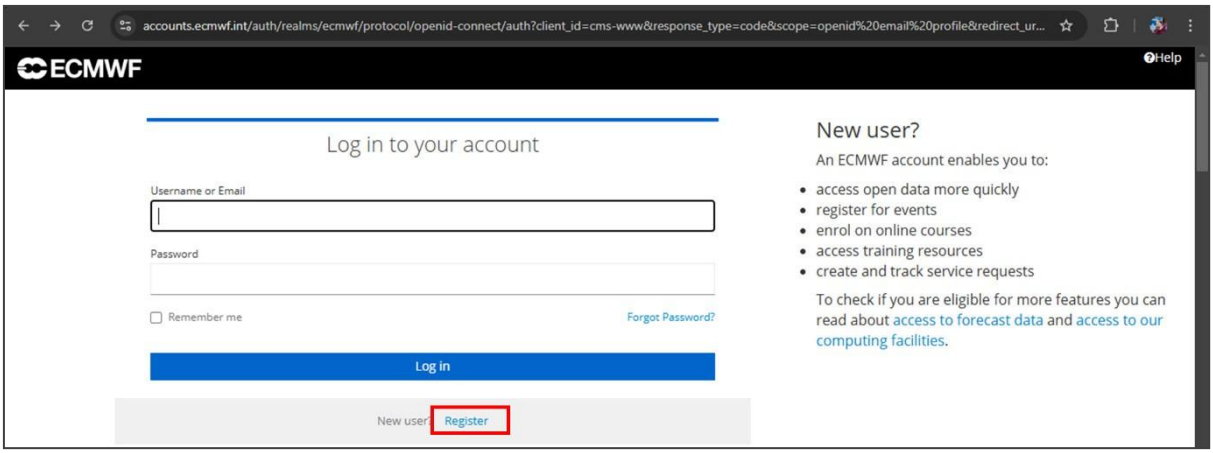


Figure 85: registration button on ECMWF website.

In the next screen, enter your details and click 'register at the bottom of the form.

After you will be prompted to confirm your registration email address. Simply open your email and click the link provided. This needs to be done within 15 minutes or the link will expire (fig. 86).

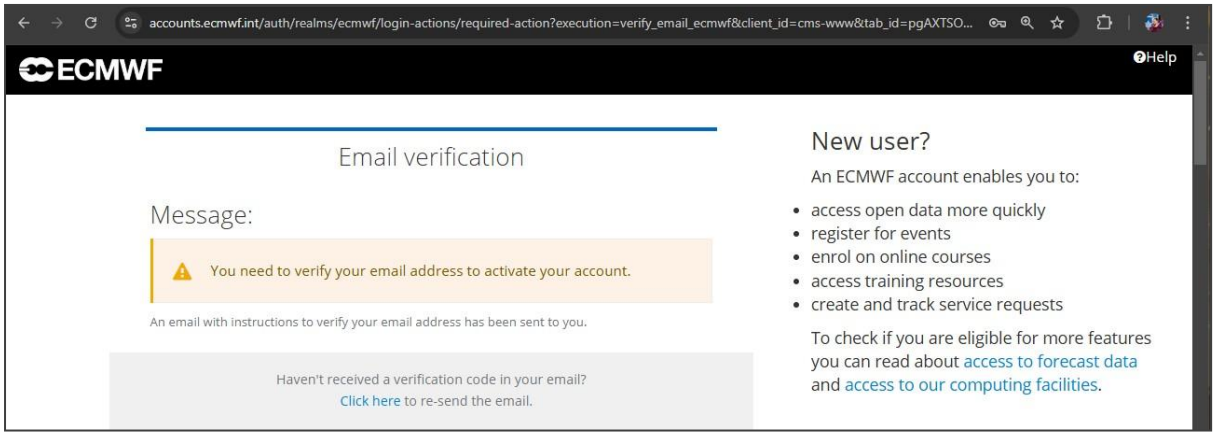


Figure 86: registration button on ECMWF website.

The credentials we created are going to be used on a different website, the Copernicus climate data store. In a web browser, navigate to the following website: https://cds.climate.copernicus.eu/ and click the 'Login – Register' button in the top right of the page (fig. 87)

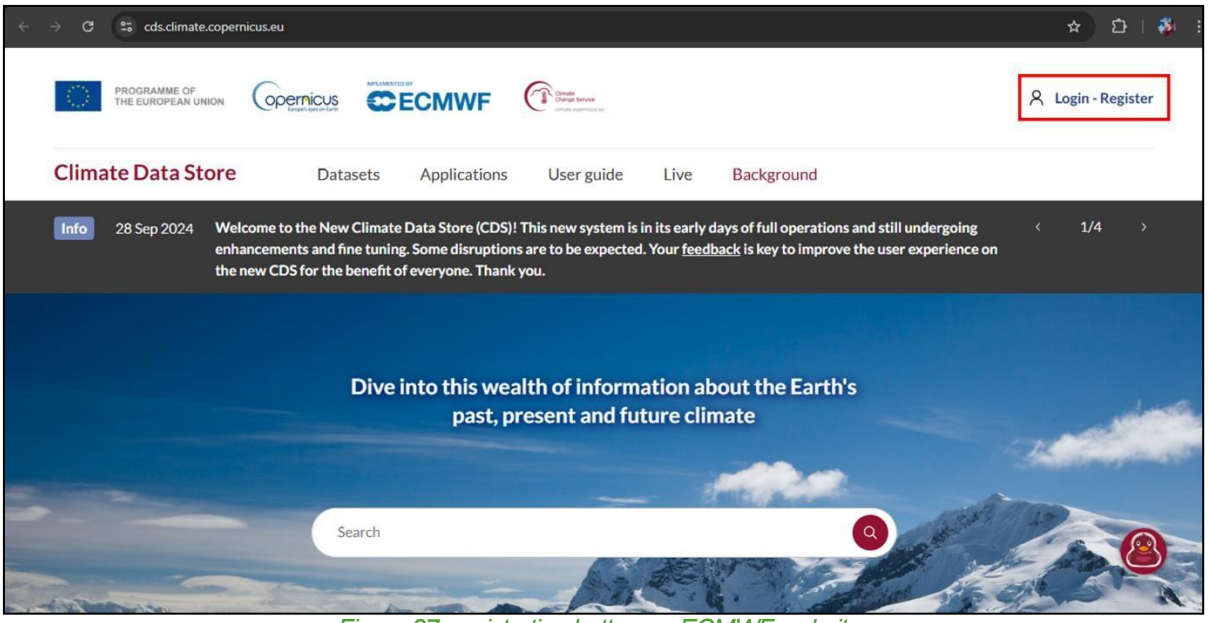


Figure 87: registration button on ECMWF website.

You will see a prompt telling you to set up the credentials on the ECMWF website that we already did a few moments ago. Click 'I understand' (fig. 88).

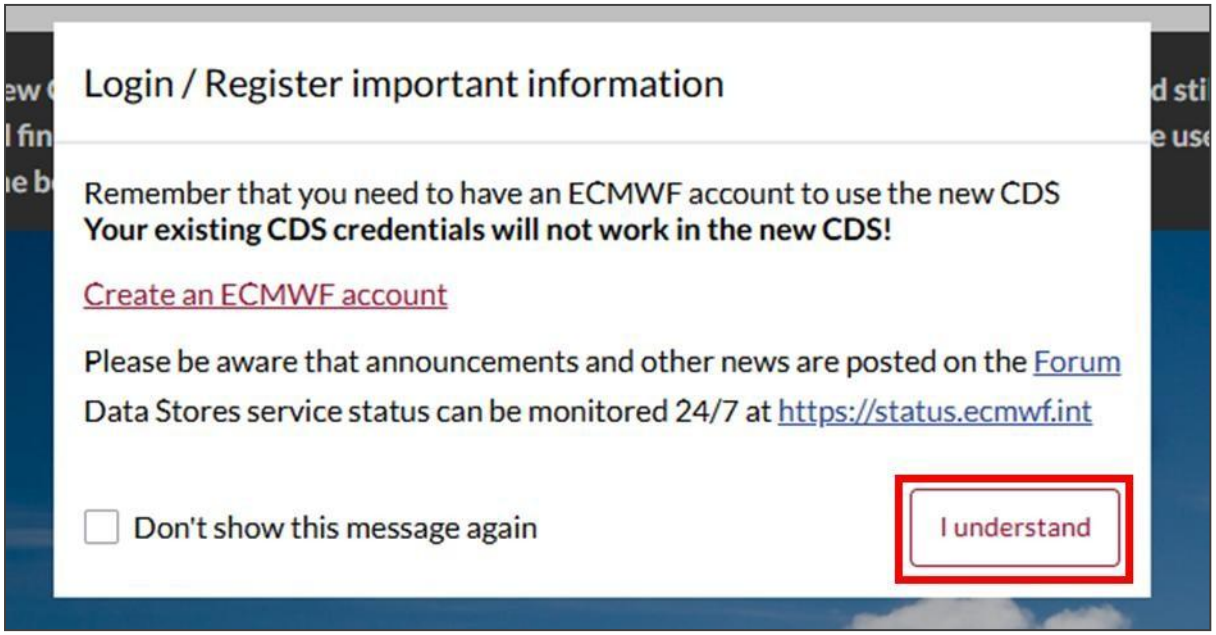


Figure 88: ECMWF registration warning on CDS website.

You should find that you are automatically logged into the CDS website (fig. 89).

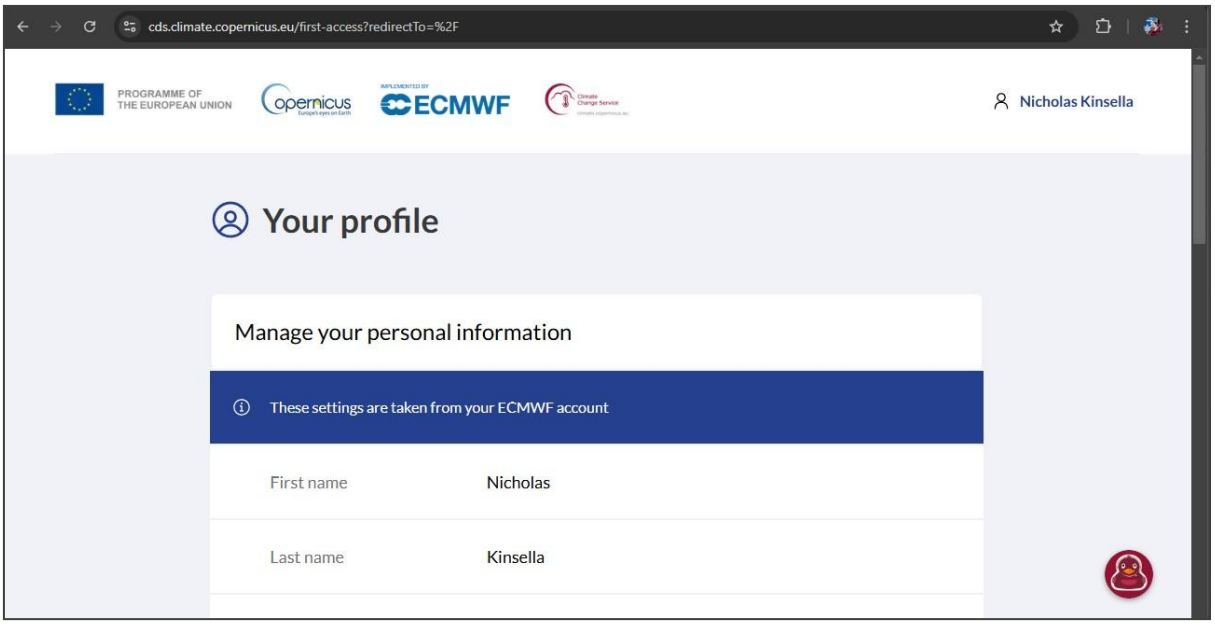


Figure 88: ECMWF registration warning on CDS website.

To fully activate the account you will need to fill in additional information further down, agree to the data protection and privacy statement, the terms of use, and finally and click the 'activate your profile' button at the bottom of the page (fig. 89).

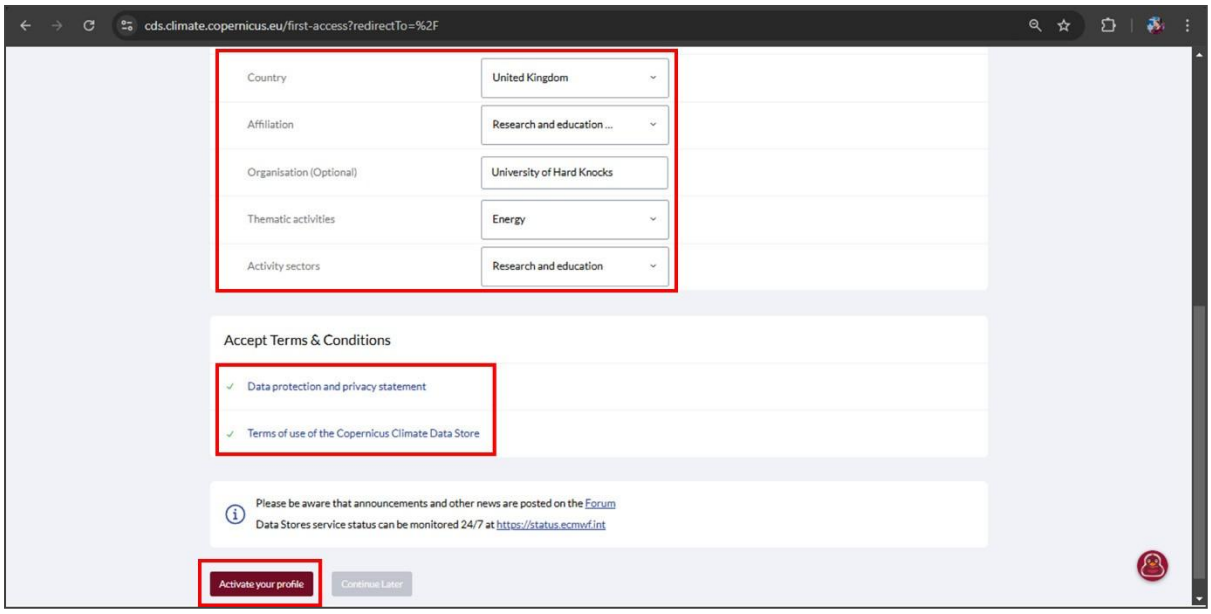


Figure 89: CDS website profile activation.

Open Anaconda Navigator, select your environment from the dropdown menu as always and launch a Command.exe prompt (fig. 90)

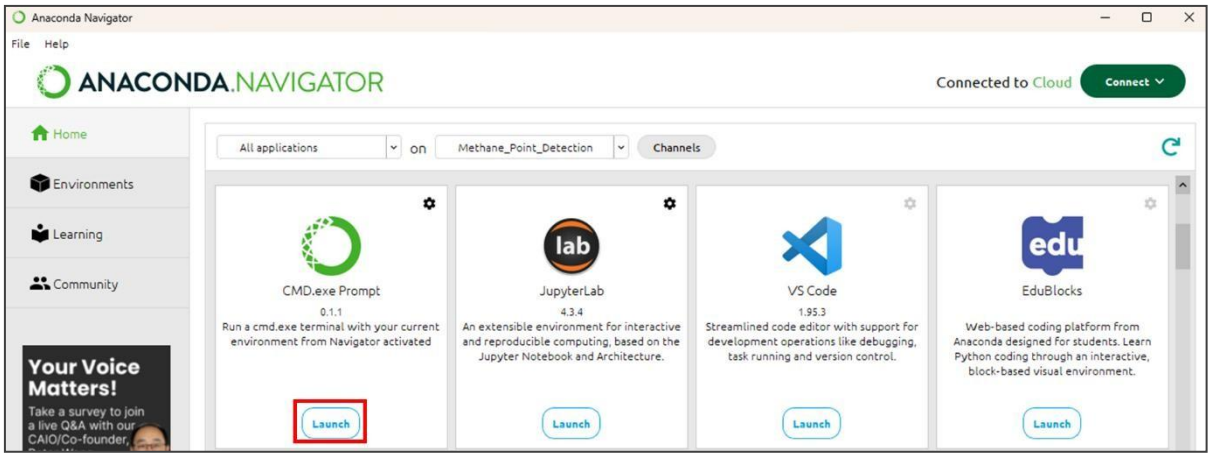


Figure 90: Anaconda Navigator home screen with CMD.exe launch button highlighted.

Enter the command 'conda install cdsapi' without quotes. This will install the CDS API (fig. 91)

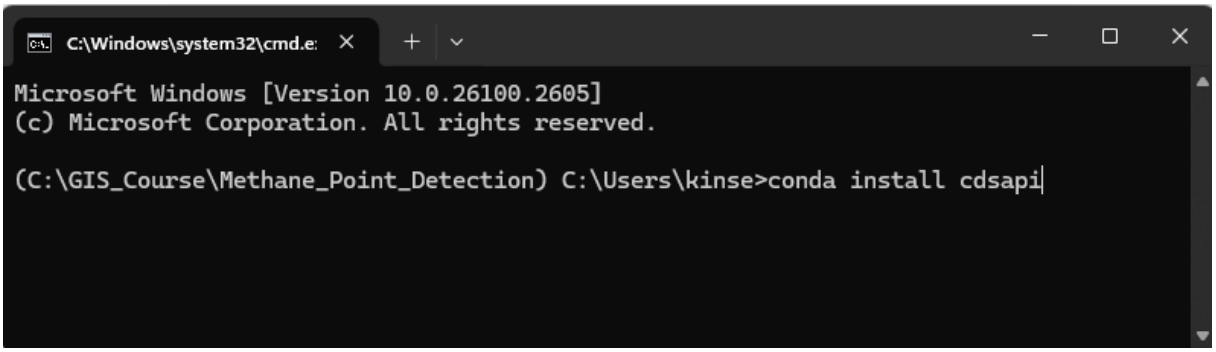


Figure 91: conda install cdsapi command in CMD.exe prompt.

The preparation will take a couple of minutes to complete. Once it is completed, you will be prompted if you would like to proceed with the installation, type 'y' and hit enter. The API will begin installing (fig. 92).

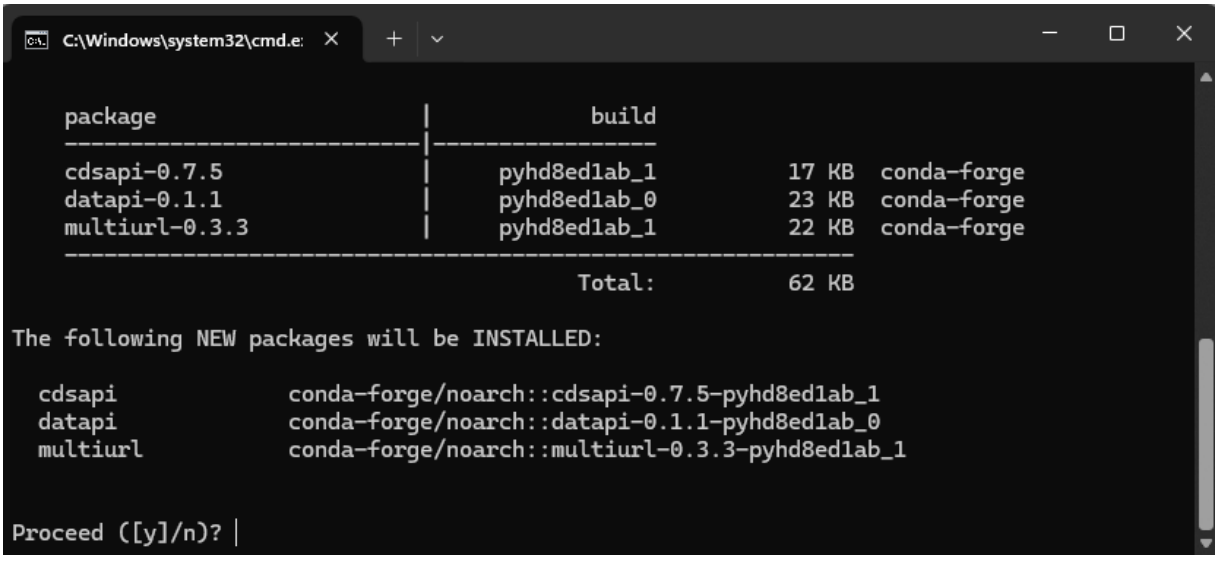


Figure 92: proceed with installation screen in CMD.exe prompt.

Once it is complete, you can close the CMD.exe window.

Before using the CDS API we need to set up the log–in credentials. Navigate the following page https://cds.climate.copernicus.eu/how–to–api and scroll down to the section called 'Setup the CDS API personal access token'. There you will see a URL and an access token in a grey box (fig. 93)

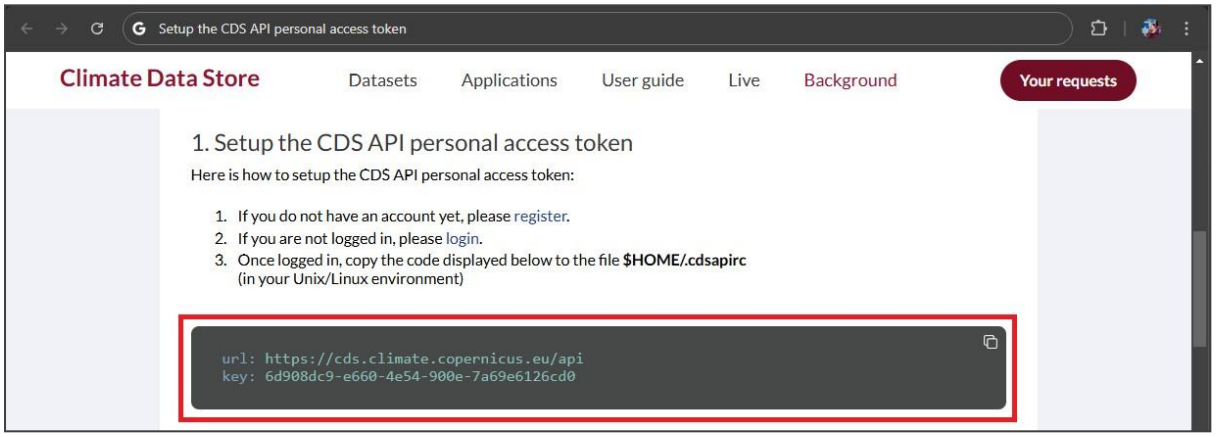


Figure 92: proceed with installation screen in CMD.exe prompt.

As the instructions on the page say, these need to be saved in a file called .cdsapirc in your home directory, for example: 'C:\Users\<YourUsername>\.cdsapirc'. It is important that the file has no extension like .txt. Microsoft notepad unfortunately adds an .txt extension to files by default, even if you don't specify this is what you want to do. Therefore a free program like VS Code should be used instead.

You can download VS Code from https://code.visualstudio.com/. On the landing page, click the button that says, "download for windows" and run the downloaded VSCodeUserSetup.exe file to install the software, accepting all the default options (fig. 93).

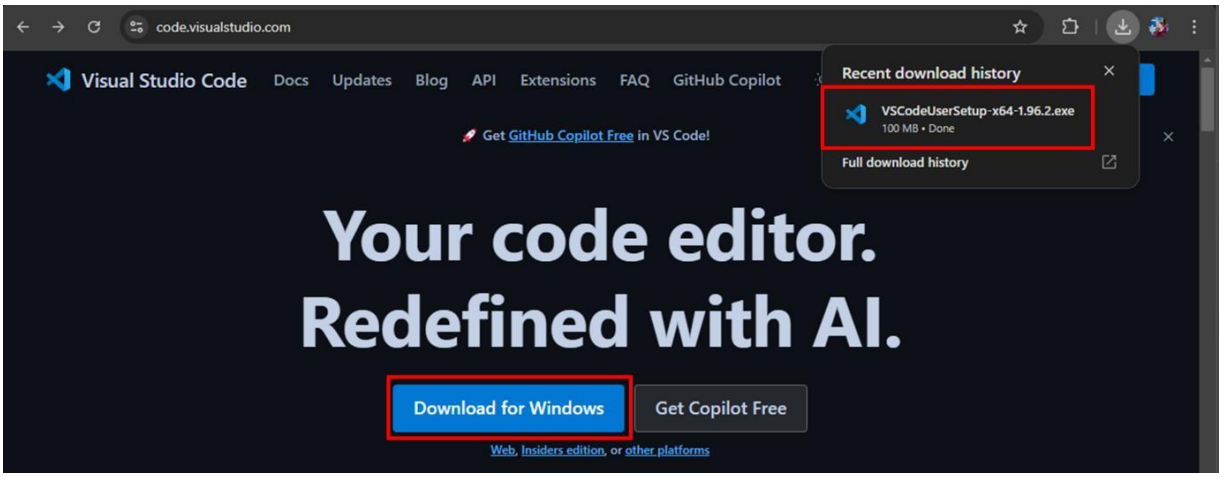


Figure 93: VS Code landing page with download button and downloaded file highlighted.

In VS Code, select file > New Text File (fig. 94)

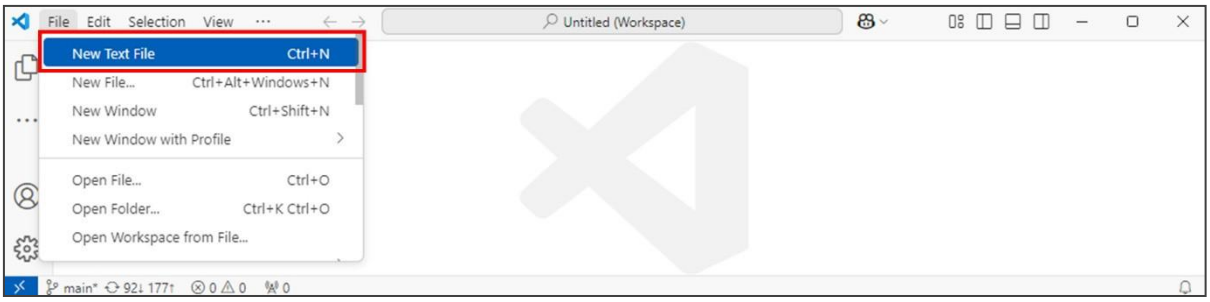


Figure 94: VS Code file > New Text File .

Then paste the credentials from the grey box as shown in figure 92. (fig. 95)

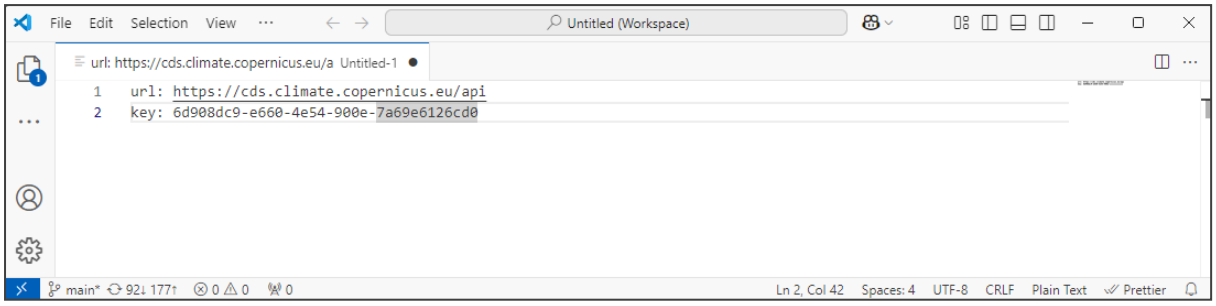


Figure 95: VS Code with CDS API Login credentials

Finally save this to 'C:\Users\<YourUsername>\' as '.cdsapirc' ensuring that 'no extension' is chosen for the file type (fig. 96).

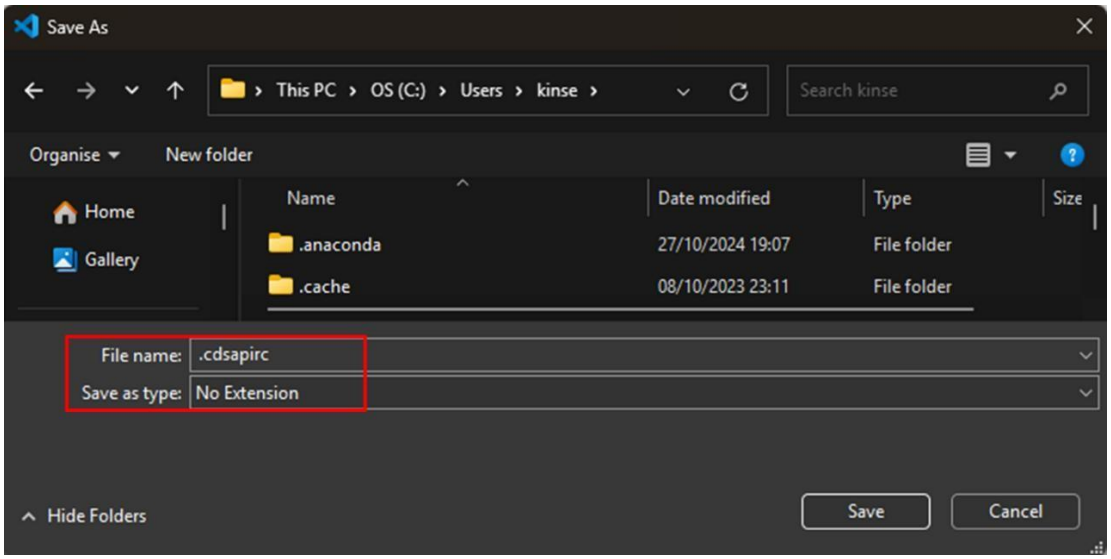


Figure 96: Saving .cdsapirc file with 'No Extension' selected

The first time you run the CH4 Sentinel code the following error will occur telling you that the required licences haven't been accepted.

```
HTTPError: 403 Client Error: Forbidden for url: https://cds.climate.copernicus.eu/api/retrieve/v1/processes/reanalysis-era5-single-levels/execution
required licences not accepted
Not all the required licences have been accepted; please visit https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels?tab=download#manage-licences to accept the required licence(s).
```

Figure 97: Initial error code when trying to access wind speed data for field.

The following page will appear. Ensure the ‘reanalysis’ box is ticked (fig. 98).

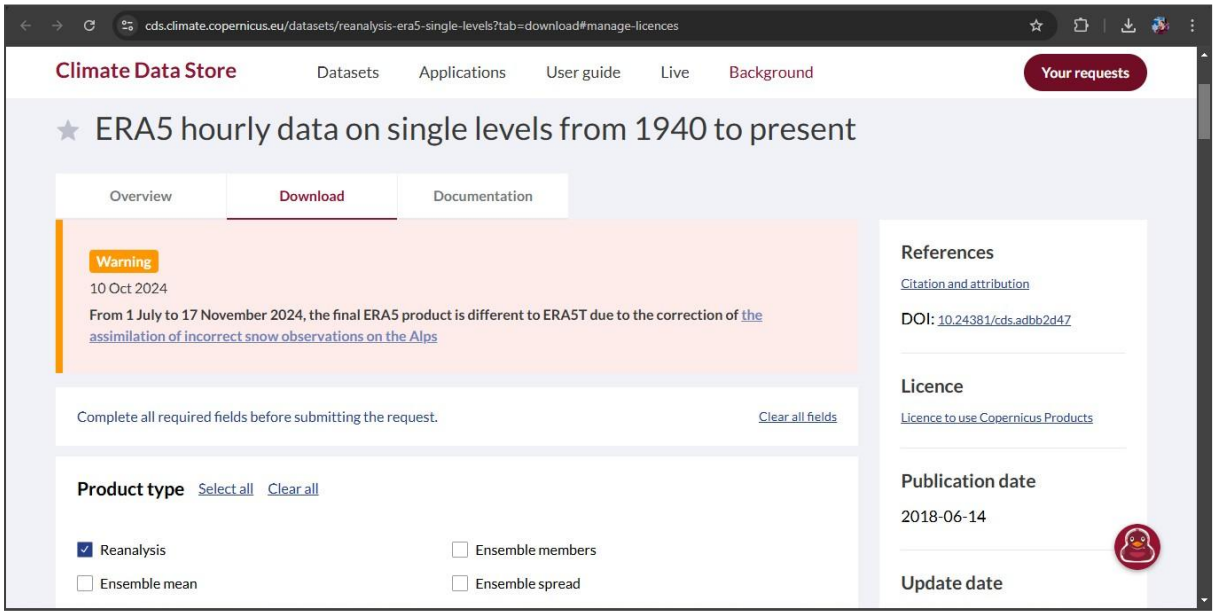


Figure 98: ERA5 licence agreement landing page

Scroll down until you see a menu item titled ‘wind’, expand the menu and select the tick boxes for ‘10m v−component of wind’, which represents north/south windspeed, and 10m ‘u−component of wind’, which represents east/west windspeed (fig. 99).

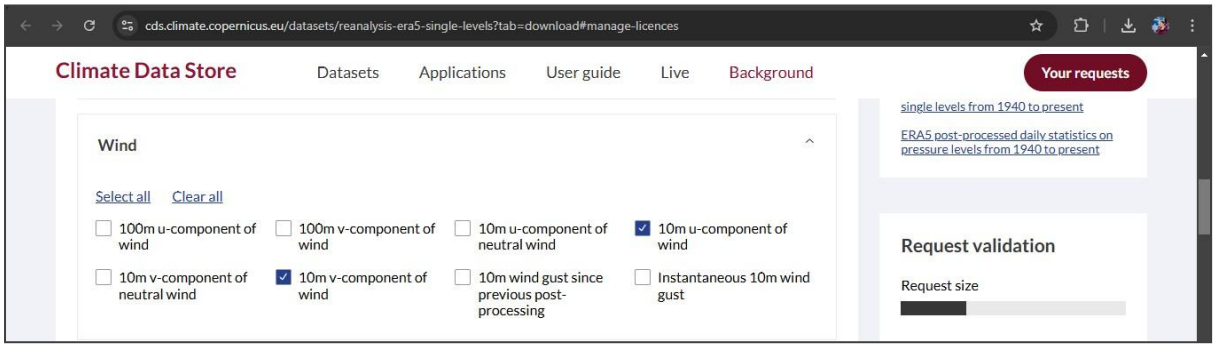


Figure 99: ERA5 licence agreement landing page wind submenu

Further down you will see the Year, Month, Day and Time expandable menus. Here you should select the date periods you are interested in (fig. 100).

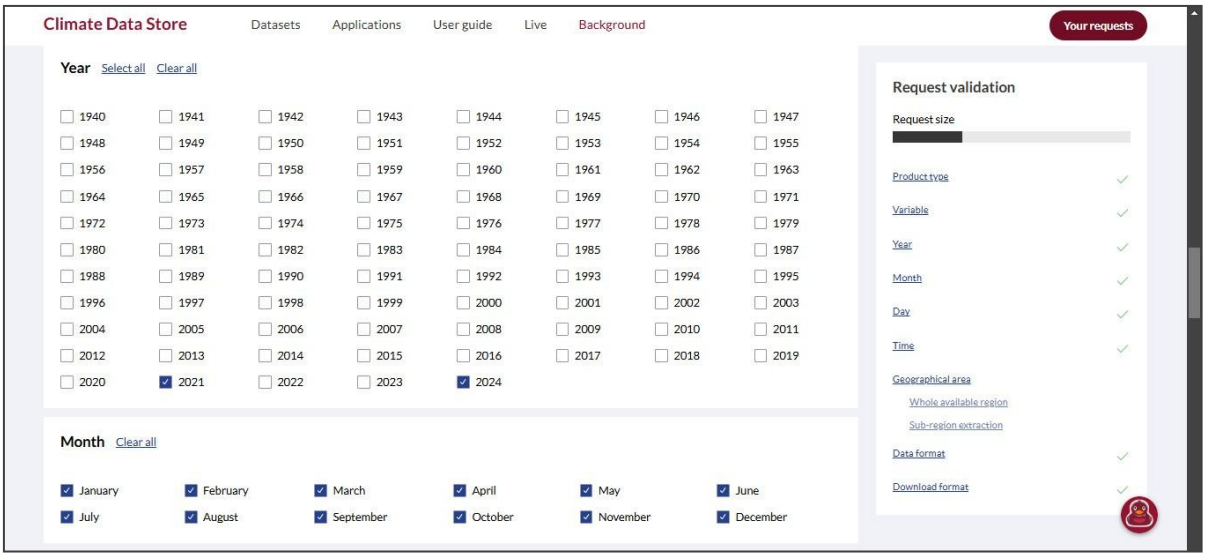


Figure 100: ERA5 licence page with Year and Month submenus. Day submenu is not shown.

Sentinel−2 operates with a sun−synchronous orbit, meaning that it passes over the earth at a similar time of day, every day, to ensure consistent lighting conditions. The overpass time is between 10:25am and 10:35am globally, so we will choose the wind conditions at 10am for our analysis (fig. 101).

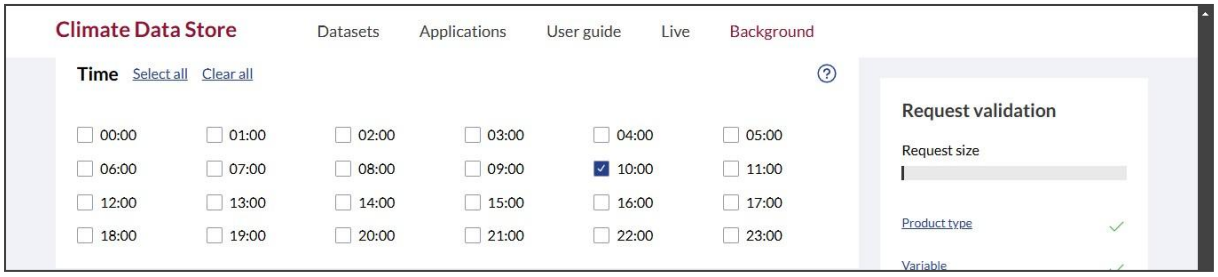


Figure 101: ERA5 licence page with time submenu.

Choose NetCDF4 for the data format and unarchived for the download format. Click accept terms (fig. 102).

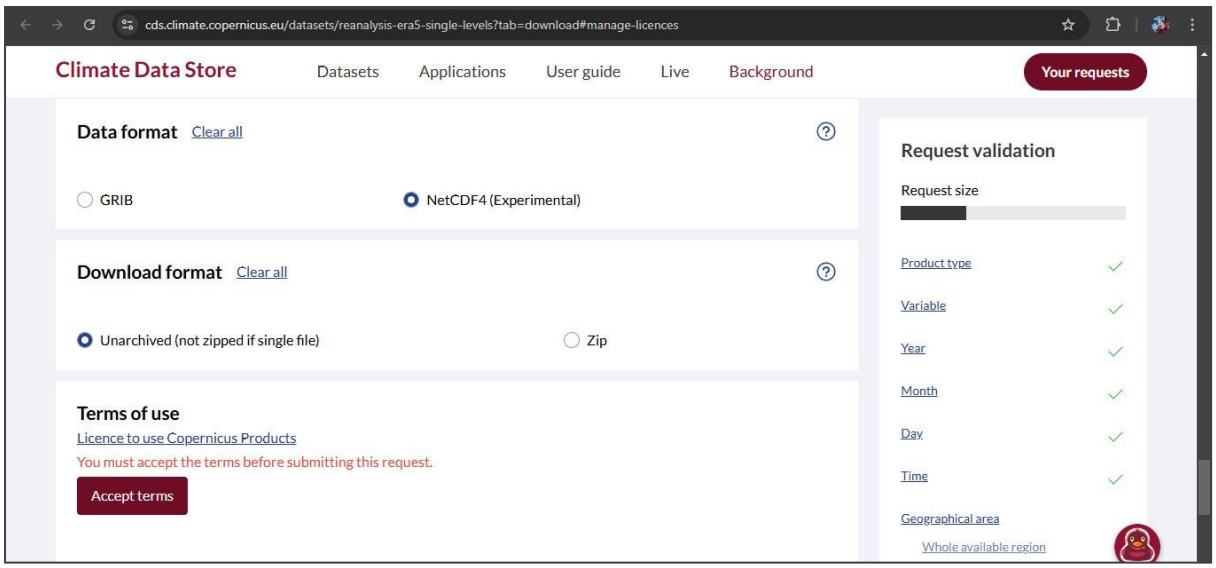


Figure 102: ERA5 licence agreement page, Data format, download format submenus and Accept Terms button.

Finally click 'Submit form' under the API request submenu (fig. 103)

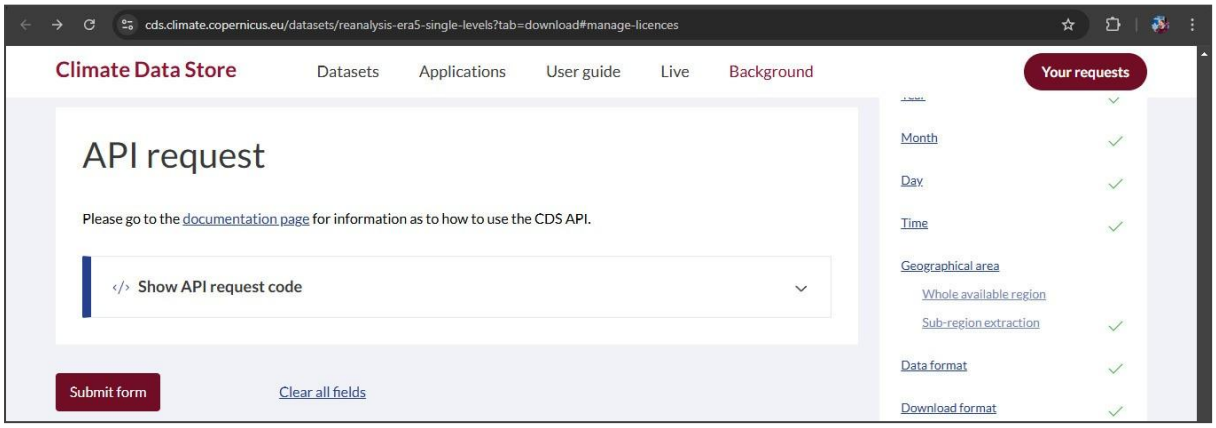


Figure 103: ERA5 licence agreement page, API request submenu and Submit form button.

3.4 CH4 Sentinel Code

Overview

This notebook provides a comprehensive workflow for detecting and analysing methane plumes from oil and gas facilities using Sentinel−2 satellite data. It combines satellite imagery processing, wind speed analysis, and regression modelling to estimate methane emission rates accurately. The tool is currently configured for Algeria but could be repurposed for other regions. Key functionalities include:

1. SWIR Analysis: Uses Sentinel−2's Short−Wave Infrared (SWIR) bands to detect methane plumes.
2. Plume Tagging: Uses user driven tagging of plume locations.
3. Regression−Based Emission Estimation: Employs a XGBoost regression model to estimate methane emission rates based on plume characteristics, solar intensity and wind speed data.
4. Dynamic Model Updates: Facilitates the addition of new training data to refine the XGBoost model for improved predictions.

E. Plume measurement: Creates an interactive map to visualise SWIR−derived plumes, and provides an estimate of their emission rates in a table in kg/h.

This tool is designed for researchers, policymakers, and environmental analysts aiming to quantify and monitor methane emissions quickly. The section below imports the packages needed to run the script.

```
# Connecting to Sentinel-2 data
import openeo

# Available Date finder imports
import requests
import pandas as pd

# SWIR and Truecolour processing imports
import numpy as np
import geopandas as gpd
import rasterio
from rasterio.features import geometry_mask
from rasterio.warp import calculate_default_transform, reproject, Resampling

# Interactive Maps and Visualisation
import folium  # For creating interactive maps
from folium import Map, LayerControl, LatLngPopup, Rectangle  # Map features and
interactions
from folium.raster_layers import ImageOverlay  # Overlay raster images on maps
from folium import FeatureGroup  # For grouping map layers
import matplotlib.pyplot as plt  # For plotting and visualisation

# Wind Speed imports
import cdsapi
from tempfile import NamedTemporaryFile
import xarray as xr

# Plume analysis imports
from scipy.ndimage import label  # For segmentation and labelling of regions
from scipy.spatial import ConvexHull  # For calculating convex hulls of shapes
from scipy.spatial.distance import pdist
from shapely.geometry import Polygon

# Imports related to predictive model
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from xgboost import XGBRegressor

# Model evaluation imports
from sklearn.metrics import mean_squared_error, r2_score
```

Code 1: Loading of dependencies for the code to run.

Connect to OpenEO

Code 2 below establishes a connection with the Copernicus openEO platform which provides a wide variety of earth observation datasets

- If this does not read as 'Authorised successfully' or 'Authenticated using refresh token', then please ensure that you have completed the setup steps as outlined in section 2.3.6 of the how to guide.
- If you have followed the steps in section 2.3.6 correctly and the problem persists, please look at https://dataspace.copernicus.eu/news for any information about service interruptions.
- If there is no news of service problems you can raise a ticket here: https://helpcenter.dataspace.copernicus.eu/hc/en-gb/requests/new

```
connection = openeo.connect(url="openeo.dataspace.copernicus.eu")
connection.authenticate_oidc()
```

Code 2: Connecting to open EO.

Choose location

This section allows you to select a specific location for analysis. Simply input your chosen centre point in decimal degrees where it says:

user_latitude = 31.73

user_longitude = 6.05

Bounding box size

Next the bounding box size determines the size of the area you want to look at. Larger bounding boxes will take longer to download. For an oil field the size of Hassi Messaoud (as featured in this tool's research paper), a buffer size of 0.3 degrees is enough to encompas the entire field. Change the following perameter to whatever seems appropiate.

buffer_degrees = 0.3

```
# Choose your location
user_latitude = 31.73
user_longitude = 6.05

# Bounding box size in degrees
buffer_degrees = 0.3

def get_spatial_extent(user_latitude, user_longitude, buffer_degrees):
    return {
        "west": user_longitude - buffer_degrees,
        "south": user_latitude - buffer_degrees,
        "east": user_longitude + buffer_degrees,
        "north": user_latitude + buffer_degrees
    }

# Reverse geocode to get a location name
geolocator = Nominatim(user_agent="my_geocoder")
location = geolocator.reverse((user_latitude, user_longitude), exactly_one=True)
place_name = location.address if location else "Unknown location"

# Generate the spatial extent
spatial_extent = get_spatial_extent(user_latitude, user_longitude, buffer_degrees)

# Confirmation message
print(f"Analysis area set around {place_name}")
print(f"Spatial Extent: {spatial_extent}")
```

Code 3: Code for selecting location for analysis.

Multi−Band Multi−Pass Analysis

The method for CH_4 plume visualisation was taken from Varon et al. (2021a) who outlined three connected approaches for detecting CH_4 columns using Sentinel−2 data: The first, Single−Band Multi−Pass (SBMP), utilised the Short−Wave Infrared band 12 (SWIR−2) on a day with a known emission, and then compared it to a day when there was no emission. Second was a Multi−Band Single−Pass method (MBSP) which took the Short−Wave Infrared band 11 (SWIR−1) and SWIR−2 readings for the same day, and extracted the CH_4 plume by least squares fitting, and then differencing the band datasets. The final method was a Multi−Band Multi−Pass method (MBMP), which effectively combines the two previous approaches by differencing SWIR−1 and SWIR−2 for a day with a known emission and then again for a day with the emission absent. The MBMP method produced the best results with the least emission artefacts and so was chosen for this study (Varon et al., 2021a).

Scenes with a maximum of E% cloud cover were chosen for both the non−emission scene and active−emission scenes. No emission and active emission scenes were processed using the multi−band−single−pass equation as outlined in Varon et al. (2021a):

$$MBSP = \frac{B12 - B11}{B11}$$

Where:

- B12 is the Sentinel−2 SWIR−2 band.
- B11 is the Sentinel−2 SWIR−1 band.

Once active emission and no emission scenes have been calculated, the following equation is used to calculate the multi−band−multi−pass raster.

$$MBMP = ActiveMBSP - NoMBSP$$

Where:

- ActiveMBSP is the multiband single pass for the active emission scene
- NoMBSP is the multiband single pass for the no emission scene.

The active emission scene and no emission scene are considered in this analysis to be one satellite pass apart unless there is a large amount of interference from features such as clouds or other plumes, in which case an earlier date should be selected.

A final step in this analysis that has been added to scale the MBMP dataset mean to zero and all other valid pixel values by that amount. This has been done to account for seasonal variations in solar radiation levels that may affect the measurements of this tool.

To begin this process we need to determine what days have available satellite data.

Available dates for the analysis.

Sentinel 2 provides data approximately once every 2 – 3 days, so not every date you can input is valid. The code below will tell you what dates are available to use for the oil/gas field of your choice.

The one parameter you need to modify before running the code is:

- temporal_extent = ["2020−01−01", "2020−01−31"] (change this to your chosen date range using "YYYY−MM−DD" format.)

Once you have done this run code box E and the available dates should appear below in a matter of seconds. This section of code was provided by Sonneveld, E. (2024).

```
# Specify the date range you want to check for available data.
temporal_extent = ["2019-07-01", "2019-12-30"]

def get_spatial_extent(user_latitude, user_longitude, buffer_degrees):
    return {
        "west": user_longitude - buffer_degrees,
        "south": user_latitude - buffer_degrees,
        "east": user_longitude + buffer_degrees,
        "north": user_latitude + buffer_degrees
    }

def fetch_available_dates(temporal_extent, user_latitude, user_longitude, buffer_degrees):
    spatial_extent = get_spatial_extent(user_latitude, user_longitude, buffer_degrees)
    catalog_url =
f"https://catalogue.dataspace.copernicus.eu/resto/api/collections/Sentinel2/search.json?box={s
patial_extent['west']}%2C{spatial_extent['south']}%2C{spatial_extent['east']}%2C{spatial_exten
t['north']}&sortParam=startDate&sortOrder=ascending&page=1&maxRecords=1000&status=ONLINE&datas
et=ESA-
DATASET&productType=L2A&startDate={temporal_extent[0]}T00%3A00%3A00Z&completionDate={temporal_
extent[1]}T00%3A00%3A00Z&cloudCover=%5B0%2C{cloud_cover}%5D"
    response = requests.get(catalog_url)
    response.raise_for_status()
    catalog = response.json()
    dates = [date.split('T')[0] for date in map(lambda x: x['properties']['startDate'],
catalog['features'])]
    return dates

cloud_cover = 5 # Specifies that only scenes with >5% cloud cover are chosen

available_dates = fetch_available_dates(temporal_extent, user_latitude, user_longitude,
buffer_degrees)
print("Available dates:", available_dates)
```

Code 4: Code for determining available dates for analysis

Choosing the "Active Emission" Date

Code 5 chooses a so−called active emission date must be chosen from one of the available datasets. This will be the chosen day we are looking for plumes.

Like before, the one parameter you need to modify before running the code is:

temporal_extent = ["2020−01−17", "2020−01−17"]

Change this to your chosen date range using "YYYY−MM−DD" format.

Please note that the temporal extent dates MUST BE IDENTICAL because we are only choosing a single date.

If you receive an error message of 'NoDataAvailable' then please check the list of available data above and try again.

```
active_temporal_extent = ["2019-11-20", "2019-11-20"] # Enter parameters for the active
emission day
sun_zenith_mean_value = None # Variable to store SZA

def active_emission(user_latitude, user_longitude, buffer_degrees, active_temporal_extent):
    def get_spatial_extent(user_latitude, user_longitude, buffer_degrees):
        return {
            "west": user_longitude - buffer_degrees,
            "south": user_latitude - buffer_degrees,
            "east": user_longitude + buffer_degrees,
            "north": user_latitude + buffer_degrees
        }
    spatial_extent = get_spatial_extent(user_latitude, user_longitude, buffer_degrees)
    active_emission = connection.load_collection(
        "SENTINEL2_L2A",
        temporal_extent=active_temporal_extent,
        spatial_extent=spatial_extent,
        bands=["B11", "B12", "sunZenithAngles", "viewZenithMean"],
    )
    filename = "Sentinel-2_active_emissionMBMP.Tiff"
    active_emission.download(filename)
    compute_mean_values(filename)
def compute_mean_values(filename):
    global sun_zenith_mean_value, view_zenith_mean_value

    with rasterio.open(filename) as dataset:
        bands = dataset.read()

        sun_zenith_mean_value = np.nanmean(bands[2])
        print(f"Mean Sun Zenith Angle: {sun_zenith_mean_value}")

active_emission(user_latitude, user_longitude, buffer_degrees, active_temporal_extent)
```

Code 5: Code for downloading active emission dataset.

Choosing the "No Emission" Date

Code 6 chooses the no emission date using the same process. This is the dataset we will compare the "Active Emission" one too. The recommended choice is the satellite overpass immediately before the "Active Emission" one.

So if your active emission day is 2020−01−17, your no emission day would be 2020−01−14

In an ideal world, the "No Emission" day should contain no emissions, but in fields with a lot of activity like Hassi Messaoud, this may not be possible. Such an instance will not cause problems in most cases. The emissions for these dates will simply appear as dark clouds on the SWIR data and can be ignored.

The one parameter you need to modify before running the code is:

temporal_extent = ["2020−01−14", "2020−01−14"]

The temporal extent dates MUST BE IDENTICAL

If you receive an error message of 'NoDataAvailable' then please check the list of available data above and try again.

```
no_temporal_extent = ["2019-10-06", "2019-10-06"] # Enter parameters for the no emission day

def no_emission(user_latitude, user_longitude, buffer_degrees, temporal_extent):
    def get_spatial_extent(user_latitude, user_longitude, buffer_degrees):
        return {
            "west": user_longitude - buffer_degrees,
            "south": user_latitude - buffer_degrees,
            "east": user_longitude + buffer_degrees,
            "north": user_latitude + buffer_degrees
        }

    spatial_extent = get_spatial_extent(user_latitude, user_longitude, buffer_degrees)

    no_emission = connection.load_collection(
        "SENTINEL2_L2A",
        temporal_extent=no_temporal_extent,
        spatial_extent=spatial_extent,
        bands=["B11", "B12"],
    )
    no_emission.download("Sentinel-2_no_emissionMBMP.Tiff")

no_emission(user_latitude, user_longitude, buffer_degrees, no_temporal_extent)
```

Code 6: Code for downloading active emission dataset.

Downloading a Background Satellite Image

Code 7 helps with locating the source of the emission by displaying a true colour satellite image of the oil/gas field that the data will be superimposed over. This will help distinguish between true emissions and visual spectrum observable clouds. No modification is needed so just click run, but this code box will take longer to run than the previous downloads.

```
""" The truecolour raster needs to be reprojected to WGS84 line up correctly with the folium
map.
The same will be done later for the SWIR dataset.
"""
def reproject_to_epsg4326(data, meta):
    target_crs = "EPSG:4326"

    # Calculate transform and metadata for the target CRS
    transform, width, height = calculate_default_transform(
        meta['crs'], target_crs, meta['width'], meta['height'], *meta['bounds']
    )

    # Update metadata for the new projection
    new_meta = meta.copy()
    new_meta.update({
        "crs": target_crs,
        "transform": transform,
        "width": width,
        "height": height,
    })

    # Prepare an array for reprojected data
    reprojected_data = []
    for i in range(meta['count']):
        # Create an empty numpy array to store the reprojected data for the band
        destination = np.empty((height, width), dtype=data[i].dtype)
        reproject(
            source=data[i],
            destination=destination,
            src_transform=meta['transform'],
            src_crs=meta['crs'],
            dst_transform=transform,
            dst_crs=target_crs,
            resampling=Resampling.nearest
        )
        reprojected_data.append(destination)

    return np.array(reprojected_data), new_meta  # Returning as numpy array instead of writing
to file


# The truecolour download uses the same date as the active_emission function.
def truecolour_image(user_latitude, user_longitude, buffer_degrees, temporal_extent):
```

```
    def get_spatial_extent(user_latitude, user_longitude, buffer_degrees):
        return {
            "west": user_longitude - buffer_degrees,
            "south": user_latitude - buffer_degrees,
            "east": user_longitude + buffer_degrees,
            "north": user_latitude + buffer_degrees
        }

    spatial_extent = get_spatial_extent(user_latitude, user_longitude, buffer_degrees)

    truecolour_image = connection.load_collection(
        "SENTINEL2_L2A",
        temporal_extent=temporal_extent,
        spatial_extent=spatial_extent,

        bands=["B02", "B03", "B04"],
    )

    # Download the true colour image
    file_path = "Sentinel-2_truecolourMBMP.Tiff"
    truecolour_image.download(file_path)

    # Read the file into memory
    with rasterio.open(file_path) as src:
        data = [src.read(i) for i in range(1, src.count + 1)]
        meta = src.meta.copy()
        meta['bounds'] = src.bounds

    # Reproject the data in memory
    reprojected_data, reprojected_meta = reproject_to_epsg4326(data, meta)

    # Return reprojected data and metadata
    return reprojected_data, reprojected_meta

# Run and store the reprojected image as a variable
temporal_extent = active_temporal_extent
reprojected_image_data, reprojected_image_meta = truecolour_image(user_latitude,
user_longitude, buffer_degrees, temporal_extent)
```

Code 7: Code for downloading true colour satellite image for data visualisation

Running Plume visualiser analysis

Code 8 will use the satellite data to display plumes above 2000kg/h in ideal conditions. Provided all the variables above have been run correctly, this next section should take moments to complete.

```
# to align the datasets on the folium map
def get_bounds(user_latitude, user_longitude, buffer_degrees):
    min_lat = user_latitude - buffer_degrees
    max_lat = user_latitude + buffer_degrees
    min_lon = user_longitude - buffer_degrees
    max_lon = user_longitude + buffer_degrees
    return [[min_lat, min_lon], [max_lat, max_lon]]

bounds = get_bounds(user_latitude, user_longitude, buffer_degrees)

# File path definitions
Active_Multiband = "Sentinel-2_active_emissionMBMP.Tiff"
No_Multiband = "Sentinel-2_no_emissionMBMP.Tiff"
output_file = "SWIR_diff.tiff"
masked_output_file = "SWIR_diff_masked_urban.tiff"
urban_geojson = r"Data\hassi_messaoud_urban.geojson"

# The main MBMP calculations begin here
with rasterio.open(Active_Multiband) as Active_img, rasterio.open(No_Multiband) as No_img:

    # These divisions convert the Sentinel-2 L1C digital numbers to reflectance data.
    Active_B11 = Active_img.read(1).astype(float) / 10000.0
    Active_B12 = Active_img.read(2).astype(float) / 10000.0
    No_B11 = No_img.read(1).astype(float) / 10000.0
    No_B12 = No_img.read(2).astype(float) / 10000.0

    #This perfoms two MBSP calculations, one for each satelite pass.
    MBSP_active = (Active_B12 - Active_B11) / Active_B11
    MBSP_no = (No_B12 - No_B11) / No_B11

    #This perfoms the MBMP calculation.
    SWIR_diff = MBSP_active - MBSP_no

# Reproject and save SWIR_diff to EPSG:4326 for the folium map
with rasterio.open(Active_Multiband) as src:
    target_crs = "EPSG:4326"
    transform, width, height = calculate_default_transform(
        src.crs, target_crs, src.width, src.height, *src.bounds
    )
    meta = src.meta.copy()
    meta.update({
        "crs": target_crs,
        "transform": transform,
        "width": width,
        "height": height,
```

```
        "count": 1,
        "dtype": SWIR_diff.dtype
    })
    with rasterio.open(output_file, "w", **meta) as dest:
        reproject(
            source=SWIR_diff,
            destination=rasterio.band(dest, 1),
            src_transform=src.transform,
            src_crs=src.crs,
            dst_transform=transform,
            dst_crs=target_crs,
            resampling=Resampling.nearest
        )
"""
Urban areas proved to be a problem whne segmenting the plume from the scene.
These have been masked but this is an imperfect solution as plume length is
a strong predictor of emission rate and this can be cut off by the mask.
It should be assumed that plumes that cross urban areas are underestimated.
"""
# Load GeoJSON and create urban mask
urban_areas = gpd.read_file(urban_geojson)
with rasterio.open(output_file) as src:
    urban_areas = urban_areas.to_crs(src.crs)

    # Rasterize the urban areas
    urban_mask = geometry_mask(
        [feature["geometry"] for feature in
urban_areas.to_crs(src.crs).__geo_interface__["features"]],
        out_shape=(src.height, src.width),
        transform=src.transform,
        invert=True
    )

    # Apply the urban area mask to SWIR_diff
    swir_diff = src.read(1)
    swir_diff_masked = np.where((urban_mask) | (swir_diff == -0.0), -32768, -swir_diff)
    swir_diff_masked = np.where(swir_diff_masked > 3000, -32768, swir_diff_masked)
    """
    to account for seasonality, the median value of the dataset (i.e. the background) has been
    adjusted to zero
    """

    target_median = 0.0  # Define target median value

    # Compute the current median (ignoring NoData values)
    current_median = np.median(swir_diff_masked[(swir_diff_masked > -3000) & (swir_diff_masked
< 3000)])

    # Compute shift needed
```

```
    shift_value = target_median - current_median

    # Apply shift to all valid pixels
    swir_diff_masked = np.where(
        (swir_diff_masked > -3000) & (swir_diff_masked < 3000),
        swir_diff_masked + shift_value,
        -32768
    )

    print(f"Adjusting median from {current_median} to {target_median}, shifting by
{shift_value}")

    # Save the masked SWIR_diff to a new file
    meta = src.meta.copy()
    meta.update(dtype=rasterio.float32, nodata=np.nan)
    with rasterio.open(masked_output_file, "w", **meta) as dest:
        dest.write(swir_diff_masked.astype(rasterio.float32), 1)

# Calculate centre for map using masked SWIR_diff raster bounds
with rasterio.open(masked_output_file) as src:
    map_bounds = src.bounds
    centre_lat = (map_bounds.top + map_bounds.bottom) / 2
    centre_lon = (map_bounds.left + map_bounds.right) / 2

# Create Folium map
m = Map(location=[centre_lat, centre_lon], zoom_start=10, control_scale=True)

# Use the reprojected image stored in memory instead of loading from a file
blue, green, red = reprojected_image_data[0], reprojected_image_data[1],
reprojected_image_data[2]

# this is to adjust the brightness of the truecolour image as it can be a little dark
sometimes.
brightness_factor = 0.03 # only change this number
blue = np.clip(blue * brightness_factor, 0, 255)
green = np.clip(green * brightness_factor, 0, 255)
red = np.clip(red * brightness_factor, 0, 255)

# Stack bands to create RGB image
rgb = np.dstack((red, green, blue))
rgb = rgb / rgb.max()
rgb = np.log1p(rgb)
rgb = rgb / rgb.max()

# Add true colour image overlay
with rasterio.open(masked_output_file) as src:
    swir_bounds = [[src.bounds.bottom, src.bounds.left], [src.bounds.top, src.bounds.right]]

truecolour_overlay = ImageOverlay(
```

```
    name="Truecolour",
    image=rgb,
    bounds=swir_bounds,
    opacity=1,  # Lower opacity for blending with SWIR overlay
    interactive=True,
    zindex=1,  # Lower zindex to place below SWIR overlay
)
truecolour_overlay.add_to(m)

# Load and stretch SWIR_diff for visualization
with rasterio.open(masked_output_file) as src:
    swir_bounds = [[src.bounds.bottom, src.bounds.left], [src.bounds.top, src.bounds.right]]
    swir_data = src.read(1)

    # Mask invalid data and clip negative values
    swir_data = np.ma.masked_invalid(swir_data)
    swir_data = np.ma.masked_where((swir_data <= -3000) | (swir_data >= 3000), swir_data)

    # Calculate mean and std only for valid data
    filtered_swir_data = swir_data[swir_data >= -3000]  # Ignore values below 3000
    mean = np.nanmean(filtered_swir_data)
    std = np.nanstd(filtered_swir_data)
    std_factor = 2  # Stretch factor

    # Calculate stretching bounds within the valid data range
    lower_bound = max(mean - std_factor * std, swir_data.min())
    upper_bound = min(mean + std_factor * std, swir_data.max())

    # Normalize the data to [0, 1]
    normalized_swir_data = np.clip((swir_data - lower_bound) / (upper_bound - lower_bound), 0,
1)

    # Apply colourmap
    cmap = plt.get_cmap('viridis')
    rgb_data = (cmap(normalized_swir_data.filled(0))[:, :, :3] * 255).astype(np.uint8)

# Add SWIR_diff overlay to map
swir_overlay = ImageOverlay(
    name="SWIR Data",
    image=rgb_data,
    bounds=swir_bounds,
    opacity=1,  # Adjust opacity for visibility
    interactive=True,
    zindex=2  # Ensure SWIR overlay is above other layers
)
swir_overlay.add_to(m)

"""
This creates a clickable lat long popup event on the
```

```
map that will be used for tagging the plumes
"""
LayerControl().add_to(m)
m.add_child(LatLngPopup())

# Display map
display(m)
```

Code 8: Code displaying the map

Plume tagging

Once code 8 has run, a map like the following will appear.

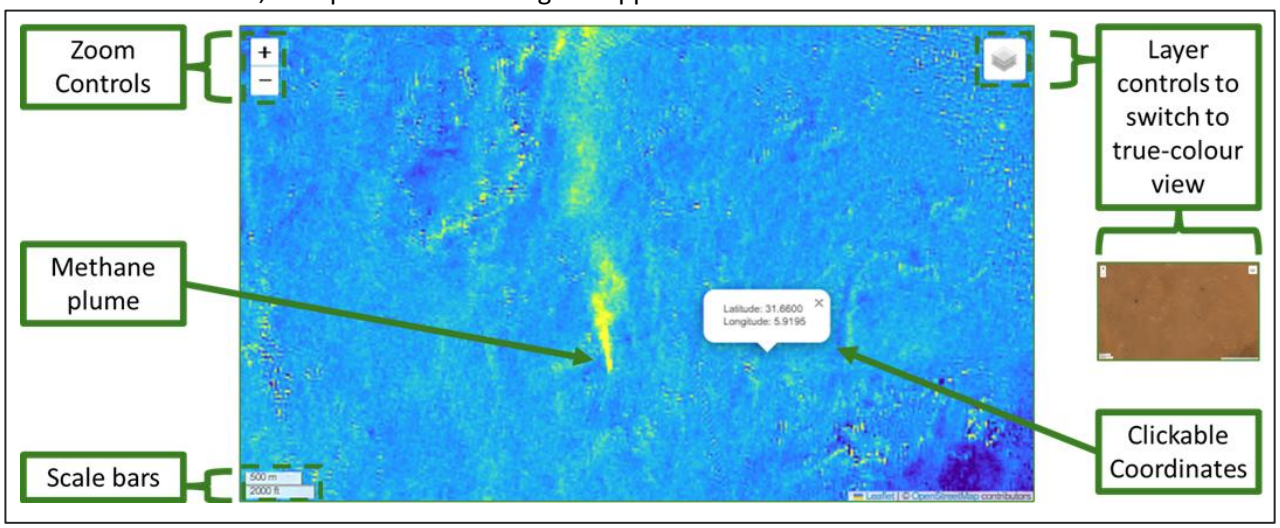


Figure 104: annotated map of code 10's plume visualiser analysis.

Over an area the size of an oil and gas field, many objects can erroneously show up as methane like signals if a method like thresholding were used. These include urban areas, agricultural irrigation projects and new constructions. Because of this, user input is required to identify which high value pixels are emissions.

CH4 Sentinel uses a tagging system whereby the user clicks the screen on an emission and copies the coordinates into a code box to run the emission rate analysis. However, due to the limitations of Sentinel-2's MSI being unable to distinguish between methane, carbon dioxide and water vapor, some expert analysis is required on the user's part. The following guide can help you identify likely candidates. Consult this tool's research paper for a full explanation.

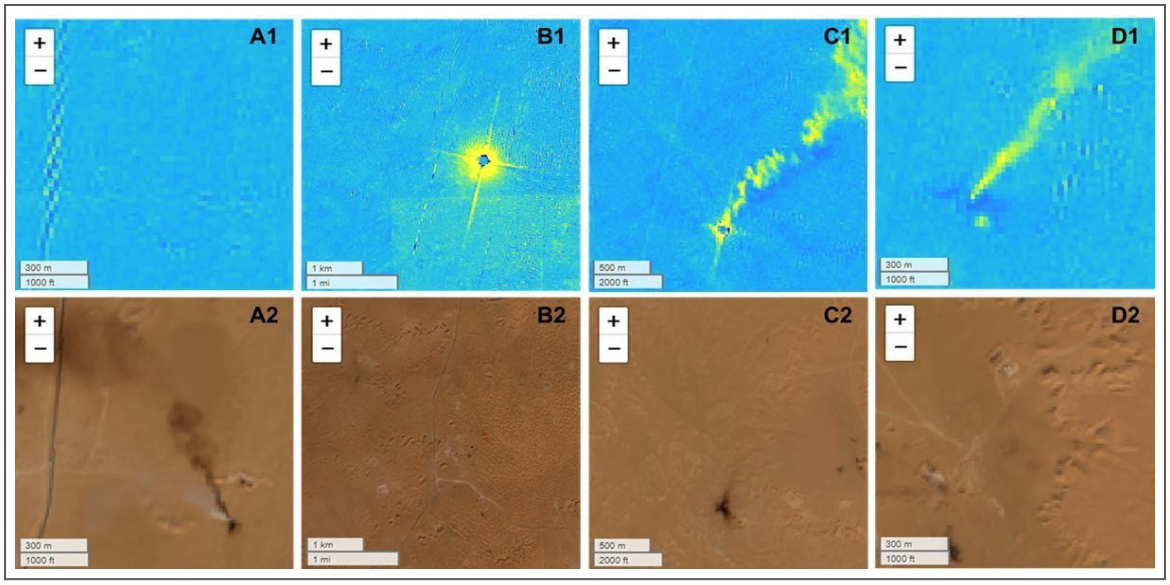


Figure 105: Emission examples. The top row shows the MBMP data and the bottom row shows the same location in true−colour. A = Non−CH_4 plume, B = Flaring with complete combustion, no CH_4 plume, C = Flaring with incomplete combustion with CH_4 plume and D = CH_4 plume with no flaring.

Each of these scenarios are explained further by the matrix in table 2.

Table 4: Plume identification matrix.

Scenario	True−colour scene	MBMP/SWIR scene	CH_4 Plume?
A	Plume visible	No plume visible	No
B	No plume visible.	Bright four−pointed star like diffraction spike	No
C	No plume visible.	Plume visible with four−pointed diffraction spike.	Yes
D	No plume visible.	Plume visible	Yes

To tag the plumes, click anywhere on a plume in the map displayed by code 9 (fig. 105), and then copy the given latitude and longitude coordinates into the code box below, using the following format:

[(31.6584, 5.9054)], # User plume 1 (latitude, longitude)

Or if the plume is segmented into more than one piece, use the following format, adding as many coordinates as needed.

[(31.6584, 5.9054), (31.6594, 5.9074)], # User plume 2 (latitude, longitude)

More lines can be added as needed. Should you wish to use any of the example lines, remove the "#" from before it as this tells CH4 Sentinel to ignore it.

```
plume_coords = [
    [(31.6584, 5.9054)],
    #[(31.7769, 5.9993), (31.7766, 5.9955)],
    #[(31.7570, 5.9423)],
]
```

Code 9: plume coordinate tags

Regression Model

A regression model is a statistical tool used to predict a dependent variable (here, methane emission rate in kg/h or "Q") based on independent variables. It works by identifying relationships in the training data and using these to estimate outcomes for new data.

To train the model, data from methane plumes were measured with the IME method outlined in Varon et al. (2018) and were measured for the following statistics:

- Plume Pixel Sum: The sum of reflectance values in the plume area.
- Plume Length: The plume's length in pixels.
- Wind Speed: ERAE reanalysis data for the wind speed at the time of observation.
- Solar Zenith Angle: The angle of the sun a given day, affecting scene brightness.

The regression analysis identifies how these factors relate to emission rates, allowing the model to predict methane emissions for other plumes based on their characteristics.

How to improve this model

The next code box connects to a .csv file that contains 70 plume measurements to train the XGBoost model. More plume measurements can be added to this if desired.

```
initial_data = pd.read_csv(r"Data\model_training_data.csv")
```

Code 10: training data for regression model.

Determining Wind Speed

Wind speed is a crucial factor in determining emission rate. Code box 11 determines the wind speed on the "Active Emission" date as an input for part of the regression analysis. Several warning messages will appear but these can be ignored.

Remember: as shown in section 3.3.9 Installing CDS API, the first time this code box is run, a "403 Client error: Forbidden for URL" error will appear. Please refer to section 3.3.9 for instructions on proceeding.

```
"""
The first function calculates a centroid for the wind speed location
using the bounding box of the study area as the ERA5 API requires a
point location.
"""
def get_location_from_site_id(user_latitude, user_longitude):
    centre_lat = user_latitude
    centre_lon = user_longitude
    return {'latitude': centre_lat, 'longitude': centre_lon}

# Get the location for the ERA5 data request
location = get_location_from_site_id(user_latitude, user_longitude)

"""
The cdsapi needs to be set up as per the instructions in the How
to Guide or this will not work!
"""
c = cdsapi.Client()

date = active_temporal_extent[0]  # this takes the date to be the same as the active_emission
function.

"""
This tool is hard coded to retrieve data for 10:00am as Sentinel-2
overpasses occur at around 10:30am If this tool is reconfigured for
another region of the world this may need to be adjusted.
"""
# Retrieve ERA5 data and store it in a temporary file
with NamedTemporaryFile(suffix='.nc') as tmp_file:
    result = c.retrieve(
        'reanalysis-era5-single-levels',
        {
            'product_type': 'reanalysis',
            'variable': ['10m_u_component_of_wind', '10m_v_component_of_wind'],
            'year': date.split('-')[0],
            'month': date.split('-')[1],
            'day': date.split('-')[2],
            'time': ['10:00'],  # Sentinel 2 overpasses are at around 10:30 am over Algeria.
            'format': 'netcdf',  # NetCDF format
            'area': [
                location['latitude'] + buffer_degrees, location['longitude'] - buffer_degrees,
                location['latitude'] - buffer_degrees, location['longitude'] + buffer_degrees,
```

```
            ],
        }
    )
    # Download data to the temporary file
    result.download(tmp_file.name)

    # Load the dataset with xarray
    ds = xr.open_dataset(tmp_file.name)

"""
ERA5 data provides wind speed in east/west (u10) and north/south (v10).
Positive u10 and v10 equals a east and north wind. Negative values are
the reverse.
"""
# Extract u and v components
u10 = ds['u10'].sel(latitude=location['latitude'], longitude=location['longitude'],
method='nearest')
v10 = ds['v10'].sel(latitude=location['latitude'], longitude=location['longitude'],
method='nearest')

"""
u10 and v10 form two sides of a right angled triangle so we can calculate
the wind speed using the A^2 + B^2 = C^2 (Pythagoras, 530 BCE). With the
wind variables this would be: u10^2 + v10^2 = windspeed^2. So this can be
reconfigured as wind speed = the squareroot of (u10² + v10²).
"""
# Wind speed calculation
wind_speed = np.sqrt(u10**2 + v10**2)

# Extract wind speed value
wind_speed_value = wind_speed.values.item()
print(f"Wind Speed at 10:00 on {date}: {wind_speed_value:.2f} m/s")
```

Code 11: code for wind speed.

Running the tagged plume analysis

Code box 12 analyses methane plumes we tagged earlier and provides the following information:

- Plume Insights: Locations, sizes, and predicted methane emission rates (kg/h) visualised on an interactive map and summarised in a table.
- Model Evaluation: Details on the regression model used to estimate emissions, including its performance metrics.

There are potentially to variables that can be adjusted should the model not identify plumes adequately. Firstly "window size" which is the size of the search box around the coordinate you selected. If a bright non−plume object is being picked up instead of the plume you clicked this can be reduced.

```
# This defines a 50 pixel by 50 pixel search box (25 pixels in each direction from the centre)
window_size = 25  <---- change this if needed
```

Secondly, for a plume to be counted as a plume, enough high value pixels need to be adjacent to one another. A cluster size of 60 may mean very small plumes are not picked up by the code. You can reduce the "min_cluster_size" to deal with this but the system will misidentify more features.

```
# Set a minimum threshold for a valid plume detection
min_cluster_size = 60  < --- change this if needed
```

```
swir_diff_path = r"SWIR_diff_masked_urban.tiff"

# Open the swir_diff_path file for analysis.
with rasterio.open(swir_diff_path) as tiff_file:
    bounds = tiff_file.bounds
    raster_data = tiff_file.read(1)  # Read the first band
    transform = tiff_file.transform

    # Define nodata_value
    nodata_value = tiff_file.nodata
    if nodata_value is None:
        nodata_value = -32768

# Converts raster data into a masked array and hides the no data pixels.
masked_data = np.ma.masked_equal(raster_data, nodata_value)

# Compute min and max from remaining pixels.
lower_bound = masked_data.min()
upper_bound = masked_data.max()
normalized_data = (masked_data - lower_bound) / (upper_bound - lower_bound)

""" calculate_plume_length determines:
    - plume_length: The longest distance between any two points in the plume (float).
    """
def calculate_plume_length(plume_pixels):
    # Compute the convex hull.
    hull = ConvexHull(plume_pixels)
    hull_points = plume_pixels[hull.vertices]
```

```
    # Compute plume length as the maximum pairwise distance between hull points.
    plume_length = pdist(hull_points).max()
    return plume_length


"""
    Using the plume_coords defined earlier, the next function:
      - Uses a 20×20 search box around each merge point in case the user doesn't quite click
the right position,
      - Identifies connected plume pixels within that box and requires that the largest
        connected group has at least min_cluster_size pixels. This is done to deal with noise
being classified as a plume.
      - Extracts the plume pixels corresponding to the most common plume label,
      - If a plume has more than one coordinate tag (the plume is segmented) then it merges
them together,
      - And returns a merged convex hull around the plume.
    """
def merge_plume_segments(labeled_array, merge_coords, transform, min_cluster_size=10):
    window_size = 25 # This defines a 20 pixel by 20 pixel search box (10 pixels in each
direction from the centre)
    merged_pixels_list = [] # empty container for the plume pixels.

    for lat, lon in merge_coords:
        # Convert lat/lon to pixel coordinates.
        row, col = rasterio.transform.rowcol(transform, lon, lat)
        row, col = int(row), int(col)

        # Define the search box boundaries.
        row_start = max(0, row - window_size)
        row_end = min(labeled_array.shape[0], row + window_size + 1)
        col_start = max(0, col - window_size)
        col_end = min(labeled_array.shape[1], col + window_size + 1)

        # Extract the local window.
        local_window = labeled_array[row_start:row_end, col_start:col_end]

        # Create a binary mask (1 for plume, 0 for background).
        binary_window = (local_window > 0).astype(int)

        # Label connected components in the local window.
        labeled_clusters, _ = label(binary_window)

        # Get the sizes of the clusters
        if labeled_clusters.size > 0:
            cluster_sizes = np.bincount(labeled_clusters.ravel())[1:]
        else:
            cluster_sizes = []

        # Check if the largest cluster meets the minimum size. Minimum cluster size is
```

```
specified at the top of this function.
        if len(cluster_sizes) == 0 or cluster_sizes.max() < min_cluster_size:
            continue

        # If a valid cluster exists, choose the plume label that appears most in the local
window.
        local_labels, counts = np.unique(local_window[local_window > 0], return_counts=True)
        if len(local_labels) == 0:
            continue
        plume_label = local_labels[np.argmax(counts)]

        # Extract all pixel coordinates for this plume label from the entire image.
        plume_pixels = np.column_stack(np.where(labeled_array == plume_label))
        merged_pixels_list.append(plume_pixels)

    if not merged_pixels_list:
        return None, None

    # Combine pixels from all merge points.
    merged_pixels = np.unique(np.vstack(merged_pixels_list), axis=0)

    # Compute the convex hull for the merged pixels.
    hull = ConvexHull(merged_pixels)
    hull_points = merged_pixels[hull.vertices]
    merged_polygon = Polygon([(pt[1], pt[0]) for pt in hull_points])

    return merged_polygon, merged_pixels

""" analyze_plume performs the main plume analysis. For each
tagged plume it will return its emission rate. It also creates the folium map and a
dataframe to show the data"""

def analyze_plume(masked_data, plume_coords, transform, initial_centre):
    # Create a folium map centered on the provided coordinates.
    noradtran_map = folium.Map(location=initial_centre, zoom_start=11, control_scale=True)

    plume_results = []  # Container for plume analysis results.

    # Identify plume regions using an absolute SWIR threshold.
    absolute_threshold = 0.009  # Pixels at or above this value are marked as plumes.
    labeled_array, _ = label(masked_data > absolute_threshold)

    # Loop through each suspected plume record (each provided as merge points).
    for i, merge_points in enumerate(plume_coords):
        try:
            # Use the first merge point as the default location.
            default_lat, default_lon = merge_points[0]
            row, col = rasterio.transform.rowcol(transform, default_lon, default_lat)
            row, col = int(row), int(col)
```

```
            # Merge plume segments using all provided merge points.
            merged_polygon, merged_pixels = merge_plume_segments(labeled_array, merge_points,
transform)
            if merged_polygon is None:
                plume_results.append({
                    "Plume": i + 1,
                    "Location": (default_lat, default_lon),
                    "Status": "No plume"
                })
                continue

            # Calculate plume length.
            plume_length = calculate_plume_length(merged_pixels)

            # Compute the centroid of the merged plume.
            centroid = merged_pixels.mean(axis=0)

            # Convert the merged polygon's pixel coordinates back to lat/lon for mapping.
            merged_polygon_latlon = []
            for x, y in merged_polygon.exterior.coords:
                lon_map, lat_map = rasterio.transform.xy(transform, int(y), int(x))
                merged_polygon_latlon.append((lat_map, lon_map))

            # Add the plume polygon to the map.
            folium.Polygon(
                locations=merged_polygon_latlon,
                color="red",
                weight=3,
                fill=False,
                popup=f"Plume {i + 1}"
            ).add_to(noradtran_map)

            # Compute the total sum of all plume pixel values in the merged region.
            plume_total_sum = np.sum(np.abs(masked_data[merged_pixels[:, 0], merged_pixels[:,
1]]))

            # Save the plume results to the container.
            plume_results.append({
                "Plume": i + 1,
                "Location": (default_lat, default_lon),
                "Length (px)": plume_length,
                "Total Sum": plume_total_sum,
                "plume_pixels": merged_pixels.tolist(),
                "Status": "Detected",
            })

        except Exception as e:
            plume_results.append({
```

```
                "Plume": i + 1,
                "Location": (default_lat, default_lon),
                "Status": f"Error: {e}"
            })

    # Return the plume statistics and the folium map.
    return plume_results, noradtran_map

""" add_swir_data_to_map loads, normalizes, and adds a Short-Wave
Infrared (SWIR) raster layer to the Folium map."""
def add_swir_data_to_map(map_object, tiff_path):
    with rasterio.open(tiff_path) as tiff_file:
        swir_data = tiff_file.read(1)
        nodata_value = -32768.0  # NaN value to be ignored.

    # Mask NaN pixels and normalise data.
    masked_data = np.ma.masked_equal(swir_data, nodata_value)
    filtered_masked_data = masked_data[masked_data >= -3000]  # Ignore values below -3000 to
deal with NaN data.
    mean, std = np.nanmean(filtered_masked_data), np.nanstd(filtered_masked_data)
    lower_bound, upper_bound = mean - std_factor * std, mean + std_factor * std
    normalized_data = (masked_data - lower_bound) / (upper_bound - lower_bound)
    normalized_data = np.clip(normalized_data, 0, 1)

    # Convert SWIR data to RGB for mapping.
    cmap = plt.get_cmap("viridis")
    swir_rgb = (cmap(normalized_data)[:, :, :3] * 255).astype(np.uint8)
    image_bounds = [[bounds.bottom, bounds.left], [bounds.top, bounds.right]]

    # Add SWIR overlay to the map.
    swir_overlay = ImageOverlay(
        name="SWIR Data",
        image=swir_rgb,
        bounds=image_bounds,
        opacity=1,  # Match Truecolour opacity
        interactive=True,
        zindex=2  # Place above Truecolour
    )
    swir_overlay.add_to(map_object)

# Convert the dataset into a DataFrame
model_df = pd.DataFrame(initial_data)

""" Update_model trains and evaluates a XGBoost regression model
to predict methane emission rates based on the QIME plume training data."""

def update_model(df):
    # feature engineering
    df["Wind_PL_ratio"] = df["Wind"] / df["PL"]
```

```
    df["PPS_times_Wind_PL"] = df["PPS"] * df["Wind_PL_ratio"]
    df["PPS_adjusted"] = df["PPS"] / np.cos(np.radians(df["SZA"]))

    # Model variables
    X = df[["PPS", "PPS_adjusted", "SZA", "Wind_PL_ratio", "PPS_times_Wind_PL"]] # independent
variables
    y = df["IME Q"] # dependent variable
    # Apply log transformation to the target
    y_log = np.log(y + 1)

    # Standardize features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Optuna optimised hyperparameters
    xgb_model = XGBRegressor(
        objective="reg:squarederror",
        n_estimators=156,
        learning_rate=0.022644961379969526,
        max_depth=9,
        min_child_weight=6,
        subsample=0.6368256402610262,
        colsample_bytree=0.7834094248951226,
        gamma=1.1110535328889372e-05,
        reg_alpha=0.22050850315842796,
        reg_lambda=0.2892449322874824,
        random_state=42
    )
    xgb_model.fit(X_scaled, y_log)

    return X_scaled, y, scaler, xgb_model

# Train the XGBoost model
X_scaled, y, scaler, xgb_model = update_model(model_df)

# Get the centre of the SWIR TIFF
def get_raster_centre(tiff_path):
    with rasterio.open(tiff_path) as tiff_file:
        bounds = tiff_file.bounds
        centre_lat = (bounds.top + bounds.bottom) / 2
        centre_lon = (bounds.left + bounds.right) / 2
    return centre_lat, centre_lon
centre_coords = get_raster_centre(swir_diff_path)

plume_analysis_results, noradtran_map = analyze_plume(masked_data, plume_coords, transform,
centre_coords)

for plume in plume_analysis_results:
    if plume["Status"] == "Detected":
```

```
        idx = plume["Plume"] - 1

        pps_adjusted = plume["Total Sum"] / np.cos(np.radians(sun_zenith_mean_value))

        input_features = pd.DataFrame([[
            plume["Total Sum"],
            pps_adjusted,  #
            sun_zenith_mean_value,
            wind_speed_value / plume["Length (px)"],
            pps_adjusted * (wind_speed_value / plume["Length (px)"])
        ]], columns=['PPS', 'PPS_adjusted', 'SZA', 'Wind_PL_ratio', 'PPS_times_Wind_PL'])

        input_features_scaled = scaler.transform(input_features)
        log_prediction = xgb_model.predict(input_features_scaled)

        plume["Predicted Emission Rate (t/h)"] = (np.exp(log_prediction[0]) - 1) / 1000
    else:
        plume["Predicted Emission Rate (t/h)"] = ""


# Results to a DataFrame
plume_df = pd.DataFrame(plume_analysis_results)
# Rename the index to "Site X" or "User X"
plume_labels = [f"Site {i + 1}" if i < 10 else f"User {i - 9}" for i in range(len(plume_df))]
plume_df.index = plume_labels

# Column renaming
plume_df = plume_df.rename(columns={
    "Length (px)": "Length (px)",
    "Total Sum": "Total Sum",
    "Predicted Emission Rate (t/h)": "Q (t/h)"
})

# Column order
column_order = ["Status", "Location", "Length (px)", "Total Sum", "Q (t/h)"]
existing_columns = [col for col in column_order if col in plume_df.columns]
plume_df = plume_df[existing_columns]

# Display DataFrame with predicted emission rates (QM)
r_squared = xgb_model.score(X_scaled, np.log(model_df["IME Q"] + 1))
print(f"Active Emission Date: {active_temporal_extent[0]}")
print(f"No Emission Date: {no_temporal_extent[0]}")
print(f"Wind Speed: {wind_speed_value:.2f} m/s")

pd.set_option("display.width", 200)  #Controls dataframe width
pd.set_option("display.max_columns", None)  # "None" puts all a plume's results on one line
# Replace NaN values with an empty string for readability
plume_df = plume_df.fillna("")

# Display the updated DataFrame
```

```
pd.options.display.float_format = '{:.2f}'.format
print(plume_df)

truecolour_overlay = ImageOverlay(
    name="Truecolour",
    image=rgb,
    bounds=swir_bounds,
    opacity=1,  # Lower opacity for blending with SWIR overlay
    interactive=True,
    zindex=1,  # Lower zindex to place below SWIR overlay
)
truecolour_overlay.add_to(noradtran_map)

# Add SWIR overlay to the map
add_swir_data_to_map(noradtran_map, swir_diff_path)

# Add a layer control to toggle map layers
LayerControl().add_to(noradtran_map)

# Display the map with updated analysis
display(noradtran_map)
```

Code 12: code for tagged plume analysis.

Once code 12 is run the analysis is complete and the information shown in figure 107 will be shown, including the estimated emission rates for the tagged plumes in t/h.

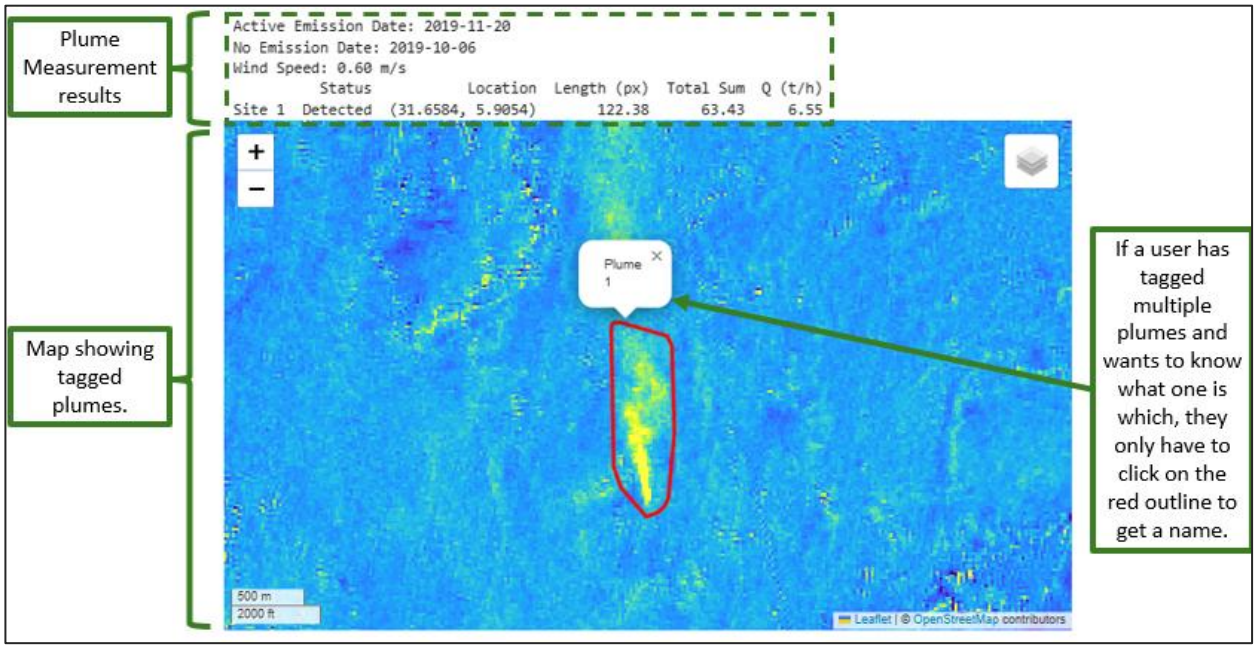


Figure 106: Results of plume analysis.

3.5 Troubleshooting

Input errors have been covered in the tool notebooks, however there are errors unrelated to a user input which can cause problems. These are detailed below.

3.5.1 Remote disconnected error

```
Error: Remote disconnected
```

Or

```
OpenEoApiError: [500] Internal: Server error: KazooTimeoutError('Connection time-out') (ref: r-2405
108830e742b59ef8ac2f28647fb5)
```

This can occur when there are issues with the Copernicus network. In the event that you see an error like this you can check page https://dataspace.copernicus.eu/news for any downtime messages and you can also contact the Copernicus dataspace team via the form at https://helpcenter.dataspace.copernicus.eu/hc/en-gb/requests/new

References

Abada, Z. and Bouharkat, M., 2018. Study of management strategy of energy resources in Algeria. *Energy Reports*, 4, pp.1−7. Available at: https://www.researchgate.net/publication/328663133_Study_of_management_strategy_of_energy_resources_in_Algeria [Accessed 18 Sep. 2024].

Dowd, E., Manning, A.J., Orth−Lashley, B., Girard, M., France, J., Fisher, R.E., Lowry, D., Lanoisellé, M., Pitt, J.R., Stanley, K.M. and O'Doherty, S., 2023. First validation of high−resolution satellite−derived methane emissions from an active gas leak in the UK. *EGUsphere*, 2023, pp.1−22.

Elkind, J., Blanton, E., Denier Van Der Gon, H., Kleinberg, R.L. and Leemhuis, A., 2020. Nowhere to hide: The implications of satellite−based methane detection for policy, industry and finance. *Columbia Center on Global Energy Policy*. Available at: https://energypolicy.columbia.edu

European Commission, 2023. EU announces €175m financial support to reduce methane emissions at COP28. *IP/23/6057*. Available at: https://ec.europa.eu/commission/presscorner/detail/en/IP_23_60E7

Ferronato, N., Torretta, V., Ragazzi, M., & Rada, E.C. (2017). Waste mismanagement in developing countries: A case study of environmental contamination. UPB Sci. Bull, 79(2), 18E−196.

Google Earth. (2024). Satellite view of Algerian petrochemical facility. Retrieved 10th of May 2024.

Integrated Methane Inversion (2024) *IMI processing steps*, Harvard University. Available at: https://imi.seas.harvard.edu/overview (Accessed: 27 November 2024).

International Energy Agency, 2022. Global Methane Tracker 2022. *IEA*, *Paris*. Available at: https://www.iea.org/reports/global−methane−tracker−2022

International Energy Agency, 2024. Global Methane Tracker 2024: Tracking pledges, targets, and action. Available at: https://www.iea.org/reports/global−methane−tracker−2024/tracking−pledges−targets−and−action [Accessed 17 Sep. 2024].

Jet Propulsion Laboratory. (2024). VISIONS: The EMIT Open Data Portal. Retrieved 10th of May 2024 from: https://tinyurl.com/2s4kkweE

Jackson, R.B., Saunois, M., Bousquet, P., Canadell, J.G., Poulter, B., Stavert, A.R., Bergamaschi, P., Niwa, Y., Segers, A., and Tsuruta, A., 2020. Increasing anthropogenic methane emissions arise equally from agricultural and fossil fuel sources. *Environmental Research Letters*, 1E(7), p.071002. Available at: https://doi.org/10.1088/1748−9326/ab9ed2 [Accessed 4 Jul. 2024].

Karimi, N., Ng, K.T.W. and Richter, A., 2021. Prediction of fugitive landfill gas hotspots using a random forest algorithm and Sentinel−2 data. *Sustainable Cities and Society*, 73, p.103097.

Liu, M., Van Der A, R., Van Weele, M., Eskes, H., Lu, X., Veefkind, P., De Laat, J., Kong, H., Wang, J., Sun, J. and Ding, J., 2021. A new divergence method to quantify methane emissions using observations of Sentinel-5P TROPOMI. *Geophysical Research Letters*, *48*(18), p.e2021GL0941E1.

Lorente, A., Borsdorff, T., Butz, A., Hasekamp, O., Aan De Brugh, J., Schneider, A., Wu, L., Hase, F., Kivi, R., Wunch, D. and Pollard, D.F., 2021. Methane retrieved from TROPOMI: Improvement of the data product and validation of the first 2 years of measurements. *Atmospheric Measurement Techniques*, *14*(1), pp.66E−684.

Naus, S., Maasakkers, J.D., Gautam, R., Omara, M., Stikker, R., Veenstra, A.K., Nathan, B., Irakulis−Loitxate, I., Guanter, L., Pandey, S. and Girard, M., 2023. Assessing the relative importance of satellite−

detected methane superemitters in quantifying total emissions for oil and gas production areas in Algeria. *Environmental Science & Technology*, E7(48), pp.19E4E−19EE6.

Malley, C.S., Borgford−Parnell, N., Haeussling, S., Howard, I.C., Lefèvre, E.N. and Kuylenstierna, J.C., 2023. A roadmap to achieve the global methane pledge. *Environmental Research: Climate*, 2(1), p.011003.

McNabb, R. (2024). Reprojecting Rasters Using Rasterio [Blog post]. Retrieved 24th of April 2024 from https://iamdonovan.github.io/teaching/egm722/practicals/raster.html

Microsoft Copilot Designer. (2024). Cover artwork. Generated on 2Eth of November 2024

OpenEO. (2024). NDVI Timeseries. [Online]. Retrieved 22nd of April 2024, from: https://documentation.dataspace.copernicus.eu/notebook−samples/openeo/NDVI_Timeseries.html

Pandey, S., van Nistelrooij, M., Maasakkers, J.D., Sutar, P., Houweling, S., Varon, D.J., Tol, P., Gains, D., Worden, J., & Aben, I. (2023). Daily detection and quantification of methane leaks using Sentinel−3: a tiered satellite observation approach with Sentinel−2 and Sentinel−Ep. Remote Sensing of Environment, 296, 113716.

Parker, R., Boesch, H., Cogan, A., Fraser, A., Feng, L., Palmer, P.I., Messerschmidt, J., Deutscher, N., Griffith, D.W., Notholt, J., & Wennberg, P.O. (2011). Methane observations from the Greenhouse Gases Observing SATellite: Comparison to ground based TCCON data and model calculations. Geophysical Research Letters, 38(1E).

Rosenthal, J., 2023. In Reverse: Natural Gas and Politics in the Maghreb and Europe. *FPRI: Foreign Policy Research Institute*. United States of America. Available at: https://coilink.org/20.E00.12E92/bf2q0b [Accessed 29 September 2024].

Sentinel Hub. (2024) "About Sentinel−2 Data." Retrieved Eth of May 2024, from: https://docs.sentinel−hub.com/api/latest/data/sentinel−2−l2a/

Sentinel Hub. (2024) "About Sentinel−EP Data." Retrieved 23rd April 2024, from: https://docs.sentinel−hub.com/api/latest/data/sentinel−Ep−l2/

Shen, L., Gautam, R., Omara, M., Zavala−Araiza, D., Maasakkers, J.D., Scarpelli, T.R., Lorente, A., Lyon, D., Sheng, J., Varon, D.J. and Nesser, H., 2022. Satellite quantification of oil and natural gas methane emissions in the US and Canada including contributions from individual basins. Atmospheric Chemistry and Physics, 22(17), pp.11203−1121E.

Sherwin, E.D., Rutherford, J.S., Zhang, Z., Chen, Y., Wetherley, E.B., Yakovlev, P.V., Berman, E.S., Jones, B.B., Cusworth, D.H., Thorpe, A.K. and Ayasse, A.K., 2024. US oil and gas system emissions from nearly one million aerial site measurements. *Nature*, *627*(8003), pp.328−334.

Sonneveld, E. (2024). Obtaining a list of available Sentinel 2 datasets for a location. Copernicus Dataspace Forum. Retrieved 2Eth of April 2024 from https://tinyurl.com/4t2trevv

Talamali, S., Chaouchi, R. and Benayad, S., 2016. Sedimentological evolution of the Lower Series formation in the southern area of the Hassi R'Mel field, Saharan Platform, Algeria. *Arabian Journal of Geosciences*, 9, p.481. DOI: 10.1007/s12E17−016−2E06−7.

United Nations Development Programme, 2022. Country Programme Document for Algeria (2023−2027). DP/DCP/DZA/4. Available at: https://www.undp.org/sites/g/files/zskgke326/files/2023−03/CPD%20Algeria%202023−2027.pdf [Accessed 17 September 2024].

Varon, D.J., Jacob, D.J., McKeever, J., Jervis, D., Durak, B.O., Xia, Y. and Huang, Y., 2018. Quantifying methane point sources from fine−scale satellite observations of atmospheric methane plumes. *Atmospheric Measurement Techniques*, 11(10), pp.E673−E686.

Varon, D. J., Jervis, D., McKeever, J., Spence, I., Gains, D., & Jacob, D. J., 2021. High−frequency monitoring of anomalous methane point sources with multispectral Sentinel−2 satellite observations. Atmospheric Measurement Techniques, 14, 2771–278E.

Varon, D.J., Jacob, D.J., Sulprizio, M., Estrada, L.A., Downs, W.B., Shen, L., Hancock, S.E., Nesser, H., Qu, Z., Penn, E., Chen, Z., Lu, X., Lorente, A., Tewari, A. and Randles, C.A., 2022. Integrated Methane Inversion (IMI 1.0): a user−friendly, cloud−based facility for inferring high−resolution methane emissions from TROPOMI satellite observations. *Geoscientific Model Development*, 1E, pp.E787–E80E. https://doi.org/10.E194/gmd−1E−E787−2022.

Vigano, I., Van Weelden, H., Holzinger, R., Keppler, F., McLeod, A., & Röckmann, T. (2008). Effect of UV radiation and temperature on the emission of methane from plant biomass and structural components. Biogeosciences, E(3), 937−947.

Wang, H., Fan, X., Jian, H. and Yan, F., 2024. Exploiting the Matched Filter to Improve the Detection of Methane Plumes with Sentinel−2 Data. *Remote Sensing*, *16*(6), p.1023.